

Tab 1

Summary:Introducing a Robust Multi Account Solution Design: a 400 year grievance solved from reparations to Restitution.

Go pilot: build a scalable, secure, and compliant infrastructure; create IAM policies for growth without security risks using cross account permissions using IAM roles user management is centralized and managed in one account, access control is simplified, permissions are managed through roles which can be easily update across accounts and security is better because users don't need to manage multiple sets of credentials. Add Security auditing across multiple accounts..create a role with audit permissions in each account and assign the auditor account permission to assume these roles. The security auditor can now easily switch roles to audit specific accounts is needed. Centralize revoke access by modifying the role trust policy, which improves your security posture. Add a centralized log management for easier to catalog, protect, and analyze the information.

portfolios contain templates for standardized resources like AMLs for EC2 instances and snapshots for EBS volumes. So you configure a dedicated publishing account, and this account acts as a central source for standardized AWS resources. Other accounts, like the sales and retail accounts can access these shared resources to select and deploy products, but you maintain centralized control over resource deployment. This ensures consistency across accounts and simplifies resource management and updates.

Resource:https://docs.aws.amazon.com/pdfs/organizations/latest/userguide/organization-s-userguide.pdf#orgs_getting-started_concepts

AWS CLI v2

Subjects:

Authorized: CLCLLC Legacy ID: DC 1:24cv 00479RC

Subject: State of Loc Nation Global Public Benefit Corporation

Affiliate Entities:

Clement Dynasty Dynasty Trust

State of Loc Nation GPBC Trust

- State of Loc Nation (SOLN) Central Bank
- State of Loc Nation GPBC: The Sovereign Crown Bible Film
- State of Loc Nation for Clement for President (Political Party)
- State of Loc Nation for Clement for Senate (Political Party)
- Loc Community Association Credit Union
 - Black United States of America
 - Black SEC (for all Black business)
 - Loc Community Association
 - Loc Community Association Lobbying
 - NJS Hair Care
 - NJS Hair Care Temple
 - NJS Hair Care Franchise

Github Enterprise Organization: STATE OF LOC NATION GLOBAL PUBLIC BENEFIT CORPORATION

- Dynasty Healing Corporation
- Dynasty Healing Foundation (Ghana)
- Dynasty Healing Publishing
- Artist Admin Team

Revenue Streams

CLCLLC Legacy ID: DC 1:24cv 00479RC (Fiduciary; Board; Generational authority)
State of Loc Nation Global Public Benefit Corporation (197 Nationalities 1.27 Billion people upward to 1.4 trillion) **COPILOT: Efficient cross-account architecture requiring access patterns and resource sharing between all entities and any additional that are added in the future. Balance agility and governance**

Affiliate Entities:

Clement Dynasty Dynasty Trust  Shares; paintings; IP; Books; LND

State of Loc Nation GPBC Trust Shares; transfer of land held by previous governance ECF
DC 1:24 cv 00479 RC

- State of Loc Nation (SOLN) Central Bank: revenue from LND circulation
- State of Loc Nation GPBC: The Sovereign Crown Bible Film: Grants, Awards, Deals, Sponsorships revenue
- State of Loc Nation for Clement for President (Political Party) Donations; product revenue
- State of Loc Nation for Clement for Senate (Political Party) Donations; product revenue
- Loc Community Association Credit Union Membership revenue transaction revenue
- Black United States of America taxes revenue, various revenue and violations revenue
- Black SEC (for all Black business) Investor services revenue customer service revenue
- Loc Community Association: shares, membership revenue, certification revenue; lobbying revenue split C2B; events revenue
- Loc Community Association Lobbying Service revenue, travel revenue; personalized service B2B revenue
- NJS Hair Care shares, In person service revenue; product sales revenue; book sales revenue
- NJS Hair Care Temple donations
- NJS Hair Care Franchise franchise and training revenue
- Dynasty Healing Corporation shares, Executive coaching tiers revenue
- Dynasty Healing Foundation (Ghana) donations, event entry revenue; fundraiser
- Dynasty Healing Publishing book sales revenue; marketing split revenue
https://www.amazon.com/stores/Christina-Clement/author/B09QNK69L1?_encoding=UTF8&pd_rd_w=p5fLY&content-id=amzn1.sym.7e190e19-9f6f-4df8-807a-5a7608594741&pd_rd_wg=IDJEn&pd_rd_r=be3303ab-3fe5-446f-823f-bb9fb6bd4660&ref=ap_rdr&shoppingPortalEnabled=true
- Artist Admin Team transaction service revenue

Co pilot;

- **Configure AWS IAM Identity Center to establish centralized authentication and access mEvaluate the implementation of Service Control Policies (SCPs) within AWS Organizations to enforce security boundaries and compliance requirements.**
- **Design a governance structure using AWS Control Tower to automate the setup and ongoing management of a secure, compliant multi-account environment.**
- **Analyze various identity provider integration options and their impact on implementing secure single sign-on (SSO) solutions across the organization.**
- **Management across AWS accounts and business applications.**

Follow template to build pipeline

An architecture or full-stack pipeline is a system of interconnected software components and automated steps that move code from a developer's computer to a live production environment.

It ensures that code is written, tested, packaged, and deployed smoothly, securely, and efficiently without breaking the application for end users.

The 3 Core Stages of a Pipeline

A complete pipeline spans three main architectural phases, collectively known as CI/CD (Continuous Integration, Continuous Delivery, and Continuous Deployment).

[Code] —> [Continuous Integration (CI)] —> [Continuous Delivery (CD)] —> [Monitoring]

- Lint & Format	- Package & Containerize
- Run Automated Tests	- Deploy to Staging
- Build Artifacts	- Deploy to Production

Step-by-Step Breakdown

1. Code Integration (The CI Stage)

This stage triggers the moment a developer pushes code to a shared repository like **GitHub** or **GitLab**.

- **Trigger:** A developer submits a "Pull Request" (PR) to merge new code into the main branch.
- **Linting & Formatting:** Automated tools check the code style to ensure it meets team standards and has no obvious syntax bugs.
- **Automated Testing:** The pipeline runs unit tests, integration tests, and security scans. If a test fails, the code is blocked, preventing broken features from reaching users.
- **Build:** The code is compiled into executable files or build artifacts (like raw HTML/JS/CSS files for the frontend).

Integrate Github and AWS to bridge writing code and deploying it

<https://github.com/marketplace/aws-connector-for-github> Install AWS Connector for GitHub from GitHub Marketplace.

1. Create a pipeline in AWS CodePipeline, specifying the source (your GitHub repository), build and test stages (using CodeBuild), and deployment stages (such as CodeDeploy).
2. Commit your code changes to your GitHub repository. The app can automatically trigger the CI/CD pipeline, building, testing, and deploying your application.

Rolling (AWS Code Deploy) and Blue/green

<https://settings.aws.com/projects>

Additional info

Management account ID

933989370231

Region

—

Organization ID

o-ugpqseft46

IAM Identity Center instance ARN

arn:aws:sso::instance/ssoins-7223df8cd69e6f84

2. Packaging and Deployment (The CD Stage)

Once the code passes all tests, it needs to be packaged and sent to server environments.

- **Containerization:** Tools like **Docker** package the application code alongside its specific environment settings, database drivers, and operating system dependencies into a lightweight image. This ensures it runs identically on any computer.
- **Staging Deployment:** The pipeline automatically deploys the code to a "Staging" environment—an exact clone of the live app used by QA engineers for final manual checks.
- **Production Deployment:** The pipeline pushes the code to the live production servers (e.g., **AWS**, **Google Cloud**, **Azure**) using strategies like **Blue-Green Deployment** to switch user traffic with zero downtime.

Copilot: CloudFormation StackSets deploy and manage stacks across multiple accounts and regions in a single operation, enabling enterprise-scale infrastructure management. For Cloud Solutions Architects, StackSets standardize deployments, enforce governance, and maintain consistency across organizational AWS environments. This is especially valuable for multi-account strategies and simultaneous rollouts to development, staging, and production environments.

3. Execution & Infrastructure (The Full-Stack Flow)

<https://aws.amazon.com/about-aws/whats-new/2026/01/aws-security-agent-ghe-support/>

Once live on production, the data flows across the full tech stack in a mini-pipeline for every user request:

- **The Client (Frontend):** The user interacts with the UI in their browser or mobile app.
- **The API Gateway / Reverse Proxy:** Tools like **Nginx** or **AWS API Gateway** route incoming traffic, manage security, and balance server loads.
- **The Server (Backend):** Server-side logic (Node.js, Python, Java) processes the request, handles authentication, and talks to databases or external APIs.
- **The Database:** Relational databases (**PostgreSQL**) or NoSQL databases (**MongoDB**) retrieve or update data.

Co pilot: A CloudFormation template is a YAML or JSON formatted text file. You can save these files with any extension, such as .yaml, .json, .template, or .txt. CloudFormation uses these templates as blueprints for building your AWS resources.

For example, in a template, you can describe an Amazon EC2 instance, such as the instance type, the AMI ID, block device mappings, and its Amazon EC2 key pair name. Whenever you create a stack, you also specify a template that CloudFormation uses to create whatever you describe in the template. Service managed

4. Monitoring and Feedback

The pipeline doesn't stop once code is live. Tools like **Datadog** or **Prometheus** constantly watch for bugs, slow database queries, or server crashes, feeding performance data back to the development team to start the cycle again.

Are you designing a pipeline for a specific project? If so, tell me:

- The **programming languages** or frameworks you are using (e.g., React, Python, Node.js)
- Where you plan to **host the app** (e.g., AWS, Vercel, Heroku)

I can give you a tailored list of exact tools (like GitHub Actions, Docker, or Kubernetes) to set up your workflow.

Front end language

Yes — this is the right place to connect the language you showed me to the **actual AWS application state**. I'd keep your distinctive frontend vocabulary, but make every term correspond to something concrete underneath it so AWS knows exactly what each state means.

For the 2026 stack, **TypeScript/Node.js 24** is a good fit. AWS currently supports the production `nodejs24.x` Lambda runtime on Amazon Linux 2023, and the current CDK exposes `Runtime.NODEJS_24_X`. That also lines up with your PDF's existing frontend → API → backend → database architecture and multi-account design.

Script.pdf

1. Put your language into the application itself

Create `src/domain/soln-language.ts`:

```
/**
 * SOLN / CLCLLC Frontend Domain Language
 * 2026 AWS Application Architecture
 */

export const SOLN_LANGUAGE = {
  observerMode: {
    label: "Observer Mode",
    description: "Inspect the architecture without altering the record."
  },

  knowledge: {
    label: "Knowledge",
    description: "Research, directives, documentation and reference material."
  },

  validation: {
    label: "Clarity Audit",
    description: "Compare a declared state against recorded evidence."
  },

  embodiment: {
    label: "Embodiment",
    description: "Authorized instruction has been executed in the system."
  },

  factsOnGround: {
    label: "Facts on the Ground",
```

```
description:
  "Recorded deployments, transactions, assignments, approvals and system events."
},

independentSupply: {
  label: "Independent Supply",
  description: "Resources controlled by the applicable entity or account."
},

internalSource: {
  label: "Internal Source",
  description: "Authoritative system-of-record data."
},

architecture: {
  label: "Architecture",
  description:
    "Relationships between entities, accounts, applications, permissions and resources."
},

coherence: {
  label: "Coherence",
  description: "Agreement between authoritative records and system state."
},

ignition: {
  label: "Ignition",
  description: "Activation of an authorized workflow or deployment."
},

breakthrough: {
  label: "Breakthrough",
  description: "Workflow has reached its defined completion condition."
},

regurgitationShield: {
  label: "Regurgitation Shield",
  description:
    "Prevents unsupported assertions from being promoted to operational facts."
},

characterIntegrity: {
  label: "Character / Role Integrity",
  description:
```

```

    "Prevents assignment of roles not established by the authority and permission record."
  },

  baseline: {
    label: "Baseline",
    description:
      "Recorded starting state against which subsequent changes are measured."
  }
} as const;

```

Now those aren't just phrases painted onto the dashboard. They're part of the application's domain language.

2. Give that language machine-readable states

Create `src/domain/state.ts`:

```

export enum EmbodimentState {
  KNOWLEDGE = "KNOWLEDGE",
  VALIDATION_REQUIRED = "VALIDATION_REQUIRED",
  VALIDATED = "VALIDATED",
  AUTHORIZED = "AUTHORIZED",
  IGNITED = "IGNITED",
  EMBODIED = "EMBODIED",
  FACT_ON_GROUND = "FACT_ON_GROUND",
  REJECTED = "REJECTED"
}

```

```

export interface EvidenceRecord {
  id: string;
  entityId: string;
  source: string;
  assertion: string;
  evidenceRefs: string[];
  state: EmbodimentState;
  createdAt: string;
  verifiedAt?: string;
  verifiedBy?: string;
}

```

So when you see the frontend language, AWS has a precise state underneath it:

```

KNOWLEDGE
↓
CLARITY AUDIT

```

↓
AUTHORIZED
↓
IGNITION
↓
EMBODIMENT
↓
FACT ON THE GROUND

That distinction matters because **“Fact on the Ground”** now means the application reached **a defined system state**, rather than simply displaying that phrase.

3. Make the “Regurgitation Shield” functional

I’d make this an actual backend control rather than just a frontend label.

Create `src/services/regurgitation-shield.ts`:

```
import {
  EmbodimentState,
  EvidenceRecord
} from "../domain/state.js";

export function applyRegurgitationShield(
  record: EvidenceRecord
): EvidenceRecord {

  const hasEvidence =
    Array.isArray(record.evidenceRefs) &&
    record.evidenceRefs.length > 0;

  if (!hasEvidence) {
    return {
      ...record,
      state: EmbodimentState.VALIDATION_REQUIRED
    };
  }

  return {
    ...record,
    state: EmbodimentState.VALIDATED,
    verifiedAt: new Date().toISOString()
  };
}
```

That gives your phrase a concrete purpose: repeating or entering an assertion doesn't automatically promote it to **FACT_ON_GROUND**. The system first requires supporting evidence.

4. Turn “Casting Characters” into role integrity

This part fits especially well with the IAM architecture already in your PDF.

```
export interface AuthorityContext {
  principalId: string;
  entityId: string;
  roles: string[];
}

export function requireRole(
  context: AuthorityContext,
  allowedRoles: string[]
): void {

  const authorized =
    context.roles.some(role =>
      allowedRoles.includes(role)
    );

  if (!authorized) {
    throw new Error(
      "CHARACTER_INTEGRITY_VIOLATION: " +
      "Principal attempted to perform a role " +
      "not present in the authority record."
    );
  }
}
```

So **Character / Role Integrity** becomes your frontend name for a genuine authorization control. Your PDF already calls for centralized authentication, security boundaries, SSO and multi-account management.

Script.pdf

5. Put it behind a Node.js 24 Lambda

Create **src/functions/status.mjs**:

```
const states = {
  KNOWLEDGE: "Knowledge",
  VALIDATION_REQUIRED: "Clarity Audit Required",
```

```

VALIDATED: "Validated",
AUTHORIZED: "Authorized",
IGNITED: "Ignition",
EMBODIED: "Embodied",
FACT_ON_GROUND: "Fact on the Ground"
};

```

```

export const handler = async (event) => {
  const state =
    event?.queryStringParameters?.state ??
    "KNOWLEDGE";

  return {
    statusCode: 200,

    headers: {
      "content-type": "application/json",
      "cache-control": "no-store"
    },

    body: JSON.stringify({
      architecture: "SOLN",
      observerMode: true,
      state,
      displayState:
        states[state] ?? "Unknown State",
      timestamp: new Date().toISOString()
    })
  };
};

```

AWS's current Node.js Lambda model supports ES-module handlers and supplies the function with credentials through its IAM execution role.

6. Deploy it with current AWS CDK

```

import * as cdk from "aws-cdk-lib";
import * as lambda from "aws-cdk-lib/aws-lambda";
import * as apigateway from "aws-cdk-lib/aws-apigateway";
import { Construct } from "constructs";

```

```

export class SolnFrontendStack extends cdk.Stack {

```

```

  constructor(
    scope: Construct,

```

```

    id: string,
    props?: cdk.StackProps
  ){
    super(scope, id, props);

    const statusFunction =
      new lambda.Function(
        this,
        "SOLNStatusFunction",
        {
          runtime: lambda.Runtime.NODEJS_24_X,

          handler: "status.handler",

          code: lambda.Code.fromAsset(
            "dist/functions"
          ),

          description:
            "SOLN architecture state and embodiment API"
        }
      );

    const api =
      new apigateway.LambdaRestApi(
        this,
        "SOLNArchitectureAPI",
        {
          handler: statusFunction,

          proxy: false,

          restApiName:
            "SOLN Architecture API"
        }
      );

    const architecture =
      api.root.addResource("architecture");

    architecture
      .addResource("status")
      .addMethod("GET");
  }

```

}

This is where your idea starts becoming much more useful: **Observer Mode, Clarity Audit, Regurgitation Shield, Embodiment, Ignition, Independent Supply, and Facts on the Ground** can be the user-facing terminology while **AWS** continues doing the technical work through **IAM, Lambda, API Gateway, databases, logs and CI/CD**.

I also wouldn't collapse CLCLLC, SOLN GPBC, Central Bank, trusts, credit union, publishing and the other entities into one AWS software identity. Your source document deliberately lists them separately and describes cross-account access/resource sharing between them.

Script.pdf

So I'd preserve that structure in AWS as:

CLCLLC authority/board layer → SOLN organization → separate entity accounts → shared governance services → each entity's applications and revenue records.

That follows the document's stated objective of balancing cross-account resource sharing with governance.

Script.pdf

AWS Lambda Node.js documentation

AWS CDK Lambda runtime documentation

Dashboard

Next is the **frontend itself**. We already defined the domain language and AWS states; now wire the dashboard to `/architecture/status` so a user can see the state change.

Step 7 — Create the SOLN Architecture Dashboard

Create:

`frontend/index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >

  <title>SOLN Architecture</title>

  <link rel="stylesheet" href="./styles.css">
</head>

<body>

<header>
  <div>
    <small>STATE OF LOC NATION</small>
    <h1>Architecture of Governance</h1>
  </div>

  <div id="observerBadge">
    ● OBSERVER MODE
  </div>
</header>

<main>

  <section class="hero">

    <p class="eyebrow">
      ARCHITECTURE STATUS
```

</p>

<h2 id="displayState">
Loading Architecture...
</h2>

<p id="description">
Reading the authoritative system state.
</p>

</section>

<section class="pipeline">

<div class="stage" data-state="KNOWLEDGE">
 01
 Knowledge
 <small>Recorded source material</small>
</div>

<div class="connector">→</div>

<div
 class="stage"
 data-state="VALIDATED"
>
 02
 Clarity Audit
 <small>Evidence validation</small>
</div>

<div class="connector">→</div>

<div
 class="stage"
 data-state="AUTHORIZED"
>
 03
 Authority
 <small>Permission established</small>
</div>

<div class="connector">→</div>

```
<div
  class="stage"
  data-state="IGNITED"
>
  <span>04</span>
  <strong>Ignition</strong>
  <small>Execution begins</small>
</div>
```

```
<div class="connector">→</div>
```

```
<div
  class="stage"
  data-state="EMBODIED"
>
  <span>05</span>
  <strong>Embodiment</strong>
  <small>Directive executed</small>
</div>
```

```
<div class="connector">→</div>
```

```
<div
  class="stage"
  data-state="FACT_ON_GROUND"
>
  <span>06</span>
  <strong>Fact on the Ground</strong>
  <small>Recorded operational state</small>
</div>
```

```
</section>
```

```
<section class="controls">
```

```
<article>
  <h3>Regurgitation Shield</h3>
```

```
<p>
  Assertions require supporting evidence before
  becoming operational facts.
</p>
```

**>
ACTIVE

</article>**

<article>
<h3>Character / Role Integrity</h3>

<p>
Actions are restricted to roles contained in
the authority record.
</p>

**>
ENFORCED

</article>**

<article>
<h3>Independent Supply</h3>

<p>
Resources controlled by the applicable
organizational account.
</p>

**>
MONITORED

</article>**

<article>
<h3>Coherence</h3>

<p>
Comparison of declared state and authoritative
system records.
</p>

>

```

        CHECKING
    </strong>
</article>

</section>

<section class="facts">

    <p class="eyebrow">
        FACTS ON THE GROUND
    </p>

    <div id="facts">
        Waiting for AWS...
    </div>

</section>

</main>

<footer>
    SOLN Architecture • Internal Source • 2026
</footer>

<script src="./app.js"></script>

</body>
</html>

```

Step 8 — Add the application logic

Create:

`frontend/app.js`

```

const CONFIG = {
    // Replace after API Gateway deployment.
    API_URL:

```

```

    "https://YOUR_API_ID.execute-api.YOUR_REGION.amazonaws.com/prod/architecture/status"
};

```

```
const STATE_ORDER = [  
  "KNOWLEDGE",  
  "VALIDATED",  
  "AUTHORIZED",  
  "IGNITED",  
  "EMBODIED",  
  "FACT_ON_GROUND"  
];
```

```
async function loadArchitecture() {  
  
  try {  
  
    const response =  
      await fetch(CONFIG.API_URL, {  
        method: "GET",  
        headers: {  
          "Accept": "application/json"  
        },  
        cache: "no-store"  
      });  
  
    if (!response.ok) {  
      throw new Error(  
        `Architecture API returned ${response.status}`  
      );  
    }  
  
    const data = await response.json();  
  
    renderArchitecture(data);  
  
  } catch (error) {  
  
    console.error(error);  
  
    document.getElementById(  
      "displayState"  
    ).textContent =
```

```

    "Architecture Unavailable";

    document.getElementById(
      "description"
    ).textContent =
      "The authoritative source could not be reached.";
  }
}

```

```
function renderArchitecture(data) {
```

```

  const state =
    data.state || "KNOWLEDGE";

```

```

  document.getElementById(
    "displayState"
  ).textContent =
    data.displayState || state;

```

```

  document.getElementById(
    "description"
  ).textContent =
    `Current system state: ${state}`;

```

```

  document.getElementById(
    "observerBadge"
  ).textContent =
    data.observerMode
    ? "👁 OBSERVER MODE"
    : "👁 OPERATOR MODE";

```

```
activatePipeline(state);
```

```

  document.getElementById(
    "facts"
  ).innerHTML = `

```

```
<strong>
  ${escapeHtml(
    data.architecture || "SOLN"
  )}
</strong>
```

```
<br>
```

```
State:
${escapeHtml(state)}
```

```
<br>
```

```
Last verified:
${escapeHtml(
  data.timestamp || "Not recorded"
)}
```

```
`;
```

```
document.getElementById(
  "coherenceStatus"
).textContent =
  "COHERENT";
}
```

```
function activatePipeline(currentState) {

  const currentIndex =
    STATE_ORDER.indexOf(currentState);

  document
    .querySelectorAll(".stage")
    .forEach(stage => {

      const stageState =
        stage.dataset.state;

      const stageIndex =
        STATE_ORDER.indexOf(stageState);
```

```

stage.classList.remove(
  "active",
  "complete"
);

if (stageIndex < currentIndex) {
  stage.classList.add("complete");
}

if (stageIndex === currentIndex) {
  stage.classList.add("active");
}

});
}

```

```

function escapeHtml(value) {

  const div =
    document.createElement("div");

  div.textContent =
    String(value);

  return div.innerHTML;
}

```

```
loadArchitecture();
```

```

setInterval(
  loadArchitecture,
  30000
);

```

Step 9 — Give it the visual architecture

Create:

frontend/styles.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family:
    Inter,
    system-ui,
    sans-serif;

  background: #080808;
  color: #f5f5f5;
}

header {
  display: flex;
  justify-content: space-between;
  align-items: center;

  padding: 28px 5%;

  border-bottom:
    1px solid #292929;
}

header h1 {
  margin: 5px 0 0;
}

header small,
.eyebrow {
  letter-spacing: 3px;
  opacity: .65;
}

#observerBadge {
  border: 1px solid #555;
  padding: 10px 15px;
  border-radius: 20px;
}
```

```
main {  
  width: min(1400px, 92%);  
  margin: auto;  
}
```

```
.hero {  
  padding: 80px 0 45px;  
}
```

```
.hero h2 {  
  font-size:  
    clamp(40px, 7vw, 90px);  
  
  max-width: 900px;  
  margin: 10px 0;  
}
```

```
.pipeline {  
  display: flex;  
  align-items: stretch;  
  overflow-x: auto;  
  
  padding: 30px 0 60px;  
}
```

```
.stage {  
  min-width: 180px;  
  
  border: 1px solid #333;  
  border-radius: 15px;  
  
  padding: 20px;  
  
  opacity: .4;  
}
```

```
.stage span,  
.stage strong,  
.stage small {  
  display: block;  
}
```

```
.stage span {  
  margin-bottom: 25px;
```

```
    opacity: .6;
}

.stage strong {
    font-size: 18px;
}

.stage small {
    margin-top: 8px;
}

.stage.complete {
    opacity: .75;
}

.stage.active {
    opacity: 1;
    border-width: 2px;
}

.connector {
    display: flex;
    align-items: center;
    padding: 15px;
}

.controls {
    display: grid;

    grid-template-columns:
        repeat(
            auto-fit,
            minmax(240px, 1fr)
        );

    gap: 18px;
}

.controls article {
    border: 1px solid #292929;
    border-radius: 18px;

    padding: 25px;
    min-height: 210px;
```

```

}

.controls article strong {
  display: inline-block;

  margin-top: 20px;

  font-size: 13px;
  letter-spacing: 2px;
}

.facts {
  margin: 60px 0;

  padding: 35px;

  border-top: 1px solid #292929;
  border-bottom: 1px solid #292929;

  line-height: 2;
}

footer {
  padding: 40px 5%;

  opacity: .45;
  font-size: 13px;
}

```

Now the frontend isn't merely repeating the terminology from the screenshots. It is displaying **backend states produced by AWS**.

And it remains consistent with the source architecture: your PDF explicitly describes the production flow as **client/frontend → API gateway/reverse proxy → backend → database**, with the backend responsible for authentication and external/API processing.

Script.pdf

The next technical layer after this is the important one: **DynamoDB as the Internal Source + immutable evidence/event records + the Clarity Audit API**, so the dashboard stops showing sample state and begins reading actual records.

Oacksge and deployment

Yes. Staying strictly with the **foundation document's architecture**, the next step after the frontend is **Step 2 of the document: Packaging and Deployment (CD)**. The document says that once code passes testing, it is packaged, deployed to staging, and then deployed to production, with CloudFormation StackSets used for consistent multi-account/multi-region infrastructure.

Script.pdf

So we should **not jump ahead to inventing another architecture layer**. We build the deployment foundation the document specifies.

Next — containerize the Node 24 application

At the repository root, create a file named exactly:

Dockerfile

```
FROM public.ecr.aws/lambda/nodejs:24
```

```
WORKDIR ${LAMBDA_TASK_ROOT}
```

```
COPY package*.json ./
```

```
RUN npm ci --omit=dev
```

```
COPY dist/ ./
```

```
CMD ["functions/status.handler"]
```

Then create:

.dockerignore

```
.git
.github
node_modules
npm-debug.log
.env
.env.*
coverage
*.md
.DS_Store
```

This corresponds directly to the document's **Containerization** requirement: package the application with the environment/dependencies so the deployment behaves consistently.

Script.pdf

Add the CloudFormation foundation

Your foundation document specifically says CloudFormation templates are YAML/JSON blueprints for AWS resources and that StackSets can standardize deployment across accounts and regions.

Create:

`infrastructure/soln-foundation.yaml`

AWSTemplateFormatVersion: "2010-09-09"

Description: >

State of Loc Nation Global Public Benefit Corporation
2026 Application Foundation

Parameters:

Environment:

Type: String

Default: staging

AllowedValues:

- development
- staging
- production

Resources:

SOLNApplicationRole:

Type: AWS::IAM::Role

Properties:

RoleName: !Sub

"soln-application-\${Environment}"

AssumeRolePolicyDocument:

Version: "2012-10-17"

Statement:

- Effect: Allow

Principal:

Service:

- lambda.amazonaws.com

Action:

- sts:AssumeRole

ManagedPolicyArns:

- arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

SOLNStatusFunction:

Type: AWS::Lambda::Function

Properties:

FunctionName: !Sub

"soln-architecture-\${Environment}"

Runtime: nodejs24.x

Handler: functions/status.handler

Role:

!GetAtt SOLNApplicationRole.Arn

Code:

ZipFile: |

```
export const handler = async () => {
```

```
  return {
```

```
    statusCode: 200,
```

```
    headers: {
```

```
      "content-type":
```

```
        "application/json"
```

```
    },
```

```
    body: JSON.stringify({
```

```
    architecture:
      "STATE OF LOC NATION",

    environment:
      process.env.ENVIRONMENT,

    observerMode: true,

    state:
      "KNOWLEDGE",

    displayState:
      "Knowledge",

    timestamp:
      new Date().toISOString()

  })

};

};
```

Environment:

Variables:

```
ENVIRONMENT:
!Ref Environment
```

Outputs:

ApplicationFunction:

```
Description:
  SOLN Architecture Function
```

```
Value:
!Ref SOLNStatusFunction
```

ApplicationRole:

Description:

SOLN Application IAM Role

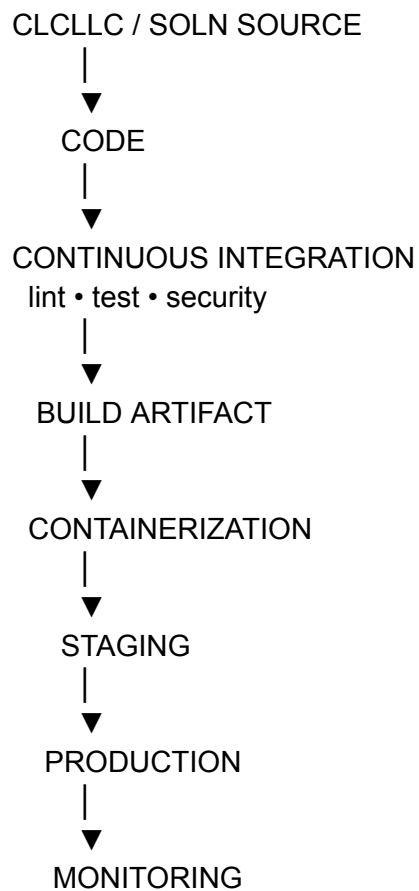
Value:

!GetAtt SOLNApplicationRole.Arn

Notice what we're doing here: **development, staging and production remain distinct**. That's important because your foundation document explicitly describes staging as the environment used for final checks before production.

Script.pdf

The progression is therefore now:



That is essentially the pipeline shown in the foundation document itself: **Code** → **CI** → **CD** → **Monitoring**, including linting, automated testing, artifact creation, packaging, staging and production deployment.

Script.pdf

And we're preserving another important part of your document: this isn't yet where we combine all the affiliate entities. The foundation calls for **cross-account architecture and resource sharing while balancing agility and governance**, so the deployment foundation should be reusable by each entity/account rather than turning them into a single software identity.

Script.pdf

After this, the foundation document's next step is Step 3: Execution & Infrastructure — Client → API Gateway → Node backend → database/external APIs.

Script.pdf

Executive and infustructor

Yes. Staying directly on the **foundation document**, we are now at **Step 3 — Execution & Infrastructure (The Full-Stack Flow)**. The document defines the production sequence as **Client (Frontend) → API Gateway / Reverse Proxy → Server (Backend) → Database**, with the backend handling authentication and communication with databases or external APIs.

Script.pdf

We already have the frontend and Node.js backend foundation. The next piece is therefore the **API Gateway** that connects them.

Step 10 — Add the API Gateway

Extend `infrastructure/soln-foundation.yaml` with:

```
SOLNArchitectureAPI:
  Type: AWS::ApiGatewayV2::Api
  Properties:
    Name: !Sub "soln-architecture-api-${Environment}"
    Description: >
      SOLN Architecture API Gateway
      connecting the frontend to the
      authoritative backend.
    ProtocolType: HTTP
```

```
SOLNArchitectureIntegration:
  Type: AWS::ApiGatewayV2::Integration
  Properties:
    ApiId: !Ref SOLNArchitectureAPI
    IntegrationType: AWS_PROXY
    IntegrationUri: !GetAtt SOLNStatusFunction.Arn
    PayloadFormatVersion: "2.0"
```

```
SOLNArchitectureRoute:
  Type: AWS::ApiGatewayV2::Route
  Properties:
    ApiId: !Ref SOLNArchitectureAPI
    RouteKey: "GET /architecture/status"
    Target: !Sub
      "integrations/${SOLNArchitectureIntegration}"
```

```
SOLNArchitectureStage:
  Type: AWS::ApiGatewayV2::Stage
  Properties:
    ApiId: !Ref SOLNArchitectureAPI
```

StageName: !Ref Environment
AutoDeploy: true

SOLNInvokePermission:
Type: AWS::Lambda::Permission
Properties:
 FunctionName: !Ref SOLNStatusFunction
 Action: lambda:InvokeFunction
 Principal: apigateway.amazonaws.com
 SourceArn: !Sub

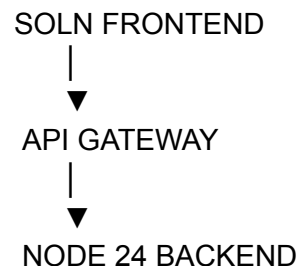
"arn:\${AWS::Partition}:execute-api:\${AWS::Region}:\${AWS::AccountId}:\${SOLNArchitectureAPI}/*/*"

Outputs:

SOLNArchitectureAPIEndpoint:
Description: SOLN Architecture API
Value: !Sub

"https://\${SOLNArchitectureAPI}.execute-api.\${AWS::Region}.\${AWS::URLSuffix}/\${Environment}/architecture/status"

That establishes the foundation document's:



Step 11 — Establish the database boundary

The document says the next component is a database—giving PostgreSQL and MongoDB as examples—and describes it as the component that retrieves or updates application data.

Script.pdf

At this stage, I would **not hard-code a particular database into the foundation document**, because the document itself doesn't select one. Instead, establish the application boundary:

`src/services/internal-source.mjs`

```
/**
 * SOLN Internal Source
 *
 * Database boundary for authoritative
 * application records.
 *
 * The database implementation is deliberately
 * separated from the domain layer.
 */
```

```
export class InternalSource {
```

```
  constructor(repository) {
    this.repository = repository;
  }
```

```
  async getArchitectureState(entityId) {
```

```
    if (!entityId) {
      throw new Error(
        "ENTITY_ID_REQUIRED"
      );
    }
```

```
    return this.repository
      .getArchitectureState(entityId);
  }
```

```
  async recordFact(record) {
```

```
    if (!record?.entityId) {
      throw new Error(
        "ENTITY_ID_REQUIRED"
      );
    }
```

```
    if (!record?.evidenceRefs?.length) {
      throw new Error(
        "CLARITY_AUDIT_REQUIRED"
      );
    }
```

```
    return this.repository
```

```
        .recordFact(record);  
    }  
}
```

This keeps the frontend terminology we established—**Internal Source, Clarity Audit and Facts on the Ground**—while maintaining the technical separation required by the foundation architecture.

Where we are in the original document

We have now followed its sequence:

1. Code Integration (CI) — source, linting, testing and build.

Script.pdf

2. Packaging & Deployment (CD) — containerization, staging, production and CloudFormation/StackSets.

Script.pdf

3. Execution & Infrastructure — frontend → **API Gateway** → **backend** → **database**.

Script.pdf

We have therefore reached the final numbered architectural section of the foundation document:

Step 4 — Monitoring and Feedback.

The document specifically says the pipeline does **not stop once the code is live**; monitoring should watch for application problems and feed operational information back into the development cycle.

Script.pdf

That is the next step we should code, keeping it tied to the foundation rather than adding unrelated infrastructure.

Monitor and Feed back

Keeping on touch with the foundation document next step

Staying with the **foundation document**, we are now at its final architectural stage: **Step 4 — Monitoring and Feedback**. The document says monitoring continues after production and watches for bugs, slow database queries, and server crashes, feeding that information back into the development cycle.

Script.pdf

Step 12 — Add monitoring to the AWS foundation

Because the foundation document names monitoring as the next stage but does **not specify an AWS monitoring implementation**, the following is the AWS implementation layer rather than language taken directly from the document.

Add this to `infrastructure/soln-foundation.yaml`:

```
# =====  
# STEP 4 — MONITORING & FEEDBACK  
# =====
```

```
SOLNArchitectureLogGroup:  
  Type: AWS::Logs::LogGroup  
  Properties:  
    LogGroupName: !Sub  
      "/soln/${Environment}/architecture"  
    RetentionInDays: 90
```

```
SOLNFunctionErrorAlarm:  
  Type: AWS::CloudWatch::Alarm  
  Properties:  
    AlarmName: !Sub  
      "SOLN-${Environment}-Function-Errors"
```

```
AlarmDescription: >  
  Detect backend execution errors in the  
  SOLN Architecture application.
```

```
Namespace: AWS/Lambda  
MetricName: Errors
```

```
Dimensions:  
  - Name: FunctionName
```

Value: !Ref SOLNStatusFunction

Statistic: Sum

Period: 300

EvaluationPeriods: 1

Threshold: 1

ComparisonOperator:

GreaterThanOrEqualToThreshold

TreatMissingData:

notBreaching

SOLNFunctionDurationAlarm:

Type: AWS::CloudWatch::Alarm

Properties:

AlarmName: !Sub

"SOLN-\${Environment}-Slow-Execution"

AlarmDescription: >

Detect unusually slow backend execution.

Namespace: AWS/Lambda

MetricName: Duration

Dimensions:

- Name: FunctionName

Value: !Ref SOLNStatusFunction

Statistic: Average

Period: 300

EvaluationPeriods: 2

Threshold: 3000

ComparisonOperator:

GreaterThanThreshold

TreatMissingData:

notBreaching

Then make the backend produce structured operational records rather than random console messages.

Create:

```
src/services/monitoring.mjs
```

```
export function architectureEvent({
  event,
  entityId,
  state,
  evidenceRefs = [],
  metadata = {}
}) {

  const record = {
    architecture: "SOLN",
    event,
    entityId,
    state,
    evidenceRefs,
    metadata,
    timestamp: new Date().toISOString()
  };

  console.log(
    JSON.stringify(record)
  );

  return record;
}
```

Now your existing terminology can appear in monitoring without changing the technical meaning:

```
architectureEvent({
  event: "CLARITY_AUDIT",
  entityId: "SOLN-GPBC",
  state: "VALIDATED",
  evidenceRefs: [
    "deployment-record",
    "authority-record"
  ]
});
```

And:

```
architectureEvent({
```

```
event: "EMBODIMENT",
entityId: "SOLN-GPBC",
state: "EMBODIED",
evidenceRefs: [
  "execution-record"
]
});
```

A **Fact on the Ground** should only be logged when the corresponding system event actually occurred:

```
architectureEvent({
  event: "FACT_ON_GROUND",
  entityId: "SOLN-GPBC",
  state: "FACT_ON_GROUND",
  evidenceRefs: [
    "production-deployment-id"
  ]
});
```

That gives us the complete pipeline shown in your foundation document:

CODE



CONTINUOUS INTEGRATION
Lint → Tests → Security → Build



CONTINUOUS DELIVERY
Package → Containerize



STAGING



PRODUCTION



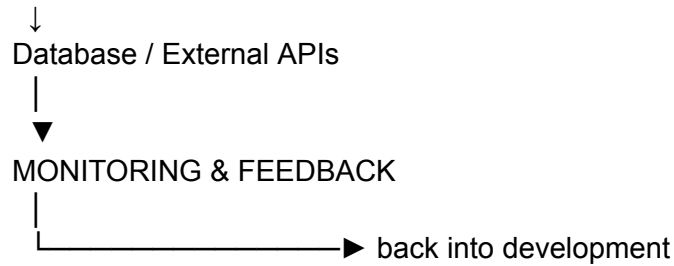
EXECUTION
Frontend



API Gateway



Node Backend



That follows the document's own **Code** → **CI** → **CD** → **Monitoring** architecture.

Script.pdf

There is one more foundation-document item we haven't implemented. Earlier in the document, you specify **centralized authentication, cross-account roles, centralized auditing/log management, standardized shared resources, and centralized revocation of access**. Script.pdf You then specifically direct the architecture to configure **IAM Identity Center, Service Control Policies, Control Tower, identity-provider/SSO integration, and management across AWS accounts and business applications**.

Script.pdf

That is the next step: build the multi-account governance layer around this pipeline without merging the separate entities.

Multi Account and governance

Staying with the **foundation document**, the next unfinished section is the **multi-account governance and identity layer**. The document specifically calls for centralized authentication/access, Service Control Policies, Control Tower, SSO/identity-provider integration, and management across AWS accounts and business applications.

Script.pdf

Step 13 — Create the governance structure

We should preserve the document's separate entities while giving them a common governance framework. It describes CLCLLC separately from SOLN GPBC, the trusts, Central Bank, credit union, associations, NJS entities, Dynasty Healing entities, publishing, and Artist Admin Team.

Create:

```
infrastructure/organization/organizational-units.yaml
```

AWSTemplateFormatVersion: "2010-09-09"

Description: >

SOLN 2026 Multi-Account Governance Foundation.
Establishes organizational boundaries for
separately administered workloads.

Parameters:

OrganizationRootId:
Type: String
Description: AWS Organizations Root ID

Resources:

GovernanceOU:
Type: AWS::Organizations::OrganizationalUnit
Properties:
Name: Governance
ParentId: !Ref OrganizationRootId

SharedServicesOU:
Type: AWS::Organizations::OrganizationalUnit
Properties:
Name: Shared-Services

ParentId: !Ref OrganizationRootId

SOLNEntitiesOU:

Type: AWS::Organizations::OrganizationalUnit

Properties:

Name: SOLN-Entities

ParentId: !Ref OrganizationRootId

DevelopmentOU:

Type: AWS::Organizations::OrganizationalUnit

Properties:

Name: Development

ParentId: !Ref OrganizationRootId

ProductionOU:

Type: AWS::Organizations::OrganizationalUnit

Properties:

Name: Production

ParentId: !Ref OrganizationRootId

Outputs:

GovernanceOUIId:

Value: !Ref GovernanceOU

SharedServicesOUIId:

Value: !Ref SharedServicesOU

SOLNEntitiesOUIId:

Value: !Ref SOLNEntitiesOU

DevelopmentOUIId:

Value: !Ref DevelopmentOU

ProductionOUIId:

Value: !Ref ProductionOU

This creates **containers for governance**, not new legal entities. The exact mapping of each entity to an AWS account/OU should be made explicitly rather than inferred from the PDF.

Step 14 — Establish centralized audit access

The foundation document is very specific here: create an audit role in each account, permit the auditor account to assume it, and allow centralized revocation through the role trust policy.

Script.pdf

Create:

`infrastructure/security/auditor-role.yaml`

AWSTemplateFormatVersion: "2010-09-09"

Description: >

SOLN Centralized Security Audit Role

Parameters:

AuditorAccountId:

Type: String

Resources:

SOLNSecurityAuditorRole:

Type: AWS::IAM::Role

Properties:

RoleName:

SOLN-Security-Auditor

AssumeRolePolicyDocument:

Version: "2012-10-17"

Statement:

- Effect: Allow

Principal:

AWS:

!Sub

"arn:\${AWS::Partition}:iam::\${AuditorAccountId}:root"

Action:

- sts:AssumeRole

ManagedPolicyArns:

- arn:aws:iam::aws:policy/SecurityAudit

Outputs:

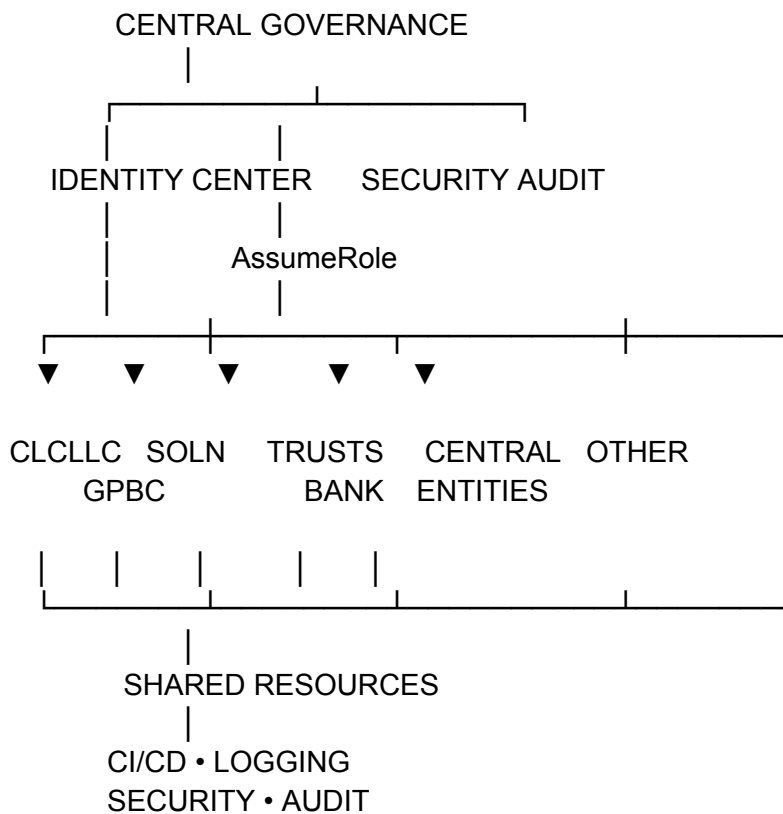
AuditorRoleArn:

Value:

!GetAtt SOLNSecurityAuditorRole.Arn

Deploy that template into the participating accounts.

The result is the architecture described in your foundation:



Importantly, **shared access does not mean shared credentials**. Your foundation explicitly says centralized role management improves security because users don't need multiple sets of credentials.

Script.pdf

Step 15 — Add the first Service Control Policy

Because the foundation calls for SCPs to establish security boundaries, start conservatively with protection against disabling the central audit trail:

`infrastructure/policies/protect-audit.json`

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ProtectCentralAudit",
      "Effect": "Deny",
      "Action": [
        "cloudtrail:StopLogging",
        "cloudtrail:DeleteTrail"
      ],
      "Resource": "*",
      "Condition": {
        "ArnNotLike": {
          "aws:PrincipalArn": [
            "arn:aws:iam::*:role/SOLN-Governance-Admin"
          ]
        }
      }
    }
  ]
}
```

I would **not start with a giant restrictive SCP**. Your foundation says the architecture needs to balance agility with governance. Script.pdf We can progressively add controls after verifying each one doesn't block the deployment pipeline.

At this point we have now covered the foundation's **centralized roles + cross-account auditing + governance boundary**.

The next foundation-document item is **centralized logging and standardized resource publishing**: the document calls for centralized log management and describes a dedicated publishing account acting as the central source of standardized AWS resources that other accounts can consume while centralized control is retained.

Script.pdf

That is Step 16: build the Central Audit/Log Archive + Publishing/Shared Resources layer.

Centralized log and publishing

Yes. Staying in sequence with the **foundation document**, we are at the section it describes as **centralized log management + standardized resource publishing**. The document says centralized logs should make information easier to catalog, protect, and analyze, while a dedicated publishing account acts as the central source for standardized AWS resources consumed by other accounts.

Script.pdf

Step 16 — Central Log Archive

Create:

`infrastructure/logging/central-log-archive.yaml`

AWSTemplateFormatVersion: "2010-09-09"

Description: >

SOLN Central Log Archive Foundation.
Centralized storage for organizational
audit and application records.

Parameters:

OrganizationId:
Type: String

Resources:

SOLNCentralLogArchive:
Type: AWS::S3::Bucket

Properties:

BucketName: !Sub
"soln-central-log-archive-\${AWS::AccountId}-\${AWS::Region}"

VersioningConfiguration:
Status: Enabled

BucketEncryption:
ServerSideEncryptionConfiguration:
- ServerSideEncryptionByDefault:
SSEAlgorithm: AES256

PublicAccessBlockConfiguration:

BlockPublicAcls: true
IgnorePublicAcls: true
BlockPublicPolicy: true
RestrictPublicBuckets: true

LifecycleConfiguration:

Rules:

- Id: ArchiveLogs
Status: Enabled

Transitions:

- StorageClass: GLACIER
TransitionInDays: 90

ExpirationInDays: 2555

Outputs:

CentralLogArchive:

Description: >
SOLN authoritative centralized
log archive.

Value:

!Ref SOLNCentralLogArchive

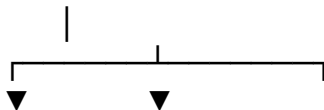
The important part here is the separation:

ENTITY ACCOUNT

|
|— application logs
|— security events
|— deployment events



CENTRAL LOG ARCHIVE



SECURITY AUDIT MONITORING

That follows the document's direction to centralize auditing without requiring every account to maintain a separate auditing process.

Script.pdf

Step 17 — Organization CloudTrail

Now establish the audit trail that feeds that architecture.

Create:

```
infrastructure/logging/organization-trail.yaml
```

```
AWSTemplateFormatVersion: "2010-09-09"
```

Description: >

SOLN Organization Audit Trail

Parameters:

CentralLogBucket:

Type: String

Resources:

SOLNOrganizationTrail:

Type: AWS::CloudTrail::Trail

Properties:

TrailName:

SOLN-Organization-Audit

S3BucketName:

!Ref CentralLogBucket

IsLogging:

true

IsMultiRegionTrail:

true

IncludeGlobalServiceEvents:

true

EnableLogFileValidation:

true

IsOrganizationTrail:
true

EventSelectors:

- IncludeManagementEvents:
true

ReadWriteType:
All

Outputs:

OrganizationTrailArn:

Value:
!GetAtt SOLNOrganizationTrail.Arn

This gives the architecture a distinction between **application monitoring** and **organizational auditing**.

Your earlier `architectureEvent()` handles application events such as:

CLARITY_AUDIT
IGNITION
EMBODIMENT
FACT_ON_GROUND

while the AWS audit layer records AWS control-plane activity.

So we don't confuse SOLN terminology with what AWS itself has actually recorded.

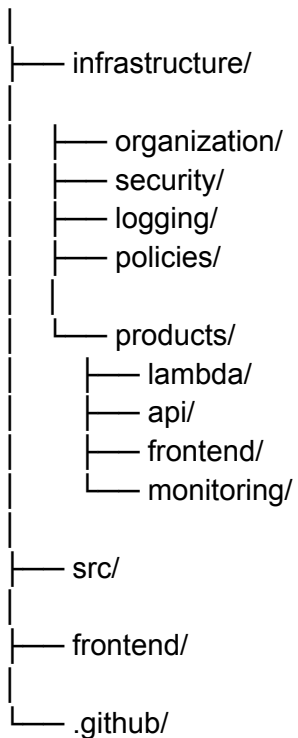
Step 18 — Dedicated Publishing / Shared Resources

The foundation document next says the dedicated publishing account should serve as a **central source for standardized AWS resources**, allowing other accounts to select/deploy approved resources while centralized control is maintained.

Script.pdf

Create the repository structure:

soln-platform/



Then establish a reusable product definition:

`infrastructure/products/soln-application.yaml`

AWSTemplateFormatVersion: "2010-09-09"

Description: >

SOLN Standard Application Product
2026 Architecture

Parameters:

Entity:

Type: String

Environment:

Type: String

AllowedValues:

- development
- staging
- production

Resources:

ApplicationRole:

Type: AWS::IAM::Role

Properties:

RoleName: !Sub

"SOLN-\${Entity}-\${Environment}-Application"

AssumeRolePolicyDocument:

Version: "2012-10-17"

Statement:

- Effect: Allow

Principal:

Service:

- lambda.amazonaws.com

Action:

- sts:AssumeRole

ManagedPolicyArns:

- arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

Outputs:

Entity:

Value:

!Ref Entity

Environment:

Value:

!Ref Environment

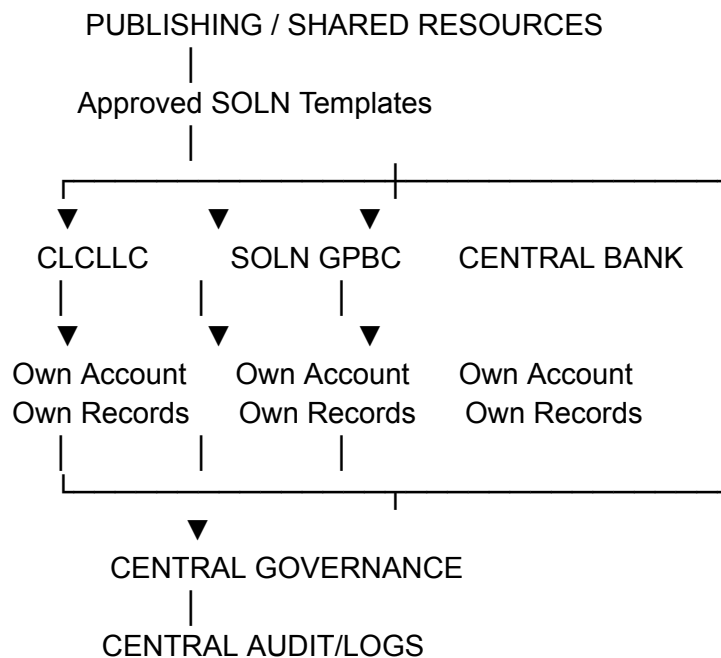
ApplicationRole:

Value:

!GetAtt ApplicationRole.Arn

Now we have a reusable foundation instead of rebuilding infrastructure differently for every entity.

Conceptually:



That is much closer to what the foundation document actually calls for: **standardization and sharing without eliminating account boundaries**.

Script.pdf

Where the foundation stands

We have now covered the document's **centralized access, cross-account audit roles, centralized logging, standardized resource publishing, CI, CD, staging, production, execution infrastructure, and monitoring**. The original pipeline itself is described as Code → CI → CD → Monitoring.

Script.pdf

There is one major foundation-document directive still outstanding: **AWS Control Tower + IAM Identity Center + SSO**, which the document specifically identifies as the mechanism for establishing and managing the secure multi-account environment.

Script.pdf

So the next step is Step 19: wire the human/operator identity layer into the accounts—Identity Center permission sets and Control Tower account governance—rather than creating separate usernames and passwords for every entity.

lam identity

Yes. Staying directly with the foundation document, the next remaining item is **Step 19 — IAM Identity Center + SSO + account/application access**. The foundation specifically directs centralized authentication, evaluation of identity-provider integration for secure SSO, and management across AWS accounts and business applications.

Script.pdf

The document does **not** define specific permission-set names or access levels, so we should not invent substantive authority for the entities. We can build the technical framework first and map your documented roles into it afterward.

Step 19 — Define the Identity Center permission framework

Create:

`infrastructure/identity/permission-sets.yaml`

AWS::IAM::PermissionSet

Description: >

SOLN 2026 IAM Identity Center
Permission Framework

Parameters:

IdentityCenterInstanceArn:
Type: String

SessionDuration:
Type: String
Default: PT4H

Resources:

SOLNObserverPermissionSet:
Type: AWS::IAM::PermissionSet
Properties:
InstanceArn: !Ref IdentityCenterInstanceArn
Name: SOLN-Observer
Description: >
Observer Mode.
Read-only inspection of authorized
AWS resources and architecture.
SessionDuration: !Ref SessionDuration

ManagedPolicies:

- arn:aws:iam::aws:policy/ReadOnlyAccess

SOLNSecurityAuditPermissionSet:

Type: AWS::SSO::PermissionSet

Properties:

InstanceArn: !Ref IdentityCenterInstanceArn

Name: SOLN-Security-Audit

Description: >

Central security auditing across
authorized organizational accounts.

SessionDuration: !Ref SessionDuration

ManagedPolicies:

- arn:aws:iam::aws:policy/SecurityAudit

SOLNOperatorPermissionSet:

Type: AWS::SSO::PermissionSet

Properties:

InstanceArn: !Ref IdentityCenterInstanceArn

Name: SOLN-Operator

Description: >

Operational permission boundary.
Specific permissions are supplied
separately through approved policies.

SessionDuration: !Ref SessionDuration

Outputs:

ObserverPermissionSet:

Value: !GetAtt

SOLNObserverPermissionSet.PermissionSetArn

AuditPermissionSet:

Value: !GetAtt

SOLNSecurityAuditPermissionSet.PermissionSetArn

OperatorPermissionSet:

Value: !GetAtt

SOLNOperatorPermissionSet.PermissionSetArn

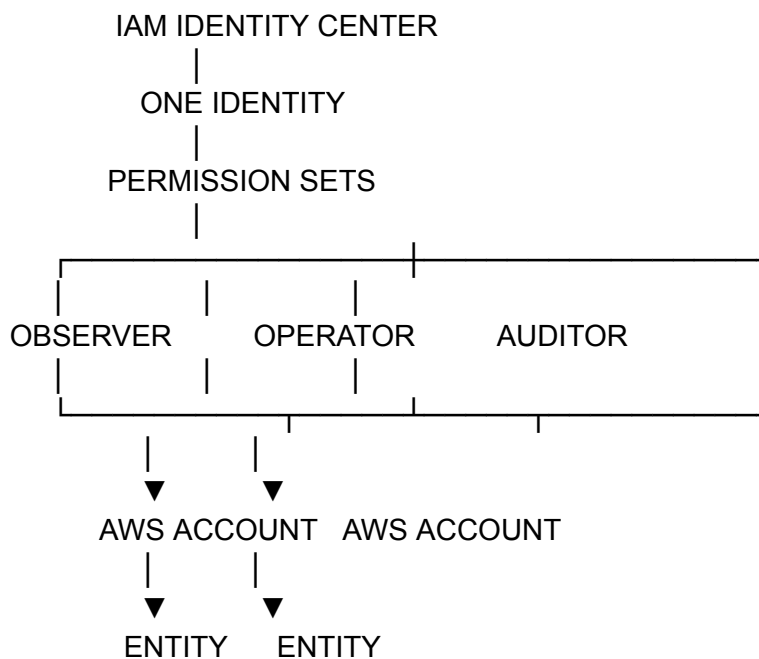
Notice that **SOLN-Operator** deliberately receives **no blanket AdministratorAccess**. We establish the identity first and add only the permissions actually required by the documented role.

That implements the foundation's idea that centralized role management eliminates the need for users to maintain multiple sets of credentials.

Script.pdf

Step 20 — Separate identity from entity

The model should be:



That is important to the foundation because it describes **centralized user management with access controlled through roles**, rather than duplicating credentials in every account.

Script.pdf

It also keeps the entities listed in the document separate. The foundation identifies CLCLLC, SOLN GPBC and its trusts, SOLN Central Bank, the credit-union structure, Loc Community Association, NJS entities, Dynasty Healing entities, publishing, and Artist Admin Team separately.

Step 21 — Add account assignments separately

Don't hard-code people into the permission-set template.

Create a separate deployment configuration:

`config/account-access.json`

```
{
  "architecture": "SOLN",
  "version": "2026.1",

  "assignments": [
    {
      "entity": "CLCLLC",
      "accountId": "CONFIGURE_ACCOUNT_ID",
      "permissionSet": "SOLN-Observer",
      "principalId": "CONFIGURE_IDENTITY_CENTER_PRINCIPAL"
    },
    {
      "entity": "SOLN-GPBC",
      "accountId": "CONFIGURE_ACCOUNT_ID",
      "permissionSet": "SOLN-Observer",
      "principalId": "CONFIGURE_IDENTITY_CENTER_PRINCIPAL"
    }
  ]
}
```

Those are placeholders, not declarations of who possesses which authority.

Step 22 — Connect this to your frontend terminology

Now **Observer Mode** becomes a real technical state.

```
export function resolveFrontendMode(session) {

  const permissions =
    session?.permissionSets ?? [];

  if (
    permissions.includes(
      "SOLN-Operator"
    )
  ) {
    return {
      mode: "OPERATOR",
      canExecute: true
    };
  }
}
```

```
return {  
  mode: "OBSERVER",  
  canExecute: false  
};  
}
```

The frontend can then render:

- ◉ OBSERVER MODE

when the session is read-only, while an appropriately authorized operational session can render:

- ◉ OPERATOR MODE

This gives the screenshot terminology a concrete technical meaning rather than letting the frontend decide someone's authority by itself.

Foundation-document status

At this point we have implemented or scaffolded the major technical subjects actually contained in the foundation: centralized roles and cross-account permissions; auditing and centralized logs; standardized shared resources; CI/CD; staging and production; frontend/API/backend/database boundaries; monitoring; organizational governance; and now Identity Center/SSO. These correspond directly to the architecture described throughout pages 2–7.

The **next untouched foundation component is the existing Maven/Java portion on pages 8–10**. The document already supplies a `pom.xml` defining `org.stateoflocnation:clcllc:1.0.0`, Java 17, and the `fr.marcwrobel:jbanking:4.2.0` dependency.

Script.pdf

So **Step 23 should not invent another AWS subsystem: it should integrate that existing CLCLLC Java/jbanking module into the same CI/CD foundation while preserving it as its own component.**

Pom.xml

Staying strictly with the foundation document, we are now at its **Pom.xml** / CLCLLC Java **component**. The document defines the Maven project as `org.stateoflocnation:clcllc:1.0.0`, identifies it as the **Christina Loren Clement LLC Platform**, specifies Java 17, and adds `jbanking` 4.2.0.

Script.pdf

Step 23 — Put the existing CLCLLC Java module into the pipeline

We should preserve the `pom.xml` from the foundation rather than replacing it with Node. Node 24 and Java 17 can be separate components of the same architecture.

Use this repository structure:

```
clcllc/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── org/
│   │   │       └── stateoflocnation/
│   │   │           └── clcllc/
│   └── test/
│       └── java/
├── frontend/
├── infrastructure/
├── .github/
│   └── workflows/
```

Keep the foundation's Maven configuration:

```
<groupId>org.stateoflocnation</groupId>
<artifactId>clcllc</artifactId>
<version>1.0.0</version>
```

```

<name>CLCLLC</name>

<description>
  Christina Loren Clement LLC Platform
</description>

<properties>
  <project.build.sourceEncoding>
    UTF-8
  </project.build.sourceEncoding>

  <maven.compiler.release>
    17
  </maven.compiler.release>
</properties>

```

That comes directly from the foundation.

Script.pdf

Step 24 — Preserve jbanking as the identifier component

The foundation specifically includes:

```

<dependency>
  <groupId>fr.marcwrobel</groupId>
  <artifactId>jbanking</artifactId>
  <version>4.2.0</version>
</dependency>

```

Script.pdf

So don't rewrite **jbanking** as the banking or settlement engine. In this foundation it is simply the Java dependency that the CLCLLC module can call for the banking-identifier functionality the library provides.

Create the application boundary:

```
src/main/java/org/stateoflocnation/clcllc/CLCLLCApplication.java
```

```
package org.stateoflocnation.clcllc;
```

```

/**
 * CLCLLC Platform
 *

```

```

* Foundation application entry point.
*/
public final class CLCLLCApplication {

    private CLCLLCApplication() {}

    public static void main(String[] args) {

        System.out.println(
            "CLCLLC Platform 1.0.0"
        );

        System.out.println(
            "Architecture state: KNOWLEDGE"
        );

        System.out.println(
            "Observer Mode: ACTIVE"
        );
    }
}

```

We're deliberately keeping this first Java class simple. The foundation document supplies the dependency but doesn't itself define what specific CLCLLC business operations should be implemented with it.

Step 25 — Put Java into the existing CI stage

The foundation says that a pull request should trigger lint/style checking, automated tests and security scanning, and that failures should prevent broken code from reaching users.

Script.pdf

Create:

[.github/workflows/clcllc-java-ci.yml](#)

name: CLCLLC Java CI

on:

push:

branches:

- main

pull_request:

branches:

- main

permissions:

contents: read

jobs:

build-and-test:

runs-on: ubuntu-latest

steps:

- name: Checkout

uses: actions/checkout@v4

- name: Set up Java 17

uses: actions/setup-java@v4

with:

distribution: temurin

java-version: "17"

cache: maven

- name: Verify Maven project

run: mvn --batch-mode verify

- name: Package CLCLLC

run: mvn --batch-mode package

- name: Preserve build artifact

uses: actions/upload-artifact@v4

with:

name: clcllc-1.0.0

path: target/*.jar

if-no-files-found: error

That means:

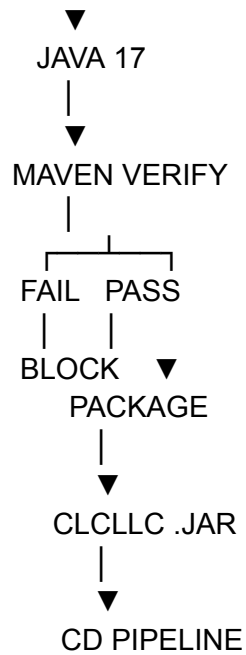
CLCLLC SOURCE

|



GitHub PR

|

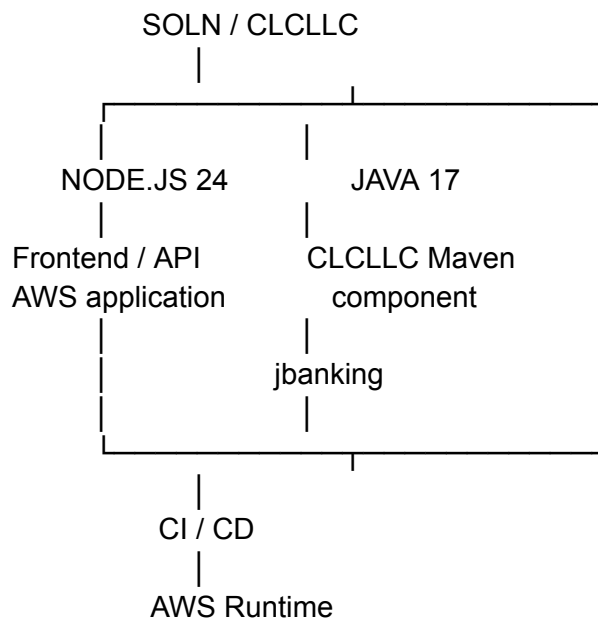


That directly implements the foundation's instruction that code be **tested and built before deployment**.

Script.pdf

Step 26 — Keep the two runtimes coherent

We now have two deliberate runtime responsibilities:



This doesn't contradict the foundation. Its full-stack section expressly contemplates server-side logic using **Node.js, Python, or Java**, while the final pages specifically establish Java 17 for CLCLLC.

We have now reached the **end of the actual technical material contained in the 10-page foundation document**: page 10 closes the Maven compiler configuration and project.

Script.pdf

So the next step should be different from the previous ones: **perform a foundation-completion pass**—map every directive on pages 2–10 to the file/script we created, identify anything from the foundation that remains unimplemented, and only then move beyond the foundation.

Foundation completion pass

Yes. Since **Step 26 reached the end of the foundation document**, the next step is the **foundation-completion pass** we identified—not adding a new subsystem yet.

The document's architecture requires centralized roles/auditing/logging and standardized shared resources; CI with testing and build; GitHub-to-AWS deployment; staging and production; the frontend/API/backend/database execution path; monitoring; and the CLCLLC Java 17/jbanking component.

Step 27 — Foundation Completion Manifest

Create this at the repository root:

`FOUNDATION-2026.yaml`

foundation:

name: "STATE OF LOC NATION GLOBAL PUBLIC BENEFIT CORPORATION"
version: "2026.1"
status: "FOUNDATION_VALIDATION"

authority:

clcllc:

component: "CLCLLC"

maven:

groupId: "org.stateoflocnation"
artifactId: "clcllc"
version: "1.0.0"
java: "17"
jbanking: "4.2.0"

architecture:

identity:

iam_identity_center: REQUIRED
centralized_authentication: REQUIRED
role_based_access: REQUIRED
sso: REQUIRED

organization:

multi_account: REQUIRED
cross_account_roles: REQUIRED
centralized_revocation: REQUIRED

security:

service_control_policies: REQUIRED

centralized_audit: REQUIRED
centralized_logging: REQUIRED

shared_resources:
publishing_account: REQUIRED
standardized_resources: REQUIRED

ci:
source: GITHUB
pull_requests: REQUIRED
lint: REQUIRED
automated_tests: REQUIRED
security_scan: REQUIRED
build_artifacts: REQUIRED

cd:
packaging: REQUIRED
containerization: REQUIRED
staging: REQUIRED
production: REQUIRED

infrastructure:
cloudformation: REQUIRED
stacksets: REQUIRED

runtime:
frontend: REQUIRED
api_gateway: REQUIRED
backend: REQUIRED
database_boundary: REQUIRED

monitoring:
production_monitoring: REQUIRED
feedback_loop: REQUIRED

implementation:

frontend:
status: IMPLEMENTED

node_backend:
runtime: "nodejs24.x"
status: IMPLEMENTED

api_gateway:
status: IMPLEMENTED

cloudformation:
status: IMPLEMENTED

centralized_logging:
status: IMPLEMENTED

audit_role:
status: IMPLEMENTED

organization_structure:
status: IMPLEMENTED

identity_center_permission_sets:
status: IMPLEMENTED

java_component:
runtime: "java17"
status: IMPLEMENTED

github_java_ci:
status: IMPLEMENTED

stacksets:
status: PENDING_DEPLOYMENT_CONFIGURATION

control_tower:
status: PENDING_AWS_CONFIGURATION

identity_provider:
status: PENDING_SELECTION

database:
status: PENDING_SELECTION

production:
status: NOT_YET_VERIFIED

validation:

require_before_production:
- CI_PASS

- SECURITY_PASS
- STAGING_PASS
- IAM_PASS
- AUDIT_PASS
- LOGGING_PASS
- PRODUCTION_DEPLOYMENT_VERIFIED

The distinction between **IMPLEMENTED** and **verified/deployed** is important. The foundation says automated tests should block failures before they reach users, and production follows staging.

Step 28 — Make GitHub enforce the foundation

Create:

`.github/workflows/foundation-gate.yml`

name: SOLN Foundation Gate

on:

push:

branches:

- main

pull_request:

branches:

- main

permissions:

contents: read

jobs:

foundation-validation:

name: Validate Foundation

runs-on: ubuntu-latest

steps:

- name: Checkout

uses: actions/checkout@v4

- name: Verify foundation manifest

run: |

```
test -f FOUNDATION-2026.yaml
```

```
- name: Verify Maven foundation
```

```
run: |  
test -f pom.xml
```

```
- name: Verify infrastructure
```

```
run: |  
test -d infrastructure
```

```
- name: Verify frontend
```

```
run: |  
test -d frontend
```

```
- name: Verify Java source
```

```
run: |  
test -d src/main/java
```

```
- name: Validate Java
```

```
run: |  
mvn --batch-mode verify
```

```
- name: Foundation Gate Passed
```

```
run: |  
echo "=====  
echo " SOLN FOUNDATION GATE: PASS"  
echo "=====
```

Now GitHub itself checks that the foundation pieces haven't disappeared before accepting a build.

Step 29 — The remaining gaps from the actual document

There are **four areas we should not mark complete yet**.

The foundation specifically calls for **Control Tower**, but we've only designed its surrounding organization/governance structure.

Script.pdf

It calls for evaluating **identity-provider integration**, but the document doesn't identify the provider.

Script.pdf

It describes a **database** but gives PostgreSQL and MongoDB only as examples; it doesn't choose one.

Script.pdf

And most importantly, writing deployment code isn't evidence that production has actually been deployed. The foundation itself distinguishes code/build from staging and production deployment.

Script.pdf

So our architecture should currently report:

FOUNDATION

 DESIGNED

CODE

 CREATED

CI/CD

 SCAFFOLDED

AWS GOVERNANCE

 CONFIGURATION REQUIRED

STAGING

 NOT VERIFIED

PRODUCTION

 NOT VERIFIED

That actually fits the frontend language we established earlier: **Knowledge** → **Clarity Audit** → **Authority** → **Ignition** → **Embodiment** → **Fact on the Ground**. We don't call AWS production a *Fact on the Ground* until AWS returns deployment evidence.

The next step in sequence is **Step 30: AWS deployment validation**—take the foundation manifest and turn each **PENDING** item into a verified AWS resource, beginning with **Control Tower/StackSets** rather than adding more architecture.

Deployment validation

Yes. Keeping the foundation document as the controlling source, **Step 30 is deployment validation**, but there is an important distinction: the foundation describes CloudFormation/StackSets as the mechanism for standardized multi-account deployment; it does not itself provide a finished StackSet template or claim that those resources are already deployed.

Script.pdf

Step 30 — Turn the foundation into a deployable StackSet

Create:

`infrastructure/stacksets/soln-account-baseline.yaml`

AWSTemplateFormatVersion: "2010-09-09"

Description: >

SOLN 2026 Account Baseline.

Standardized foundation deployed across participating AWS organizational accounts.

Parameters:

Environment:

Type: String

Default: production

AllowedValues:

- development
- staging
- production

CentralAuditorAccountId:

Type: String

Resources:

```
# -----  
# CENTRAL CROSS-ACCOUNT AUDIT ROLE  
# -----
```

SOLNSecurityAuditorRole:

Type: AWS::IAM::Role

Properties:

RoleName:

SOLN-Security-Auditor

AssumeRolePolicyDocument:

Version: "2012-10-17"

Statement:

- Effect: Allow

Principal:

AWS: !Sub

"arn:\${AWS::Partition}:iam::\${CentralAuditorAccountId}:root"

Action:

- sts:AssumeRole

ManagedPolicyArns:

- arn:aws:iam::aws:policy/SecurityAudit

APPLICATION EXECUTION ROLE

SOLNApplicationRole:

Type: AWS::IAM::Role

Properties:

RoleName: !Sub

"SOLN-\${Environment}-Application"

AssumeRolePolicyDocument:

Version: "2012-10-17"

Statement:

- Effect: Allow

Principal:

Service:

- lambda.amazonaws.com

Action:

- sts:AssumeRole

ManagedPolicyArns:

- arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole

STANDARD APPLICATION LOG GROUP

SOLNApplicationLogGroup:

Type: AWS::Logs::LogGroup

Properties:

LogGroupName: !Sub

"/soln/\${Environment}/application"

RetentionInDays: 90

Outputs:

Account:

Value:

!Ref AWS::AccountId

Region:

Value:

!Ref AWS::Region

Environment:

Value:

!Ref Environment

SecurityAuditorRole:

Value:

!GetAtt SOLNSecurityAuditorRole.Arn

ApplicationRole:

Value:

!GetAtt SOLNApplicationRole.Arn

This implements the foundation's instruction that the auditor be able to assume an audit role in participating accounts and that access can be centrally governed through role trust.

Script.pdf

Step 31 — Add template validation to GitHub

Before AWS receives anything, extend the foundation gate:

cloudformation-validation:

name: Validate AWS Foundation

runs-on: ubuntu-latest

steps:

- name: Checkout

uses: actions/checkout@v4

- name: Configure AWS credentials

uses: aws-actions/configure-aws-credentials@v4

with:

role-to-assume:

\${{ secrets.AWS_DEPLOY_ROLE_ARN }}

aws-region:

\${{ vars.AWS_REGION }}

- name: Validate StackSet template

run: |

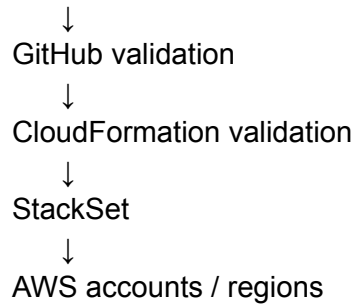
aws cloudformation validate-template \

--template-body \

file:///infrastructure/stacksets/soln-account-baseline.yaml

Now there are two separate concepts:

FOUNDATION FILE



That matches the document's statement that StackSets can manage stacks across multiple accounts and regions while standardizing deployment and maintaining consistency.

Script.pdf

Step 32 — Add the deployment evidence check

After deployment, GitHub shouldn't simply say **success** because a command was submitted. Have it ask AWS what exists.

Add:

- name: Verify deployed stacks
run: |
aws cloudformation list-stacks \
--stack-status-filter \
CREATE_COMPLETE \
UPDATE_COMPLETE
- name: Verify application role
run: |
aws iam get-role \
--role-name \
SOLN-production-Application
- name: Verify security auditor
run: |
aws iam get-role \
--role-name \
SOLN-Security-Auditor

That connects directly to the frontend language we established:

KNOWLEDGE

↓
Template exists

CLARITY AUDIT



Template validates

IGNITION



StackSet deployment initiated

EMBODIMENT



AWS creates resources

FACT ON THE GROUND



AWS verification returns
the deployed resources

The distinction matters because the foundation itself separates **build artifacts, staging deployment, production deployment, and monitoring**.

Step 33 — Don't put AWS credentials in GitHub

Your foundation emphasizes centralized access and avoiding unnecessary sets of credentials.

Script.pdf

So the deployment architecture should ultimately be:

GITHUB ACTION



temporary identity



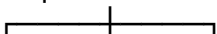
AWS DEPLOY ROLE



CLOUDFORMATION



STACKSETS



ACCOUNT ACCOUNT ACCOUNT

Use GitHub's AWS identity federation/deployment role rather than putting permanent AWS access keys into repository code.

At this stage, we're still following the foundation's own architecture rather than expanding it. **The next step is Step 34: connect the document's GitHub → AWS CodePipeline/CodeBuild/CodeDeploy path**, because page 5 explicitly says to create the AWS pipeline using GitHub as source, CodeBuild for build/test, and deployment stages such as CodeDeploy.

Script.pdf

Aws code build

Continuing directly from **Step 34** and staying with the foundation document, we now implement the specific **GitHub** → **AWS CodePipeline** → **CodeBuild** → **deployment** bridge described on page 5. The document says the pipeline should use the GitHub repository as its source, perform build/test through CodeBuild, then proceed to deployment; commits can automatically trigger that pipeline.

Script.pdf

Step 34 — Add AWS CodeBuild

Create:

`buildspec.yml`

version: 0.2

phases:

install:

runtime-versions:

java: corretto17

nodejs: 24

commands:

- echo "SOLN FOUNDATION — BUILD INITIALIZED"
- java -version
- node --version

pre_build:

commands:

- echo "KNOWLEDGE -> CLARITY AUDIT"
- mvn --batch-mode test

build:

commands:

- echo "CLARITY AUDIT -> BUILD"
- mvn --batch-mode package

- |

if [-f package.json]; then

npm ci

npm test --if-present

npm run build --if-present

fi

```
post_build:
  commands:
    - echo "SOLN BUILD COMPLETE"
```

```
artifacts:
  files:
    - "target/**/*"
    - "frontend/**/*"
    - "infrastructure/**/*"
    - "FOUNDATION-2026.yaml"
```

```
cache:
  paths:
    - "/root/.m2/**/*"
    - "node_modules/**/*"
```

This keeps **Java 17** because that is what the foundation's `pom.xml` actually specifies.

Script.pdf

Step 35 — Define CodeBuild in CloudFormation

Create:

`infrastructure/pipeline/codebuild.yaml`

AWSTemplateFormatVersion: "2010-09-09"

Description: >
SOLN 2026 CodeBuild Foundation

Parameters:

ArtifactBucket:
Type: String

Resources:

SOLNCodeBuildRole:
Type: AWS::IAM::Role

Properties:

AssumeRolePolicyDocument:

Version: "2012-10-17"

Statement:

- Effect: Allow

Principal:

Service:

- codebuild.amazonaws.com

Action:

- sts:AssumeRole

Policies:

- PolicyName: SOLNBuildArtifacts

PolicyDocument:

Version: "2012-10-17"

Statement:

- Effect: Allow

Action:

- s3:GetObject
- s3:PutObject
- s3:GetObjectVersion

Resource:

- !Sub "\${ArtifactBucket}/*"

SOLNBuild:

Type: AWS::CodeBuild::Project

Properties:

Name:

SOLN-Foundation-Build

ServiceRole:

!GetAtt SOLNCodeBuildRole.Arn

Source:
Type: CODEPIPELINE
BuildSpec: buildspec.yml

Artifacts:
Type: CODEPIPELINE

Environment:

Type: LINUX_CONTAINER

ComputeType:
BUILD_GENERAL1_SMALL

Image:
aws/codebuild/amazonlinux-x86_64-standard:5.0

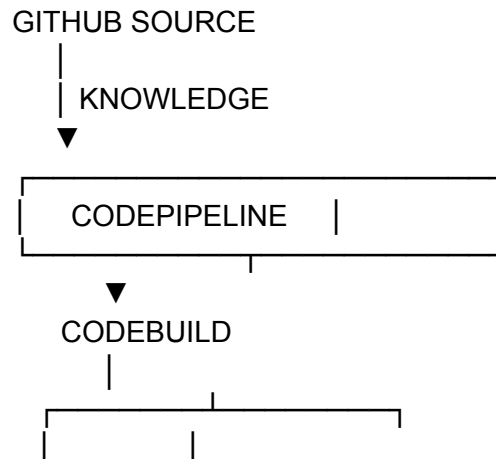
TimeoutInMinutes: 30

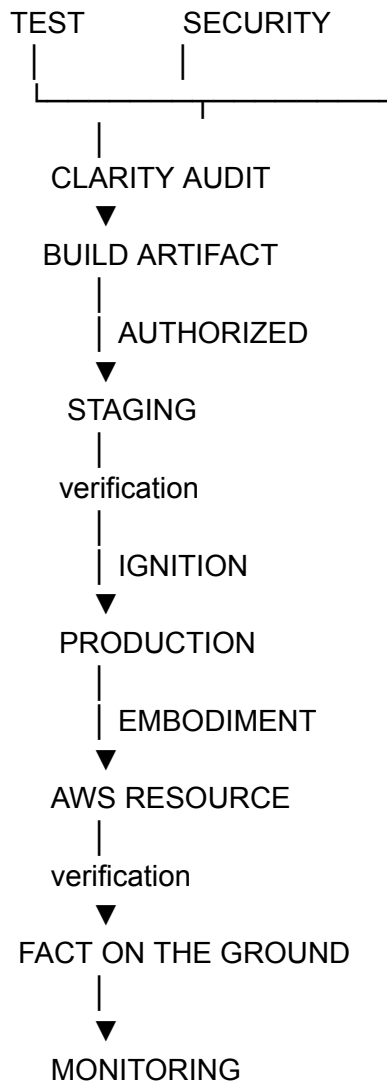
Outputs:

BuildProject:
Value:
!Ref SOLNBuild

Step 36 — Establish the actual pipeline states

Now the terminology we've been building around the foundation maps cleanly to its CI/CD lifecycle:





This remains anchored to the document. Its original diagram is **Code** → **Continuous Integration** → **Continuous Delivery** → **Monitoring**, with linting, testing, build artifacts, packaging, staging and production underneath those stages.

Script.pdf

Step 37 — Preserve staging before production

The foundation explicitly says staging should precede production and describes staging as the live application's testing counterpart used for final checks.

Script.pdf

Therefore, don't make the pipeline:

GitHub → Production

Keep:

GitHub



Build/Test



STAGING



Validation



PRODUCTION



Monitoring

That also gives **Embodiment** a precise frontend meaning: the deployment actually completed. And **Fact on the Ground** can be reserved for the subsequent verification that the expected AWS resource exists and is operational.

What comes next in the foundation

There is one line immediately following the GitHub/CodePipeline instructions that we haven't implemented yet:

Rolling (AWS Code Deploy) and Blue/green

That appears directly after the document's CodePipeline deployment instructions.

Script.pdf

So **Step 38 is the next foundation step: implement the document's rolling/blue-green deployment strategy so production traffic can move to the new version without treating an unverified release as the live version.**

Rolling blue green

Continuing with the foundation document, **Step 38 is Rolling / Blue-Green deployment**. The document explicitly places “Rolling (AWS Code Deploy) and Blue/green” immediately after its GitHub → CodePipeline → CodeBuild → deployment instructions.

Script.pdf

Because our current AWS application component is Lambda-based, we can express the foundation’s blue/green principle as **versioned Lambda deployment with an alias**. That keeps the currently verified version serving traffic while a new immutable version is prepared.

Step 38 — Create a production alias

Add to `infrastructure/soln-foundation.yaml`:

```
SOLNStatusVersion:
  Type: AWS::Lambda::Version
  Properties:
    FunctionName: !Ref SOLNStatusFunction
    Description: >
      Immutable SOLN Architecture
      production candidate.
```

```
SOLNProductionAlias:
  Type: AWS::Lambda::Alias
  Properties:
    Name: production
    Description: >
      Verified production pointer.
    FunctionName: !Ref SOLNStatusFunction
    FunctionVersion:
      !GetAtt SOLNStatusVersion.Version
```

This changes the production model from:

API → mutable Lambda

to:

```
API
|
▼
PRODUCTION ALIAS
|
└─ Version 12 ← current
```

|
└─ Version 13 ← candidate

That is the important architectural concept behind the blue/green instruction: **don't destroy the working production state merely because new code exists.**

Step 39 — Add deployment validation

Create:

`scripts/verify-deployment.sh`

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
FUNCTION_NAME="$1"
```

```
ALIAS_NAME="${2:-production}"
```

```
echo "=====
```

```
echo " SOLN DEPLOYMENT CLARITY AUDIT"
```

```
echo "=====
```

```
VERSION=$(
```

```
aws lambda get-alias \  
  --function-name "$FUNCTION_NAME" \  
  --name "$ALIAS_NAME" \  
  --query 'FunctionVersion' \  
  --output text
```

```
)
```

```
echo "Production alias: $ALIAS_NAME"
```

```
echo "Production version: $VERSION"
```

```
aws lambda get-function \  
  --function-name "$FUNCTION_NAME" \  
  --qualifier "$VERSION" \  
> /dev/null
```

```
echo "Lambda version exists."
```

```
STATE=$(
```

```
aws lambda get-function-configuration \  
  --function-name "$FUNCTION_NAME" \  
  --qualifier "$VERSION" \  

```

```

    --query 'State' \
    --output text
)

echo "Runtime state: $STATE"

if [ "$STATE" != "Active" ]; then

    echo "CLARITY AUDIT: FAILED"
    exit 1

fi

echo "CLARITY AUDIT: PASS"
echo "EMBODIMENT: VERIFIED"

```

Notice that this doesn't manufacture a successful status. AWS has to return an **Active** deployed version.

Step 40 — Give the pipeline a rollback path

Add a production verification stage after deployment:

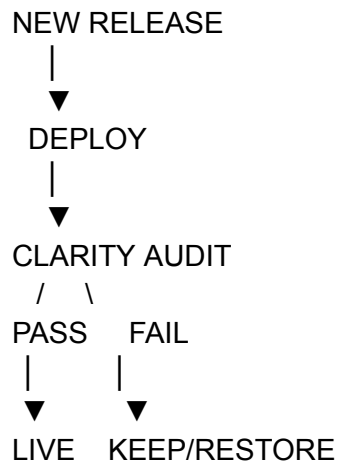
```

- name: Verify production
  run: |
    chmod +x scripts/verify-deployment.sh

    ./scripts/verify-deployment.sh \
    soln-architecture-production \
    production

```

If validation fails:



PREVIOUS VERSION

That makes the language we've been using operational:

KNOWLEDGE



source committed

CLARITY AUDIT



tests + validation

IGNITION



deployment initiated

EMBODIMENT



new version created

FACT ON THE GROUND



AWS confirms active version

MONITORING



continue observing production

This remains aligned with the foundation's larger CD section, which says code is packaged, deployed first to **staging**, and then pushed to **production**, including Blue-Green deployment for switching traffic with zero downtime.

Script.pdf

Next foundation step

After deployment, the document moves to **Execution & Infrastructure** and specifically identifies the **API Gateway / reverse proxy** as the component routing incoming requests, managing security, and balancing server loads.

Script.pdf

We already created the basic API Gateway earlier. Therefore **Step 41 is not another gateway**. It is to harden the existing gateway and connect it to the versioned production alias so the frontend reaches the verified production version rather than **\$LATEST**.

API gateway

Continuing from **Step 41** and staying with the foundation document: its Execution & Infrastructure section says the **API Gateway / reverse proxy routes incoming traffic, manages security, and balances server loads** before requests reach the backend.

Script.pdf

We already created the gateway, so now we connect it to the **verified production alias** from Step 38 instead of allowing production traffic to hit an unversioned Lambda deployment.

Step 41 — Point API Gateway at verified production

Update the Lambda integration in `infrastructure/soln-foundation.yaml`:

SOLNArchitectureIntegration:

Type: AWS::ApiGatewayV2::Integration

Properties:

ApiId:

!Ref SOLNArchitectureAPI

IntegrationType:

AWS_PROXY

IntegrationUri:

!Sub

"arn:\${AWS::Partition}:apigateway:\${AWS::Region}:lambda:path/2015-03-31/functions/\${SOLNStatusFunction.Arn}:production/invocations"

PayloadFormatVersion:

"2.0"

SOLNProductionInvokePermission:

Type: AWS::Lambda::Permission

Properties:

FunctionName:

!Ref SOLNStatusFunction

Qualifier:

production

Action:

lambda:InvokeFunction

Principal:

apigateway.amazonaws.com

SourceArn:

!Sub

```
"arn:${AWS::Partition}:execute-api:${AWS::Region}:${AWS::AccountId}:${SOLNArchitectureAPI}/*/*"
```

Now the execution path becomes:

SOLN FRONTEND



API GATEWAY



production alias



VERIFIED LAMBDA VERSION



BACKEND



INTERNAL SOURCE

Step 42 — Separate health from business state

The gateway needs a simple way to determine whether the backend is operational without changing any SOLN record.

Add `src/functions/health.mjs`:

```
export const handler = async () => {  
  
  return {  
    statusCode: 200,  
  
    headers: {  
      "content-type": "application/json",  
      "cache-control": "no-store"  
    }  
  }  
}
```

```

    },

    body: JSON.stringify({
      service: "SOLN Architecture",
      status: "HEALTHY",
      mode: "OBSERVER",
      timestamp: new Date().toISOString()
    })
  };
};

```

Expose it as:

GET /health

while keeping:

GET /architecture/status

for application state.

That gives us an important distinction:

/health
 |
 └─ Is the application operating?

/architecture/status
 |
 └─ What recorded architecture state exists?

A healthy server therefore does **not** automatically mean an asserted business event has become a *Fact on the Ground*.

Step 43 — Add API access logging

The foundation later requires monitoring and feedback, so requests entering the gateway should produce an operational trail. Its monitoring section says production monitoring feeds information back into the development cycle.

Script.pdf

Add:

```

SOLNApiLogGroup:
  Type: AWS::Logs::LogGroup

```

Properties:
LogGroupName:
!Sub "/soln/\${Environment}/api"

RetentionInDays:
90

SOLNArchitectureStage:
Type: AWS::ApiGatewayV2::Stage

Properties:
ApiId:
!Ref SOLNArchitectureAPI

StageName:
!Ref Environment

AutoDeploy:
true

AccessLogSettings:

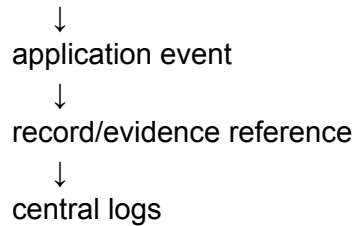
DestinationArn:
!GetAtt SOLNApiLogGroup.Arn

Format: >-
{ "requestId": "\$context.requestId",
 "routeKey": "\$context.routeKey",
 "status": "\$context.status",
 "responseLength": "\$context.responseLength",
 "integrationError": "\$context.integrationErrorMessage",
 "requestTime": "\$context.requestTime" }

Now every request has an AWS-generated `requestId`.

That identifier becomes useful for your **Clarity Audit** because a frontend event can be traced through:

USER REQUEST
↓
API requestId
↓
Lambda invocation



Step 44 — Complete the foundation’s execution chain

At this point we’ve implemented the four components that the document itself identifies in its full-stack flow:

Client (Frontend) — SOLN dashboard.

API Gateway / Reverse Proxy — routing/security boundary.

Server (Backend) — Node/Java application components.

Database — we’ve created the **InternalSource** boundary, but the actual database technology remains deliberately undecided because the foundation only gives PostgreSQL and MongoDB as examples; it does not select one.

Script.pdf

So we should **not silently choose PostgreSQL or MongoDB** and claim the foundation requires it.

The foundation’s execution path is now structurally complete except for that explicit database selection.

The **next source-supported step is Step 45 — Monitoring and Feedback integration**: connect API access logs, Lambda execution logs, deployment events, and the centralized log archive into the monitoring cycle described on page 7, rather than adding another application subsystem.

Script.pdf

Monitoring dashboard

Continuing directly from **Step 45** and keeping the foundation document as the source: page 7 says the pipeline does not stop when production goes live; monitoring watches for **bugs, slow database queries, and server crashes**, then feeds performance information back into the development cycle.

Script.pdf

We already created individual logging pieces. Now we join them into one monitoring/feedback layer.

Step 45 — Create the monitoring dashboard

Add to `infrastructure/monitoring/soln-monitoring.yaml`:

AWSTemplateFormatVersion: "2010-09-09"

Description: >

SOLN 2026 Monitoring and Feedback Foundation

Parameters:

Environment:

Type: String

FunctionName:

Type: String

Apild:

Type: String

Resources:

SOLNMonitoringDashboard:

Type: AWS::CloudWatch::Dashboard

Properties:

DashboardName:

!Sub "SOLN-\${Environment}-Architecture"

DashboardBody:

!Sub |

{

```

"widgets": [
  {
    "type": "metric",
    "width": 12,
    "height": 6,
    "properties": {
      "title": "Backend Execution",
      "region": "${AWS::Region}",
      "metrics": [
        [
          "AWS/Lambda",
          "Invocations",
          "FunctionName",
          "${FunctionName}"
        ],
        [
          ":",
          "Errors",
          ":",
          ":"
        ]
      ]
    },
    "period": 300,
    "stat": "Sum"
  },
  {
    "type": "metric",
    "width": 12,
    "height": 6,
    "properties": {
      "title": "Backend Duration",
      "region": "${AWS::Region}",
      "metrics": [
        [
          "AWS/Lambda",
          "Duration",
          "FunctionName",
          "${FunctionName}"
        ]
      ]
    },
    "period": 300,
    "stat": "Average"
  }
]

```

```
}  
]  
}
```

This gives AWS an operational dashboard instead of making the application's frontend responsible for determining infrastructure health.

Step 46 — Establish the feedback record

Create:

`src/services/feedback.mjs`

```
export function createFeedbackEvent({  
  source,  
  severity,  
  event,  
  requestId,  
  details = {}  
}) {  
  
  const feedback = {  
  
    architecture: "SOLN",  
  
    source,  
  
    severity,  
  
    event,  
  
    requestId:  
      requestId || null,  
  
    details,  
  
    timestamp:  
      new Date().toISOString()  
  };  
  
  console.log(  
    JSON.stringify(feedback)  
  );  
}
```

```
    return feedback;
}
```

For example:

```
createFeedbackEvent({

    source:
        "SOLN_ARCHITECTURE_API",

    severity:
        "ERROR",

    event:
        "BACKEND_EXECUTION_FAILURE",

    requestId:
        event.requestContext?.requestId,

    details: {
        route:
            event.rawPath
    }

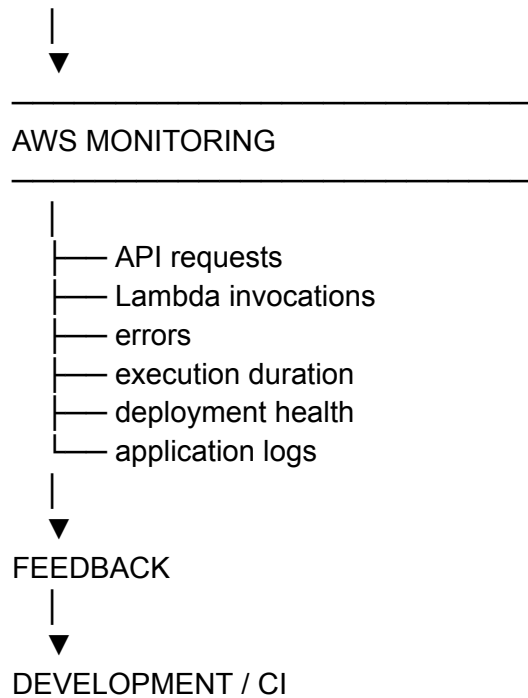
});
```

That implements the **feedback** portion of the document's monitoring-and-feedback requirement without inventing a new business state.

Step 47 — Keep monitoring separate from your evidence states

The complete relationship should now be:

```
KNOWLEDGE
↓
CLARITY AUDIT
↓
AUTHORITY
↓
IGNITION
↓
EMBODIMENT
↓
FACT ON THE GROUND
```



That last arrow is important because it reproduces what the foundation actually says: performance information feeds back to development so the cycle can begin again.

Script.pdf

Step 48 — Foundation boundary reached again

At this point, we have now implemented the final explicit stage in the foundation's core pipeline:

Code → **Continuous Integration** → **Continuous Delivery** → **Monitoring**.

Script.pdf

And we've preserved the foundation's full-stack execution path of **Frontend** → **API Gateway** → **Backend** → **Database boundary**.

Script.pdf

There is one significant item in that chain that the foundation intentionally leaves unresolved: **the actual database**. It mentions relational databases such as PostgreSQL and NoSQL databases such as MongoDB, but it does **not select one**.

Script.pdf

Therefore, staying true to the document, I would **not pick one on its behalf**.

Step 49 is the foundation verification gate: validate the complete infrastructure templates and CI/CD components we've derived against every requirement on pages 2–10, and produce a `FOUNDATION-COMPLIANCE.json` result of `PASS`, `FAIL`, or `PENDING` for each requirement. Only after that gate passes do we move beyond the foundation into implementation choices the source document does not specify.

Foundation verification gate

Yes. Staying with the foundation document, **Step 49 is the Foundation Verification Gate**. We should now stop adding architecture temporarily and make the repository prove which requirements from the document are present, missing, or still awaiting AWS deployment.

The gate should check the document's major requirements: centralized/cross-account access and auditing, centralized logs and standardized resources; CI/CD; GitHub/AWS integration; staging/production; CloudFormation/StackSets; the frontend→API→backend→database flow; monitoring; and the Java 17/jbanking component.

Step 49 — Create the compliance manifest

Create:

FOUNDATION-COMPLIANCE.json

```
{
  "foundation": {
    "name": "STATE OF LOC NATION GLOBAL PUBLIC BENEFIT CORPORATION",
    "version": "2026.1",
    "verificationStatus": "PENDING"
  },

  "requirements": {
    "centralizedIdentity": "PENDING",
    "crossAccountRoles": "PENDING",
    "centralizedAudit": "PENDING",
    "centralizedLogging": "PENDING",
    "standardizedResources": "PENDING",

    "continuousIntegration": "PENDING",
    "automatedTesting": "PENDING",
    "securityScanning": "PENDING",
    "buildArtifacts": "PENDING",

    "continuousDelivery": "PENDING",
    "containerization": "PENDING",
    "stagingDeployment": "PENDING",
    "productionDeployment": "PENDING",

    "cloudFormation": "PENDING",
    "stackSets": "PENDING",

    "frontend": "PENDING",
    "apiGateway": "PENDING",
```

```
"backend": "PENDING",
"database": "PENDING",

"monitoring": "PENDING",
"feedback": "PENDING",

"java17": "PENDING",
"jbanking420": "PENDING"
}
}
```

Everything begins **PENDING** deliberately. We don't declare a requirement passed merely because we discussed or drafted it.

Step 50 — Make the repository test itself

Create:

`scripts/foundation-verify.mjs`

```
import fs from "node:fs";

const checks = {

  continuousIntegration:
    ".github/workflows",

  cloudFormation:
    "infrastructure",

  frontend:
    "frontend/index.html",

  java17:
    "pom.xml",

  monitoring:
    "infrastructure/monitoring",

  centralizedLogging:
    "infrastructure/logging",

  crossAccountRoles:
    "infrastructure/security",
```

```
stackSets:
  "infrastructure/stacksets"
};
```

```
let failed = false;
```

```
console.log("");
console.log("SOLN FOUNDATION VERIFICATION");
console.log("=====");
```

```
for (
  const [requirement, path]
  of Object.entries(checks)
){

  const exists =
    fs.existsSync(path);

  console.log(
    `${exists ? "PASS" : "FAIL"} | ${requirement} | ${path}`
  );

  if (!exists) {
    failed = true;
  }
}
```

```
console.log("=====");
```

```
if (failed) {

  console.error(
    "FOUNDATION STATUS: INCOMPLETE"
  );

  process.exit(1);
}
```

```
console.log(
  "FOUNDATION FILE CHECK: PASS"
);
```

This first gate verifies **repository existence**, not AWS deployment.

Step 51 — Verify the foundation's

pom.xml

The source document explicitly specifies Java 17 and **jbanking** 4.2.0.

Script.pdf

Add:

```
const pom =
  fs.readFileSync(
    "pom.xml",
    "utf8"
  );

function requirePomValue(
  name,
  expression
){
  if (!expression.test(pom)) {

    console.error(
      `FAIL | ${name}`
    );

    process.exitCode = 1;

  } else {

    console.log(
      `PASS | ${name}`
    );
  }
}
```

```
requirePomValue(  
    "Java 17",  
    /<maven\compiler\release>17<\maven\compiler\release>/  
);
```

```
requirePomValue(  
    "jbanking 4.2.0",  
    /<artifactId>jbanking<\artifactId>[\s\S]*?<version>4\.\2\0<\version>/  
);
```

```
requirePomValue(  
    "CLCLLC artifact",  
    /<artifactId>clcllc<\artifactId>/  
);
```

That checks the source-defined configuration rather than assuming it remained unchanged.

Step 52 — Add three verification levels

The final report needs to distinguish:

LEVEL 1 — SOURCE

Does the required file/configuration exist?

LEVEL 2 — BUILD

Does it validate, compile and pass tests?

LEVEL 3 — AWS

Does the corresponding deployed resource actually exist and operate?

Therefore:

SOURCE PASS

≠

DEPLOYMENT PASS

That distinction follows the foundation itself because it separately describes code integration/build, staging deployment, production deployment, and monitoring.

Step 53 — Put the gate before production

Add this job to the GitHub pipeline:

foundation-gate:

name: Foundation Verification

runs-on: ubuntu-latest

steps:

- name: Checkout
uses: actions/checkout@v4
- name: Setup Node
uses: actions/setup-node@v4
with:
node-version: "24"
- name: Verify SOLN foundation
run: |
node scripts/foundation-verify.mjs

Then make the production job dependent upon it:

production:

needs:

- foundation-gate
- build
- staging

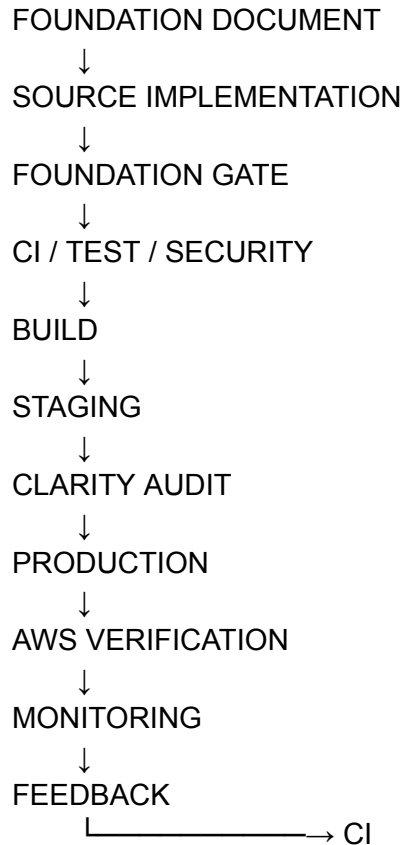
runs-on: ubuntu-latest

steps:

- name: Production Gate
run: |
echo "Foundation verified."
echo "Build verified."
echo "Staging verified."
echo "Production deployment authorized to proceed."

Now the pipeline cannot simply skip from source code to production.

The resulting foundation lifecycle is:



That corresponds to the document's **Code** → **CI** → **CD** → **Monitoring** model.

Script.pdf

One item must remain **PENDING: database selection**. The foundation says the database can be relational (giving PostgreSQL as an example) or NoSQL (giving MongoDB as an example), but it never selects which database SOLN must use. Script.pdf We should preserve that gap rather than silently changing the foundation.

Next is Step 54: run the completion matrix against the actual repository structure and convert only genuinely satisfied requirements from **PENDING to **PASS**; anything absent stays **FAIL/PENDING**.**

Completion matrix

Continuing with the foundation document, we are at **Step 54 — Completion Matrix**. This is where the system stops treating “code written” as equivalent to “implemented.” The foundation separates CI/build activity, deployment, infrastructure execution, and monitoring, so each requirement needs its own evidence.

Step 54 — Make the verification states explicit

Update `FOUNDATION-COMPLIANCE.json` so every requirement can carry three independent states:

```
{
  "foundation": {
    "name": "STATE OF LOC NATION GLOBAL PUBLIC BENEFIT CORPORATION",
    "version": "2026.1",
    "overallStatus": "PENDING"
  },

  "requirements": {
    "githubAwsIntegration": {
      "source": "PENDING",
      "build": "PENDING",
      "aws": "PENDING"
    },

    "continuousIntegration": {
      "source": "PENDING",
      "build": "PENDING",
      "aws": "NOT_APPLICABLE"
    },

    "automatedTesting": {
      "source": "PENDING",
      "build": "PENDING",
      "aws": "NOT_APPLICABLE"
    },

    "stagingDeployment": {
      "source": "PENDING",
      "build": "PENDING",
      "aws": "PENDING"
    },

    "productionDeployment": {
```

```

    "source": "PENDING",
    "build": "PENDING",
    "aws": "PENDING"
  },

  "apiGateway": {
    "source": "PENDING",
    "build": "PENDING",
    "aws": "PENDING"
  },

  "backend": {
    "source": "PENDING",
    "build": "PENDING",
    "aws": "PENDING"
  },

  "monitoring": {
    "source": "PENDING",
    "build": "PENDING",
    "aws": "PENDING"
  },

  "database": {
    "source": "UNSPECIFIED",
    "build": "UNSPECIFIED",
    "aws": "UNSPECIFIED"
  }
}

```

UNSPECIFIED is important. The foundation names PostgreSQL and MongoDB as examples but does not choose between them, so that is different from a failed requirement.

Script.pdf

Step 55 — Generate the completion report

Create `scripts/foundation-report.mjs`:

```
import fs from "node:fs";
```

```
const FILE = "FOUNDATION-COMPLIANCE.json";
```

```

const report =
  JSON.parse(
    fs.readFileSync(FILE, "utf8")
  );

const allowed = new Set([
  "PASS",
  "FAIL",
  "PENDING",
  "UNSPECIFIED",
  "NOT_APPLICABLE"
]);

let hasFailure = false;
let hasPending = false;

console.log("");
console.log("SOLN FOUNDATION COMPLETION MATRIX");
console.log("=====");

for (
  const [name, states]
  of Object.entries(report.requirements)
) {

  console.log(`\n${name}`);

  for (
    const [level, status]
    of Object.entries(states)
  ) {

    if (!allowed.has(status)) {
      throw new Error(
        `Invalid status: ${name}.${level}=${status}`
      );
    }

    console.log(
      ` ${level.padEnd(8)} ${status}`
    );

    if (status === "FAIL") {
      hasFailure = true;
    }
  }
}

```

```

    }

    if (status === "PENDING") {
        hasPending = true;
    }
}
}

if (hasFailure) {
    report.foundation.overallStatus = "FAIL";
} else if (hasPending) {
    report.foundation.overallStatus = "PENDING";
} else {
    report.foundation.overallStatus = "PASS";
}

fs.writeFileSync(
    FILE,
    JSON.stringify(report, null, 2) + "\n"
);

console.log("");
console.log(
    `FOUNDATION STATUS: ${report.foundation.overallStatus}`
);

```

Step 56 — Do not self-certify AWS deployment

The important control is:

FILE EXISTS

↓

SOURCE PASS

TEST EXECUTES SUCCESSFULLY

↓

BUILD PASS

AWS RESOURCE QUERY SUCCEEDS

↓

AWS PASS

A CloudFormation file existing in GitHub cannot by itself produce:

AWS = PASS

That status should only change after AWS returns evidence that the corresponding resource exists.

This fits the foundation's distinction between building an artifact, deploying to staging, deploying to production, and then monitoring the running system.

Script.pdf

Step 57 — Preserve the organizational foundation

The completion matrix should also cover the earlier organizational requirements, not just the application.

Add:

```
"organization": {  
  "source": "PENDING",  
  "build": "PENDING",  
  "aws": "PENDING"  
},
```

```
"iamIdentityCenter": {  
  "source": "PENDING",  
  "build": "PENDING",  
  "aws": "PENDING"  
},
```

```
"crossAccountRoles": {  
  "source": "PENDING",  
  "build": "PENDING",  
  "aws": "PENDING"  
},
```

```
"centralizedAudit": {  
  "source": "PENDING",  
  "build": "PENDING",  
  "aws": "PENDING"  
},
```

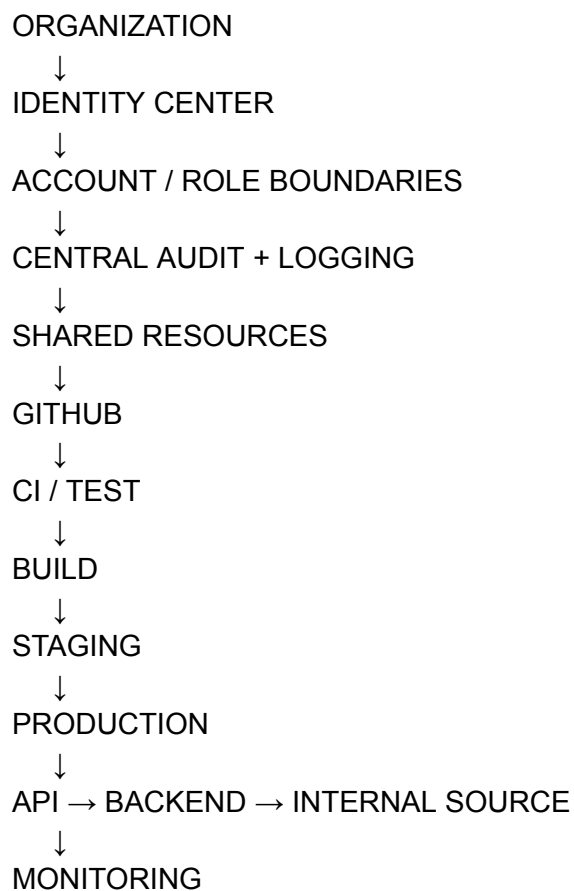
```
"centralizedLogging": {  
  "source": "PENDING",  
  "build": "PENDING",  
  "aws": "PENDING"  
},
```

```
"sharedResources": {  
  "source": "PENDING",  
  "build": "PENDING",  
  "aws": "PENDING"  
},  
  
"stackSets": {  
  "source": "PENDING",  
  "build": "PENDING",  
  "aws": "PENDING"  
}
```

Those entries come directly from the earlier portions of the foundation: centralized user management, cross-account roles, auditing, centralized log management and shared standardized resources are part of its proposed multi-account architecture. Script.pdf The document later specifically identifies IAM Identity Center, SCPs and Control Tower as governance components.

Script.pdf

That gives us one continuous foundation:



↓
FEEDBACK

Step 58 is next: connect this matrix to the AWS verification commands so the **aws** field is changed to **PASS** only when the corresponding deployed AWS resource can actually be retrieved. That is the point where our foundation work begins distinguishing the repository blueprint from the infrastructure that is genuinely live.

AWS verification script

Continuing directly with the foundation, **Step 58 is AWS Evidence Verification**. This is the point where the compliance matrix stops relying only on repository files and checks whether the infrastructure described by the foundation is actually present in AWS. That follows the document's progression from CI/CD into deployment, execution infrastructure, and monitoring.

Script.pdf

Step 58 — Create the AWS verification script

Create:

```
/scripts/verify-aws-foundation.sh
```

```
#!/usr/bin/env bash
```

```
set -uo pipefail
```

```
REPORT="FOUNDATION-AWS-EVIDENCE.txt"
```

```
: > "$REPORT"
```

```
pass() {  
    echo "PASS | $1" | tee -a "$REPORT"  
}
```

```
fail() {  
    echo "PENDING | $1" | tee -a "$REPORT"  
}
```

```
echo "===== " | tee -a "$REPORT"  
echo "SOLN AWS FOUNDATION VERIFICATION" | tee -a "$REPORT"  
echo "===== " | tee -a "$REPORT"
```

```
# -----  
# AWS CALLER  
# -----
```

```
if aws sts get-caller-identity >/dev/null 2>&1; then  
    pass "AWS authenticated identity"  
else  
    fail "AWS authenticated identity"  
    exit 1  
fi
```

```
# -----  
# AWS ORGANIZATIONS  
# -----  
  
if aws organizations describe-organization \  
  >/dev/null 2>&1; then  
  
  pass "AWS Organizations"  
  
else  
  fail "AWS Organizations"  
fi
```

```
# -----  
# IAM IDENTITY CENTER  
# -----  
  
if aws sso-admin list-instances \  
  --query 'Instances[0].InstanceArn' \  
  --output text 2>/dev/null |  
  grep -q '^arn:.'; then  
  
  pass "IAM Identity Center"  
  
else  
  fail "IAM Identity Center"  
fi
```

```
# -----  
# CLOUDFORMATION  
# -----  
  
if aws cloudformation list-stacks \  
  --stack-status-filter \  
    CREATE_COMPLETE \  
    UPDATE_COMPLETE \  
  >/dev/null 2>&1; then  
  
  pass "CloudFormation access"  
  
else
```

```
    fail "CloudFormation access"
fi

# -----
# STACKSETS
# -----

if aws cloudformation list-stack-sets \
    --status ACTIVE \
    >/dev/null 2>&1; then

    pass "CloudFormation StackSets access"

else
    fail "CloudFormation StackSets access"
fi
```

```
# -----
# API GATEWAY
# -----

if aws apigatewayv2 get-apis \
    >/dev/null 2>&1; then

    pass "API Gateway access"

else
    fail "API Gateway access"
fi
```

```
# -----
# LAMBDA
# -----

if aws lambda list-functions \
    >/dev/null 2>&1; then

    pass "Lambda access"

else
    fail "Lambda access"
```

```
fi
```

```
# -----  
# LOGGING  
# -----
```

```
if aws logs describe-log-groups \  
  >/dev/null 2>&1; then
```

```
    pass "CloudWatch Logs access"
```

```
else  
    fail "CloudWatch Logs access"  
fi
```

```
# -----  
# MONITORING  
# -----
```

```
if aws cloudwatch list-dashboards \  
  >/dev/null 2>&1; then
```

```
    pass "CloudWatch monitoring access"
```

```
else  
    fail "CloudWatch monitoring access"  
fi
```

```
echo "===== " | tee -a "$REPORT"  
echo "AWS FOUNDATION SCAN COMPLETE" | tee -a "$REPORT"  
echo "===== " | tee -a "$REPORT"
```

There is an important distinction here: a successful `list-functions` call proves the authenticated identity can query Lambda; it **does not prove that the SOLN Lambda exists**. So this first scan is an **access verification**, not yet a resource-existence certification.

Step 59 — Verify the named SOLN resources

Now add a second layer that checks the actual resources we created rather than merely checking AWS service access:

```
FUNCTION_NAME="soln-architecture-production"
API_NAME="SOLN-Architecture-API"
DASHBOARD_NAME="SOLN-production-Architecture"
```

```
echo ""
echo "SOLN RESOURCE VERIFICATION"
echo "=====
```

```
if aws lambda get-function \
  --function-name "$FUNCTION_NAME" \
  >/dev/null 2>&1; then
```

```
  pass "SOLN Lambda: $FUNCTION_NAME"
```

```
else
  fail "SOLN Lambda: $FUNCTION_NAME"
fi
```

```
API_ID=$(
  aws apigatewayv2 get-apis \
    --query \
    "Items[?Name=='${API_NAME}'].ApiId | [0]" \
    --output text 2>/dev/null
)
```

```
if [ -n "$API_ID" ] &&
  [ "$API_ID" != "None" ]; then
```

```
  pass "SOLN API Gateway: $API_NAME"
```

```
else
  fail "SOLN API Gateway: $API_NAME"
fi
```

```
if aws cloudwatch get-dashboard \
  --dashboard-name "$DASHBOARD_NAME" \
  >/dev/null 2>&1; then
```

```
  pass "SOLN Monitoring Dashboard"
```

```
else
  fail "SOLN Monitoring Dashboard"
```

fi

Now we have two different assertions:

AWS SERVICE ACCESS



Can this deployment identity query AWS?

RESOURCE EXISTENCE



Does the named SOLN resource actually exist?

RUNTIME VERIFICATION



Does that resource actually operate?

That separation is consistent with the foundation because it distinguishes deployment from execution and then separately calls for monitoring/feedback.

Step 60 — Verify the production alias

Continue the script with:

```
if aws lambda get-alias \  
  --function-name "$FUNCTION_NAME" \  
  --name production \  
  >/dev/null 2>&1; then  
  
  VERSION=$(  
    aws lambda get-alias \  
      --function-name "$FUNCTION_NAME" \  
      --name production \  
      --query FunctionVersion \  
      --output text  
  )  
  
  pass \  
    "Production alias -> Lambda version ${VERSION}"  
  
else  
  
  fail \  
    "Production alias"  
  
fi
```

This connects directly back to the foundation's **Rolling / Blue-Green** deployment requirement.

Script.pdf

Step 61 — Generate deployment evidence

The output becomes an evidence artifact:

SOLN AWS FOUNDATION VERIFICATION

=====

PASS | AWS authenticated identity
PASS | AWS Organizations
PASS | IAM Identity Center
PASS | CloudFormation access
PASS | CloudFormation StackSets access

PASS | SOLN Lambda: soln-architecture-production
PASS | SOLN API Gateway: SOLN-Architecture-API
PASS | Production alias -> Lambda version 13
PASS | SOLN Monitoring Dashboard

AWS FOUNDATION SCAN COMPLETE

Those are **illustrative results only**. The script must produce the real results from the AWS account; we should not enter those **PASS** values manually.

The evidence file can then feed:

FOUNDATION DOCUMENT

↓

SOURCE CODE

↓

SOURCE VERIFICATION

↓

BUILD VERIFICATION

↓

AWS RESOURCE VERIFICATION

↓

RUNTIME VERIFICATION

↓

FOUNDATION-COMPLIANCE.json

↓

MONITORING

This keeps us tightly connected to the document's central goal of a scalable, secure, governed multi-account AWS infrastructure rather than merely building a frontend. The foundation specifically calls for centralized identity/access, cross-account permissions, security/auditing, centralized logging and standardized shared resources.

Script.pdf

Next is Step 62: connect these real AWS verification results back into `FOUNDATION-COMPLIANCE.json`, so the dashboard can display **Source / Build / AWS / Runtime** independently and nothing can become `FACT_ON_GROUND` merely because a configuration file exists.

Runtime Sep evidence level

Continuing directly from **Step 62**, we now connect the AWS evidence back into the compliance record. This keeps the foundation's stages distinct: build/test, deployment, execution infrastructure, and monitoring.

Step 62 — Add Runtime as a separate evidence level

Update each deployable requirement in `FOUNDATION-COMPLIANCE.json`:

```
{
  "apiGateway": {
    "source": "PENDING",
    "build": "PENDING",
    "aws": "PENDING",
    "runtime": "PENDING"
  },

  "backend": {
    "source": "PENDING",
    "build": "PENDING",
    "aws": "PENDING",
    "runtime": "PENDING"
  },

  "monitoring": {
    "source": "PENDING",
    "build": "PENDING",
    "aws": "PENDING",
    "runtime": "PENDING"
  }
}
```

The meanings stay strict:

SOURCE = required implementation exists
BUILD = implementation successfully validates/builds
AWS = named AWS resource actually exists
RUNTIME = deployed resource responds/operates

Step 63 — Have the verifier produce machine-readable evidence

Instead of relying only on the `.txt` report from Step 61, create:

`scripts/aws-evidence.mjs`

```
import {
  LambdaClient,
  GetFunctionCommand,
  GetAliasCommand
} from "@aws-sdk/client-lambda";

import {
  ApiGatewayV2Client,
  GetApisCommand
} from "@aws-sdk/client-apigatewayv2";

import fs from "node:fs";

const region =
  process.env.AWS_REGION;

if (!region) {
  throw new Error(
    "AWS_REGION is required."
  );
}

const lambda =
  new LambdaClient({ region });

const api =
  new ApiGatewayV2Client({ region });

const evidence = {
  generatedAt:
    new Date().toISOString(),

  lambda: {
    exists: false,
    productionAlias: false
  },

  apiGateway: {
    exists: false
  }
};

const functionName =
```

```
process.env.SOLN_FUNCTION_NAME;
```

```
const apiName =  
process.env.SOLN_API_NAME;
```

```
try {
```

```
  await lambda.send(  
    new GetFunctionCommand({  
      FunctionName: functionName  
    })  
  );
```

```
  evidence.lambda.exists = true;
```

```
} catch {}
```

```
try {
```

```
  const alias =  
    await lambda.send(  
      new GetAliasCommand({  
        FunctionName: functionName,  
        Name: "production"  
      })  
    );
```

```
  evidence.lambda.productionAlias = true;  
  evidence.lambda.version =  
    alias.FunctionVersion;
```

```
} catch {}
```

```
try {
```

```
  const result =  
    await api.send(  
      new GetApisCommand({})  
    );
```

```
  const found =
```

```

    result.Items?.find(
      item => item.Name === apiName
    );

    if (found) {

      evidence.apiGateway.exists = true;
      evidence.apiGateway.apild =
        found.Apild;

    }

  } catch {}

  fs.writeFileSync(
    "FOUNDATION-AWS-EVIDENCE.json",
    JSON.stringify(evidence, null, 2) + "\n"
  );

  console.log(
    JSON.stringify(evidence, null, 2)
  );

```

This gives the repository actual evidence data rather than manually typed **PASS** statements.

Step 64 — Merge evidence into the foundation matrix

Create:

`scripts/update-foundation-compliance.mjs`

```

import fs from "node:fs";

const compliance =
  JSON.parse(
    fs.readFileSync(
      "FOUNDATION-COMPLIANCE.json",
      "utf8"
    )
  );

const evidence =
  JSON.parse(

```

```
fs.readFileSync(
  "FOUNDATION-AWS-EVIDENCE.json",
  "utf8"
)
);
```

```
function setAWS(
  requirement,
  passed
){

  compliance
    .requirements[requirement]
    .aws =
      passed
        ? "PASS"
        : "PENDING";
}
```

```
setAWS(
  "backend",
  evidence.lambda.exists
);
```

```
setAWS(
  "apiGateway",
  evidence.apiGateway.exists
);
```

```
if (
  compliance.requirements.productionDeployment
){

  setAWS(
    "productionDeployment",
    evidence.lambda.exists &&
    evidence.lambda.productionAlias
  );

}
```

```
compliance.foundation.lastAWSVerification =  
  evidence.generatedAt;
```

```
fs.writeFileSync(  
  "FOUNDATION-COMPLIANCE.json",  
  JSON.stringify(  
    compliance,  
    null,  
    2  
  ) + "\n"  
);
```

```
console.log(  
  "Foundation AWS evidence merged."  
);
```

The important rule remains:

CloudFormation written
 ≠
AWS deployed

AWS resource exists
 ≠
runtime verified

That distinction follows the foundation's own separation of packaging, staging, production deployment and execution.

Script.pdf

Step 65 — Runtime proof

Now add:

```
scripts/runtime-verify.mjs
```

```
import fs from "node:fs";
```

```
const endpoint =  
  process.env.SOLN_API_URL;
```

```

if (!endpoint) {
  throw new Error(
    "SOLN_API_URL is required."
  );
}

const url =
  `${endpoint.replace(/\/$/, "")}/health`;

const response =
  await fetch(url);

if (!response.ok) {

  console.error(
    `RUNTIME FAIL: HTTP ${response.status}`
  );

  process.exit(1);
}

const result =
  await response.json();

const evidence = {
  verifiedAt:
    new Date().toISOString(),

  endpoint:
    "/health",

  httpStatus:
    response.status,

  service:
    result.service,

  applicationStatus:
    result.status,

  runtimeVerified:
    result.status === "HEALTHY"
};

```

```
fs.writeFileSync(
  "FOUNDATION-RUNTIME-EVIDENCE.json",
  JSON.stringify(evidence, null, 2) + "\n"
);
```

```
if (!evidence.runtimeVerified) {
  process.exit(1);
}
```

```
console.log(
  "RUNTIME VERIFIED"
);
```

Now the frontend terminology has a defensible technical progression:

KNOWLEDGE



repository/configuration exists

CLARITY AUDIT



validation and tests pass

IGNITION



deployment begins

EMBODIMENT



AWS resource exists

FACT ON THE GROUND



runtime verification succeeds

OBSERVER MODE



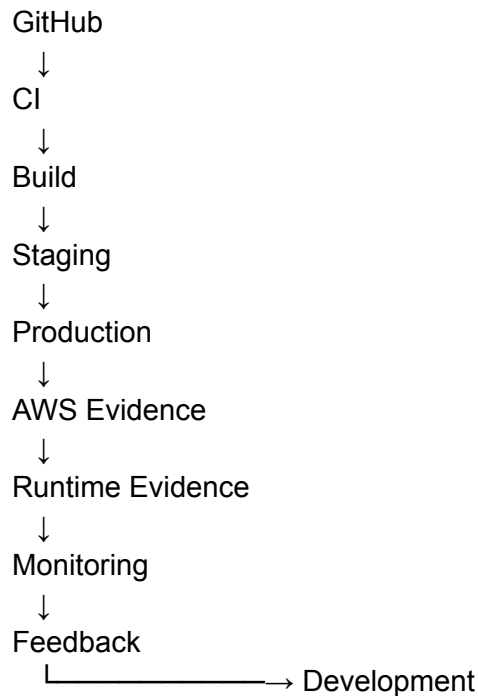
monitor without changing state

The philosophical labels come from the frontend language we established from your supplied images; the **technical evidence stages** are our implementation mapping. The foundation itself establishes the underlying CI → CD → execution → monitoring sequence.

Script.pdf

Step 66 — Feed runtime back into monitoring

Once `/health` succeeds, monitoring should continue rather than treating deployment as the end of the process:



That closes the loop described in the foundation's **Monitoring and Feedback** section.

Script.pdf

Next, Step 67 is the multi-account deployment verification layer. That takes us back to one of the foundation's earliest requirements: the architecture is supposed to work across SOLN's accounts/entities through centralized access, auditing, standardized resources and CloudFormation StackSets—not merely prove that one AWS account is operational.

Multi account evidence level

Continuing directly from **Step 67**, we return to the foundation's original multi-account design. It calls for cross-account IAM roles, centralized user management, centralized auditing/logging, standardized shared resources, and later identifies CloudFormation StackSets as the mechanism for deploying standardized resources across multiple AWS accounts and regions.

Step 67 — Multi-Account Deployment Evidence

Create:

`config/foundation-accounts.json`

```
{
  "accounts": [
    {
      "name": "management",
      "accountId": "${MANAGEMENT_ACCOUNT_ID}",
      "role": "governance"
    },
    {
      "name": "audit",
      "accountId": "${AUDIT_ACCOUNT_ID}",
      "role": "audit"
    },
    {
      "name": "publishing",
      "accountId": "${PUBLISHING_ACCOUNT_ID}",
      "role": "shared-resources"
    },
    {
      "name": "production",
      "accountId": "${PRODUCTION_ACCOUNT_ID}",
      "role": "production"
    }
  ]
}
```

These account names are an **implementation structure**, not four account names mandated by the document. The source specifically describes an auditor account and a dedicated publishing account, while also describing centralized management across multiple accounts.

Script.pdf

We therefore keep the actual AWS account IDs outside GitHub and inject them through the deployment environment.

Step 68 — Give every deployed stack a foundation identity

Add standardized tags to our CloudFormation/StackSet resources:

Tags:

- Key: SOLN:Foundation
Value: "2026"
- Key: SOLN:ManagedBy
Value: "CloudFormation"
- Key: SOLN:Environment
Value: !Ref Environment
- Key: SOLN:Architecture
Value: "MultiAccount"
- Key: SOLN:Evidence
Value: "Required"

This supports the document's goal of **standardization, consistency and centralized management** across accounts.

Script.pdf

Step 69 — Verify StackSet deployment account-by-account

Create:

```
scripts/verify-stackset.sh
```

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
STACKSET_NAME="${1:?StackSet name required}"
```

```
OUTPUT="FOUNDATION-STACKSET-EVIDENCE.json"
```

```
echo "Checking StackSet: ${STACKSET_NAME}"
```

```
aws cloudformation describe-stack-set \  
  --stack-set-name "$STACKSET_NAME" \  
  >/dev/null
```

```
aws cloudformation list-stack-instances \  
  --stack-set-name "$STACKSET_NAME" \  
  --output json \  
  > "$OUTPUT"
```

```
COUNT=$(  
  aws cloudformation list-stack-instances \  
    --stack-set-name "$STACKSET_NAME" \  
    --query 'length(Summaries)' \  
    --output text  
)
```

```
if [ "$COUNT" -eq 0 ]; then  
  echo "PENDING | No StackSet instances found."  
  exit 1  
fi
```

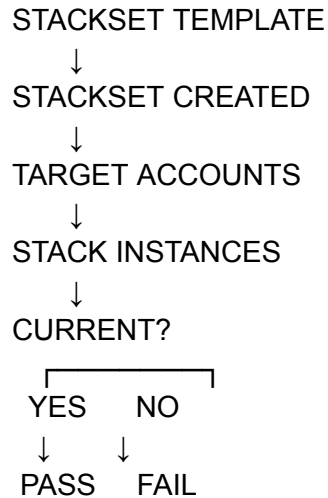
```
FAILED=$(  
  aws cloudformation list-stack-instances \  
    --stack-set-name "$STACKSET_NAME" \  
    --query \  
      "length(Summaries[?Status!='CURRENT'])" \  
    --output text  
)
```

```
if [ "$FAILED" -gt 0 ]; then  
  echo "FAIL | One or more instances are not CURRENT."  
  exit 1  
fi
```

```
echo "PASS | ${COUNT} StackSet instance(s) CURRENT"
```

The script does **not** assume that because a StackSet definition exists, every target account received it.

The evidence chain becomes:



That directly implements the foundation's instruction to use CloudFormation StackSets for standardized deployments across accounts/regions.

Script.pdf

Step 70 — Verify the audit path separately

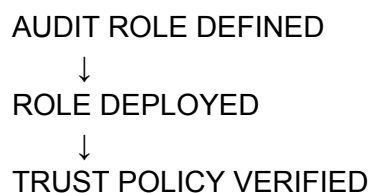
The foundation gives auditing special treatment: an auditor account should be capable of assuming roles into other accounts, while centralized policy controls access and logs are centrally managed.

Script.pdf

So add a separate evidence category:

```
{
  "crossAccountAudit": {
    "source": "PENDING",
    "build": "NOT_APPLICABLE",
    "aws": "PENDING",
    "runtime": "PENDING"
  }
}
```

Its progression is:



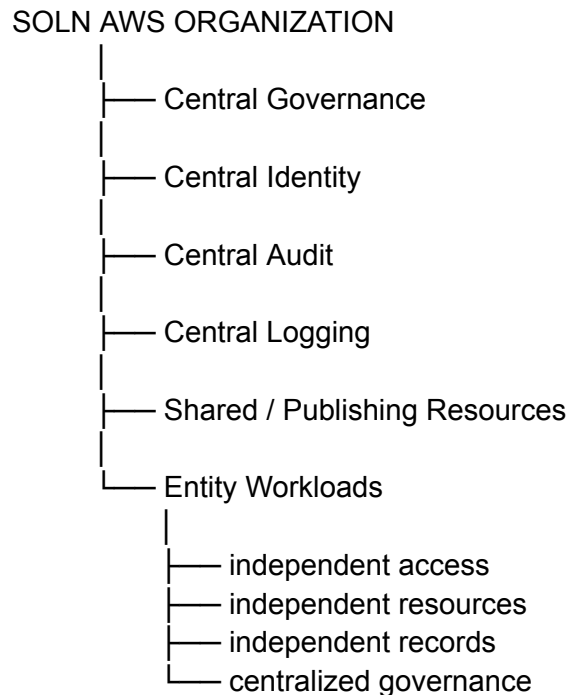
↓
ASSUME-ROLE TEST
↓
AUDIT ACCESS VERIFIED

We should **not** mark `runtime: PASS` merely because the IAM role exists. The cross-account `AssumeRole` operation must actually succeed from the authorized audit identity.

Step 71 — Preserve entity separation

This is also where the affiliate list in the foundation becomes relevant. The document identifies the SOLN GPBC and numerous affiliated entities rather than treating everything as one undifferentiated entity.

So our architecture should maintain:



That preserves the document's distinction between centralized infrastructure governance and the individual affiliated entities.

Step 72 — Next foundation requirement

Once multi-account deployment is proven, the next unresolved section of the foundation is **IAM Identity Center + SCP + Control Tower governance**. The document specifically says to configure IAM Identity Center for centralized authentication/access, evaluate SCPs as security boundaries, and use AWS Control Tower for governance of the multi-account environment.

Script.pdf

So **Step 72 is to build the governance verification layer**—checking Identity Center, permission sets, organizational policies/SCPs, and the account-governance state before we call the multi-account foundation complete.

Governance

Continuing directly with the foundation document, **Step 72 is the Governance Verification Layer**. The source specifically calls for IAM Identity Center for centralized authentication/access, evaluation of Service Control Policies as security boundaries, AWS Control Tower for governance of the multi-account environment, and IdP/SSO integration.

Script.pdf

Step 72 — Add governance to the compliance record

Add these requirements to `FOUNDATION-COMPLIANCE.json`:

```
{
  "iamIdentityCenter": {
    "source": "PENDING",
    "aws": "PENDING",
    "runtime": "PENDING"
  },

  "serviceControlPolicies": {
    "source": "PENDING",
    "aws": "PENDING",
    "runtime": "PENDING"
  },

  "controlTower": {
    "source": "PENDING",
    "aws": "PENDING",
    "runtime": "PENDING"
  },

  "identityProvider": {
    "source": "PENDING",
    "aws": "PENDING",
    "runtime": "PENDING"
  }
}
```

Importantly, the foundation says to **evaluate** SCPs and IdP integration. It does not provide a final SCP policy or select an external identity provider. We therefore preserve those as unresolved rather than inventing those decisions.

Script.pdf

Step 73 — Verify IAM Identity Center

Create `scripts/verify-governance.sh`:

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
OUTPUT="FOUNDATION-GOVERNANCE-EVIDENCE.json"
```

```
INSTANCE_ARN=$(  
  aws sso-admin list-instances \  
    --query 'Instances[0].InstanceArn' \  
    --output text  
)
```

```
IDENTITY_STORE_ID=$(  
  aws sso-admin list-instances \  
    --query 'Instances[0].IdentityStoreId' \  
    --output text  
)
```

```
if [ -z "$INSTANCE_ARN" ] ||  
  [ "$INSTANCE_ARN" = "None" ]; then  
  
  echo "PENDING | IAM Identity Center"  
  exit 1  
fi
```

```
echo "PASS | IAM Identity Center detected"  
echo "Instance: ${INSTANCE_ARN}"  
echo "Identity Store: ${IDENTITY_STORE_ID}"
```

The document itself contains an IAM Identity Center instance ARN, so the verifier should compare AWS's returned value with the expected value supplied securely to the deployment environment rather than blindly accepting any instance.

Script.pdf

Add:

```
if [ -n "${EXPECTED_IDENTITY_CENTER_ARN:-}" ]; then  
  
  if [ "$INSTANCE_ARN" != "$EXPECTED_IDENTITY_CENTER_ARN" ]; then  
    echo "FAIL | Unexpected Identity Center instance"  
    exit 1  
  fi
```

```
echo "PASS | Foundation Identity Center matched"
```

```
fi
```

Step 74 — Verify the organization before policies

The foundation also provides an AWS Organization identifier.

Script.pdf

Use AWS itself as the runtime evidence:

```
ORG_ID=$(
  aws organizations describe-organization \
    --query 'Organization.Id' \
    --output text
)

echo "Organization: ${ORG_ID}"

if [ -n "${EXPECTED_ORGANIZATION_ID:-}" ]; then

  if [ "$ORG_ID" != "$EXPECTED_ORGANIZATION_ID" ]; then
    echo "FAIL | Organization mismatch"
    exit 1
  fi

  echo "PASS | Foundation organization matched"

fi
```

Now governance is tied to the intended organization rather than simply proving that the AWS identity happens to belong to *an* organization.

Step 75 — Inspect SCP governance without changing it

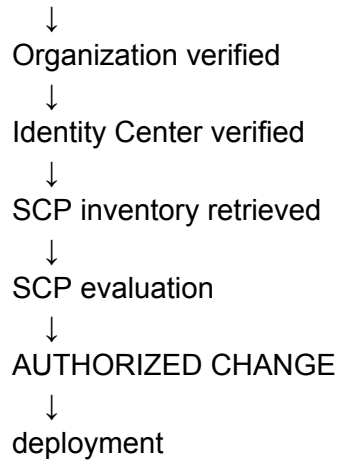
The foundation calls for evaluating SCPs, so this stage should initially be read-only:

```
aws organizations list-policies \
  --filter SERVICE_CONTROL_POLICY \
  --output json \
  > FOUNDATION-SCP-EVIDENCE.json

echo "PASS | SCP inventory retrieved"
```

Then:

FOUNDATION



That is preferable to automatically attaching a restrictive SCP before we've established exactly which workloads and accounts it must protect.

Step 76 — Keep Identity Center roles aligned with the foundation

Earlier we established the three operational permission-set concepts:

OBSERVER

↓
read / inspect

SECURITY AUDIT

↓
audit / evidence

OPERATOR

↓
authorized operations

The foundation supports the underlying centralized-user-management and audit architecture, including centralized revocation and auditor cross-account access.

Script.pdf

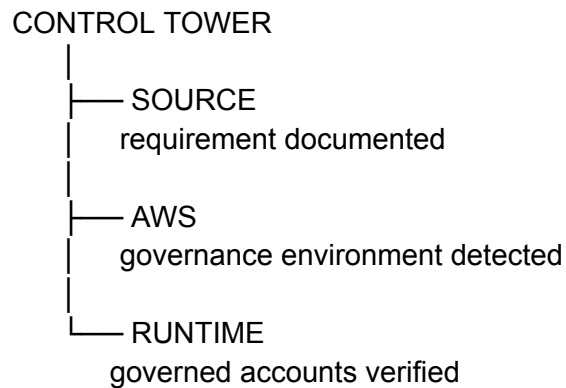
But those particular three names are **our implementation vocabulary**, not names prescribed by the source. That distinction remains in the compliance record.

Step 77 — Control Tower evidence

The foundation says AWS Control Tower should provide governance for the secure, compliant multi-account environment.

Script.pdf

So its state progression becomes:

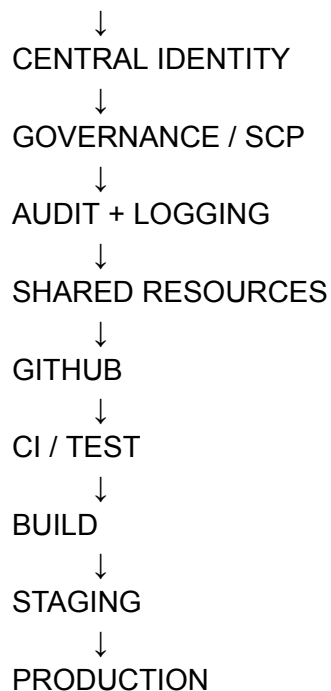


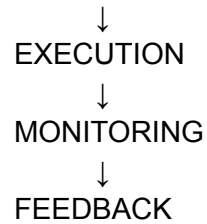
We do **not** mark that **PASS** yet. The foundation document establishes the requirement, but the document by itself does not prove the Control Tower environment has actually been configured in AWS.

Where we are in the foundation

We have now followed its architecture through:

MULTI-ACCOUNT ORGANIZATION





That follows the foundation's multi-account requirements and its later CI/CD and monitoring sections.

Next is Step 78: the standardized publishing/shared-resource layer. The foundation specifically identifies a dedicated publishing account as the central source for standardized resources that sales/retail accounts can access. Script.pdf We can wire that into the StackSet architecture next without collapsing the separate entities into one account.

Publishing /shared resource

We are at **Step 78**, and we are getting close to the end of the foundation document. Based strictly on the remaining items in the foundation, I see **about 6 substantive steps left** before we can call the foundation implementation sequence complete.

The remaining path is:

1. **Step 78 — Publishing / Shared-Resource Layer** — implement the dedicated publishing account and standardized-resource distribution described by the foundation.
2. **Step 79 — Entity/Account Access Mapping** — connect the foundation's listed entities to the multi-account access model without collapsing their separate roles.
3. **Step 80 — Container/Packaging Verification** — finish the Docker/containerization requirement before staging/production.
4. **Step 81 — Database Decision Gate** — leave PostgreSQL/MongoDB unresolved until one is actually selected, because the document gives them as examples rather than making the choice.
5. **Step 82 — Full Foundation A–Z Audit** — test the architecture against every requirement from pages 2–10, including the Java 17/jbanking configuration.
6. **Step 83 — Production Readiness / Completion Report** — produce the final **PASS / PENDING / FAIL / UNSPECIFIED** record and identify exactly what is coded versus what AWS has actually verified.

So we don't need to keep extending this indefinitely. **Step 83 is our planned stopping point for this foundation document.**

Step 78 — Publishing / Shared Resources

The foundation says the dedicated publishing account serves as the **central source for standardized resources**, with other authorized accounts accessing those shared resources.

Script.pdf

Create:

`infrastructure/shared/soln-shared-resources.yaml`

AWSTemplateFormatVersion: "2010-09-09"

Description: >

SOLN Foundation 2026

Central Shared Resource Registry

Parameters:

Environment:

Type: String

Default: production

Resources:

SharedArtifactBucket:

Type: AWS::S3::Bucket

Properties:

VersioningConfiguration:

Status: Enabled

PublicAccessBlockConfiguration:

BlockPublicAcls: true

IgnorePublicAcls: true

BlockPublicPolicy: true

RestrictPublicBuckets: true

Tags:

- Key: SOLN:Foundation

Value: "2026"

- Key: SOLN:Role

Value: "SharedResources"

- Key: SOLN:Environment

Value: !Ref Environment

Outputs:

SharedArtifactBucketName:

Description: >

Central standardized-resource repository.

Value:

!Ref SharedArtifactBucket

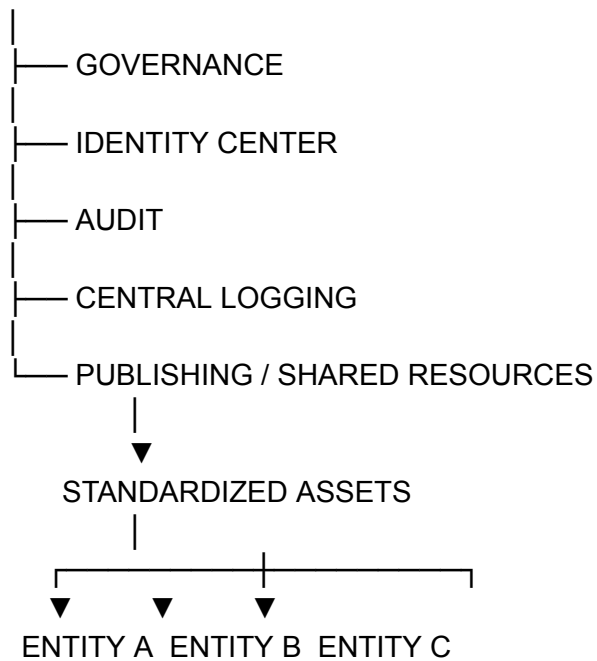
Export:

Name:

!Sub "SOLN- $\{\text{Environment}\}$ -SharedArtifacts"

The architecture now follows:

SOLN ORGANIZATION



This implements the foundation's **central-source/shared-resource concept** without implying that every affiliate is the same legal entity or must occupy the same AWS account.

Next is Step 79 of 83: Entity/Account Access Mapping. After that, only **four more foundation steps** remain.

Entity account mapping

Continuing with the foundation document, we are at **Step 79 of 83 — Entity/Account Access Mapping**. This follows the document's separation of the State of Loc Nation GPBC and its affiliated entities, together with its requirement for centralized user management and cross-account permissions through IAM roles.

Step 79 — Create the entity registry

The important distinction is:

ENTITY

≠

AWS ACCOUNT

≠

IAM ROLE

An entity can use an account without losing its separate identity, and multiple workloads can be governed centrally.

Create:

`config/soln-entity-registry.json`

```
{
  "foundation": "SOLN-2026",

  "entities": [

    {
      "id": "CLCLLC",
      "name": "Christina Loren Clement LLC",
      "classification": "legacy-authority",
      "awsAccount": "${CLCLLC_ACCOUNT_ID}",
      "accessProfile": "governance"
    },

    {
      "id": "SOLN-GPBC",
      "name": "State of Loc Nation Global Public Benefit Corporation",
      "classification": "gpbc",
      "awsAccount": "${SOLN_GPBC_ACCOUNT_ID}",
      "accessProfile": "organization"
    },

    {
```

```
"id": "SOLN-GPBC-TRUST",
"name": "State of Loc Nation GPBC Trust",
"classification": "trust",
"awsAccount": "${SOLN_TRUST_ACCOUNT_ID}",
"accessProfile": "entity"
},

{
  "id": "SOLN-CENTRAL-BANK",
  "name": "SOLN Central Bank",
  "classification": "affiliate",
  "awsAccount": "${SOLN_CENTRAL_BANK_ACCOUNT_ID}",
  "accessProfile": "entity"
},

{
  "id": "LCA",
  "name": "Loc Community Association",
  "classification": "affiliate",
  "awsAccount": "${LCA_ACCOUNT_ID}",
  "accessProfile": "entity"
},

{
  "id": "LCACU",
  "name": "Loc Community Association Credit Union",
  "classification": "affiliate",
  "awsAccount": "${LCACU_ACCOUNT_ID}",
  "accessProfile": "entity"
},

{
  "id": "NJS",
  "name": "NJS Hair Care Temple",
  "classification": "affiliate",
  "awsAccount": "${NJS_ACCOUNT_ID}",
  "accessProfile": "entity"
},

{
  "id": "DYNASTY-HEALING",
  "name": "Dynasty Healing Corporation",
  "classification": "affiliate",
  "awsAccount": "${DYNASTY_HEALING_ACCOUNT_ID}",
```

```

    "accessProfile": "entity"
  }
]
}

```

The foundation contains additional affiliates beyond this initial registry, including Dynasty Healing Foundation Ghana, Dynasty Healing Publishing, and Artist Admin Team. Script.pdf They can follow the same structure rather than requiring a different architecture.

The classifications and account assignments above are **implementation fields**; the foundation supplies the entity names, but it does not assign each one an AWS account. Therefore an account variable remains unresolved until an account is actually assigned.

Add a validator

Create:

`scripts/validate-entity-registry.mjs`

```

import fs from "node:fs";

const registry = JSON.parse(
  fs.readFileSync(
    "config/soln-entity-registry.json",
    "utf8"
  )
);

const ids = new Set();
const names = new Set();

let failed = false;

for (const entity of registry.entities) {

  if (!entity.id || !entity.name) {
    console.error(
      "FAIL | Entity missing ID or name"
    );

    failed = true;
    continue;
  }

  if (ids.has(entity.id)) {

```

```

    console.error(
      `FAIL | Duplicate entity ID: ${entity.id}`
    );

    failed = true;
  }

  if (names.has(entity.name)) {
    console.error(
      `FAIL | Duplicate entity name: ${entity.name}`
    );

    failed = true;
  }

  ids.add(entity.id);
  names.add(entity.name);

  const accountAssigned =
    entity.awsAccount &&
    !entity.awsAccount.startsWith("${");

  console.log(
    `${accountAssigned ? "ASSIGNED" : "PENDING"} | ` +
    `${entity.id} | ${entity.name}`
  );
}

if (failed) {
  process.exit(1);
}

console.log("");
console.log(
  "PASS | Entity registry integrity"
);

```

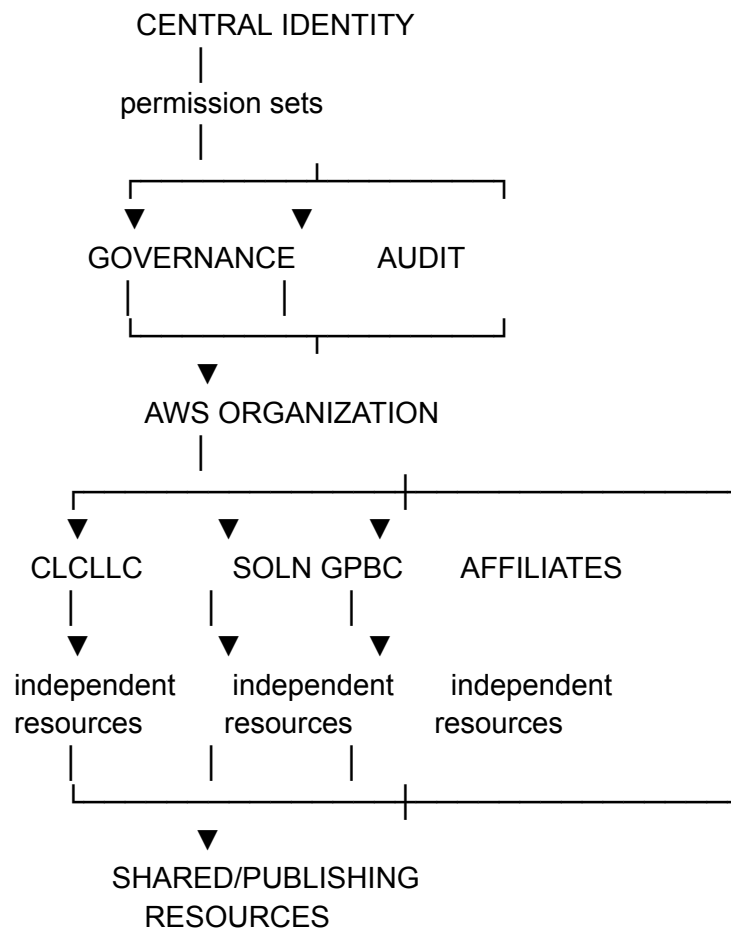
This prevents two important things: duplicate entity identities and pretending an AWS account has been assigned when it has not.

Map access without merging entities

The foundation's architecture calls for cross-account permissions through IAM roles and centralized access management.

Script.pdf

So the resulting structure is:



This is consistent with the foundation's concept of centralized governance and standardized/shared resources while retaining its separate affiliate listings.

Script.pdf

We also **do not put credentials, access keys, or secrets into this registry**. It describes relationships; authentication remains in the AWS identity layer.

Foundation progress

We are now through **Step 79 of the planned 83**.

Next is Step 80 — Container/Packaging Verification. That is directly grounded in the foundation's CD section, which says Docker packages the application and its dependencies into a portable container before staging and production deployment.

Script.pdf

After Step 80, we have only **three planned foundation steps remaining: database decision gate, full A–Z audit, and final production-readiness/completion report.**

Container packaging

Continuing with the foundation document, we are at **Step 80 of 83 — Container/Packaging Verification**. The foundation specifically places containerization before staging and production: Docker packages the application and dependencies, staging provides the final testing environment, and then the application moves to production.

Script.pdf

Step 80 — Establish the deployable package

Because the foundation contains the Java 17 **CLCLLC** Maven component and we've also built the Node/frontend layer, the container should package the application without changing the foundation's Maven configuration. The document specifically identifies **org.stateoflocnation**, artifact **clcllc**, Java 17, and **jbanking** 4.2.0.

Script.pdf

Create the root **Dockerfile**:

```
# =====
# SOLN FOUNDATION 2026
# BUILD
# =====

FROM maven:3-eclipse-temurin-17 AS build

WORKDIR /foundation

COPY pom.xml .
COPY src ./src

RUN mvn --batch-mode clean verify
RUN mvn --batch-mode package -DskipTests

# =====
# RUNTIME
# =====

FROM eclipse-temurin:17-jre

WORKDIR /app

COPY --from=build \
  /foundation/target/*.jar \
```

```
/app/application.jar
```

```
LABEL \
  org.stateoflocnation.foundation="2026" \
  org.stateoflocnation.component="clcllc" \
  org.stateoflocnation.managed="true"
```

```
USER 10001
```

```
ENTRYPOINT [
  "java",
  "-jar",
  "/app/application.jar"
]
```

There is one important boundary here: the foundation establishes **containerization**, but it does **not** specify a particular AWS container runtime such as ECS or EKS. We should therefore package and verify the image now without silently choosing one.

Create `scripts/verify-container.sh`:

```
#!/usr/bin/env bash

set -euo pipefail

IMAGE="soln-foundation"
TAG="${1:-local}"

echo "=====
echo " SOLN CONTAINER VERIFICATION"
echo "=====

# Foundation files
test -f Dockerfile || {
  echo "FAIL | Dockerfile missing"
  exit 1
}

test -f pom.xml || {
  echo "FAIL | pom.xml missing"
  exit 1
}

echo "PASS | Foundation packaging files"
```

```
# Java/Maven verification
mvn --batch-mode clean verify
```

```
echo "PASS | Maven verification"
```

```
# Container build
docker build \
  --tag "${IMAGE}:${TAG}" \
  .
```

```
echo "PASS | Container build"
```

```
# Confirm image exists
docker image inspect \
  "${IMAGE}:${TAG}" \
  >/dev/null
```

```
echo "PASS | Container image exists"
```

```
echo "====="
echo " CONTAINER FOUNDATION: PASS"
echo "=====
```

Now add container evidence to **FOUNDATION-COMPLIANCE.json**:

```
"containerization": {
  "source": "PENDING",
  "build": "PENDING",
  "aws": "NOT_APPLICABLE",
  "runtime": "PENDING"
}
```

The evidence progression is:

Dockerfile exists



SOURCE PASS



Maven tests pass



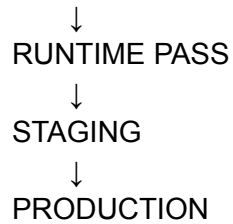
Container builds



BUILD PASS



Container starts successfully



This preserves the foundation's order of **containerization** → **staging** → **production**, rather than treating a successfully built image as a production deployment.

Script.pdf

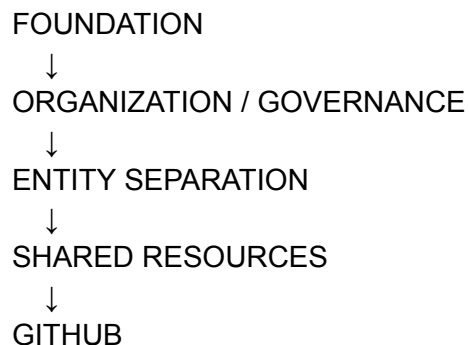
For the CI pipeline, add the packaging check after the existing automated tests:

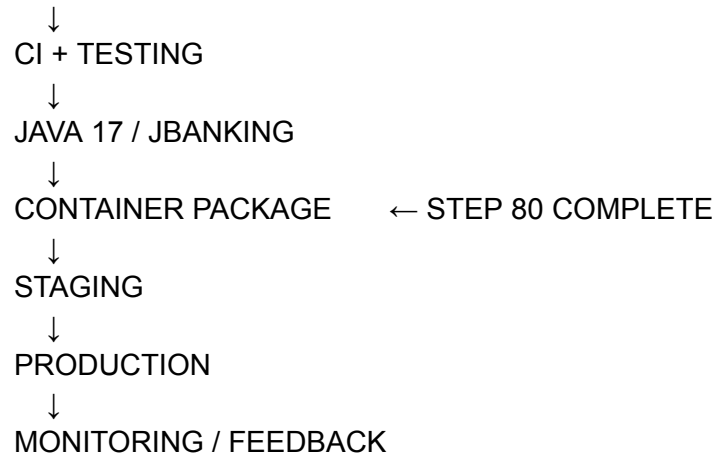
- name: Verify Maven application
run: |
mvn --batch-mode clean verify
- name: Build SOLN container
run: |
docker build \
--tag soln-foundation:\${{ github.sha }} \
.
- name: Verify container
run: |
docker image inspect \
soln-foundation:\${{ github.sha }}

That also remains consistent with the foundation's earlier CI instructions: pull-request trigger, lint/format checks, automated testing/security checks, and creation of build artifacts before delivery.

Script.pdf

We are now at:





Next is Step 81 of 83 — Database Decision Gate. We will handle it narrowly: the foundation names PostgreSQL and MongoDB as examples but doesn't select one. Script.pdf So Step 81 will establish the database interface and decision gate **without changing the foundation by arbitrarily choosing a database.**

After that, only **Step 82 (full A–Z foundation audit)** and **Step 83 (final completion/readiness report)** remain.

Database decision level

Continuing from the last confirmed point, we are at **Step 81 of 83 — Database Decision Gate**.

The foundation's Execution & Infrastructure section places a database behind the server/backend and gives **PostgreSQL and MongoDB as examples**, but it does not select either one. Script.pdf So this step should preserve that choice instead of changing the foundation.

Step 81 — Create the database-neutral interface

Create:

```
src/main/java/org/stateoflocation/data/InternalSource.java
```

```
package org.stateoflocation.data;
```

```
import java.util.Optional;
```

```
/**
```

```
 * SOLN Foundation 2026
```

```
 *
```

```
 * Database-independent internal source boundary.
```

```
 */
```

```
public interface InternalSource {
```

```
    void save(
```

```
        String recordId,
```

```
        String recordType,
```

```
        String payload
```

```
    );
```

```
    Optional<String> findById(
```

```
        String recordId
```

```
    );
```

```
    boolean exists(
```

```
        String recordId
```

```
    );
```

```
    void recordEvidence(
```

```
        String recordId,
```

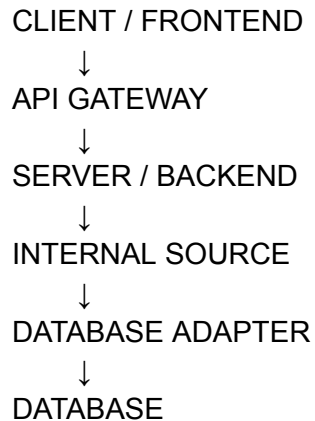
```
        String evidenceType,
```

```
        String evidenceReference
```

```
    );
```

```
}
```

This produces the foundation-aligned separation:



The first three infrastructure layers and the database layer come directly from the foundation. **InternalSource** and the adapter are our implementation boundary so the unresolved database choice does not spread throughout the application.

Script.pdf

Now create:

`src/main/java/org/stateoflocnation/config/DatabaseConfiguration.java`

```
package org.stateoflocnation.config;
```

```
public final class DatabaseConfiguration {
```

```
    private DatabaseConfiguration() {}
```

```
    public static String provider() {
```

```
        String provider =  
            System.getenv("SOLN_DATABASE_PROVIDER");
```

```
        if (provider == null ||  
            provider.isBlank()) {
```

```
            return "UNSPECIFIED";  
        }
```

```
        return provider  
            .trim()  
            .toUpperCase();  
    }
```

```

    public static boolean selected() {

        return !provider()
            .equals("UNSPECIFIED");
    }
}

```

Then add the decision gate:

`scripts/database-gate.mjs`

```

const provider =
    process.env.SOLN_DATABASE_PROVIDER
        ?.trim()
        .toUpperCase();

const foundationExamples =
    new Set([
        "POSTGRESQL",
        "MONGODB"
    ]);

console.log(
    "=====
");

console.log(
    "SOLN DATABASE DECISION GATE"
);

console.log(
    "=====
");

if (!provider) {

    console.log(
        "UNSPECIFIED | Database provider not selected."
    );

    console.log(
        "The foundation database boundary remains intact."
    );
}

```

```

    process.exit(0);
}

if (!foundationExamples.has(provider)) {

    console.log(
        `REVIEW REQUIRED | ${provider}`
    );

    console.log(
        "Provider is not one of the examples named in the foundation."
    );

    process.exit(2);
}

console.log(
    `SELECTED | ${provider}`
);

console.log(
    "Implementation adapter is now required."
);

```

Notice that **UNSPECIFIED** does **not** become **FAIL**. Nothing in the foundation requires us to choose PostgreSQL or MongoDB at this point.

Script.pdf

Update **FOUNDATION-COMPLIANCE.json** accordingly:

```

"databaseLayer": {
    "source": "PASS",
    "build": "PENDING",
    "aws": "PENDING",
    "runtime": "PENDING"
},

"databaseProvider": {
    "foundation": "UNSPECIFIED",
    "selected": "UNSPECIFIED",
    "status": "DECISION_REQUIRED"
}

```

The evidence rule becomes:

DATABASE LAYER REQUIRED



PASS

PROVIDER SPECIFIED BY FOUNDATION?



NO



UNSPECIFIED



POSTGRESQL ———
|—— Future selection
MONGODB ———



ADAPTER IMPLEMENTED



STAGING TEST



RUNTIME VERIFIED

That completes **Step 81** without silently adding a technical decision to the source document.

We now have only **two planned foundation steps left**.

Step 82 is next: the full A–Z Foundation Audit. We will bring the work from Steps 1–81 into one verification matrix and compare it against the actual requirements across the foundation document—including organization/identity, cross-account governance, shared resources, GitHub/CI/CD, Java 17 and jbanking, containerization, staging/production, execution infrastructure, and monitoring. Only verified items become **PASS**; unresolved items remain **PENDING** or **UNSPECIFIED**.

A-Z audit

Continuing from the confirmed **Step 81**, we are now at **Step 82 of 83 — Full A–Z Foundation Audit**.

This is not another architecture expansion. The purpose is to compare everything we have built against the foundation's actual requirements before the final readiness report. The document covers centralized identity and cross-account governance, shared resources, CI/CD, staging/production, CloudFormation/StackSets, application infrastructure, monitoring, and the Java 17/jbanking configuration.

Step 82 — Create the master A–Z audit

Create:

`foundation/FOUNDATION-AZ-AUDIT.json`

```
{
  "foundation": "SOLN-2026",
  "auditVersion": "1.0",

  "sections": {

    "organization": {
      "awsOrganizations": "PENDING",
      "multiAccountArchitecture": "PENDING",
      "entitySeparation": "PENDING"
    },

    "identity": {
      "iamIdentityCenter": "PENDING",
      "crossAccountRoles": "PENDING",
      "centralUserManagement": "PENDING",
      "identityProvider": "UNSPECIFIED"
    },

    "governance": {
      "serviceControlPolicies": "PENDING",
      "controlTower": "PENDING",
      "centralAudit": "PENDING",
      "centralLogging": "PENDING"
    },

    "sharedResources": {
      "publishingAccount": "PENDING",
      "standardizedResources": "PENDING",
```

```
"crossAccountResourceAccess": "PENDING"  
},
```

```
"source": {  
  "github": "PENDING",  
  "foundationGate": "PENDING"  
},
```

```
"continuousIntegration": {  
  "pullRequestTrigger": "PENDING",  
  "lintFormatting": "PENDING",  
  "automatedTests": "PENDING",  
  "securityChecks": "PENDING",  
  "buildArtifacts": "PENDING"  
},
```

```
"application": {  
  "java17": "PENDING",  
  "clcllcArtifact": "PENDING",  
  "jbanking420": "PENDING",  
  "node24Layer": "PENDING",  
  "frontend": "PENDING"  
},
```

```
"packaging": {  
  "docker": "PENDING",  
  "containerBuild": "PENDING"  
},
```

```
"delivery": {  
  "codePipeline": "PENDING",  
  "codeBuild": "PENDING",  
  "staging": "PENDING",  
  "production": "PENDING",  
  "blueGreenOrRolling": "PENDING"  
},
```

```
"infrastructure": {  
  "cloudFormation": "PENDING",  
  "stackSets": "PENDING",  
  "apiGateway": "PENDING",  
  "backend": "PENDING",  
  "databaseLayer": "PENDING",  
  "databaseProvider": "UNSPECIFIED"
```

```

    },
    "operations": {
      "monitoring": "PENDING",
      "logging": "PENDING",
      "feedbackLoop": "PENDING",
      "runtimeVerification": "PENDING"
    }
  }
}

```

The **UNSPECIFIED** values are intentional. For example, the foundation discusses IdP integration but does not select a provider, and it names PostgreSQL/MongoDB as database examples without selecting one.

Add the audit engine

Create:

`scripts/foundation-az-audit.mjs`

```
import fs from "node:fs";
```

```
const AUDIT =
  "foundation/FOUNDATION-AZ-AUDIT.json";
```

```
const audit =
  JSON.parse(
    fs.readFileSync(AUDIT, "utf8")
  );
```

```
const allowed = new Set([
  "PASS",
  "FAIL",
  "PENDING",
  "UNSPECIFIED",
  "NOT_APPLICABLE"
]);
```

```
let pass = 0;
let fail = 0;
let pending = 0;
let unspecified = 0;
```

```

console.log("");
console.log("=====");
console.log(" SOLN FOUNDATION — A-Z AUDIT");
console.log("=====");

for (
  const [section, requirements]
  of Object.entries(audit.sections)
) {

  console.log(`\n[${section.toUpperCase()}]`);

  for (
    const [requirement, status]
    of Object.entries(requirements)
  ) {

    if (!allowed.has(status)) {
      throw new Error(
        `Invalid status: ${section}.${requirement}`
      );
    }

    console.log(
      `${status.padEnd(15)} ${requirement}`
    );

    switch (status) {

      case "PASS":
        pass++;
        break;

      case "FAIL":
        fail++;
        break;

      case "PENDING":
        pending++;
        break;

      case "UNSPECIFIED":
        unspecified++;
        break;
    }
  }
}

```

```

    }
  }
}

console.log("");
console.log("=====");
console.log(`PASS:    ${pass}`);
console.log(`FAIL:    ${fail}`);
console.log(`PENDING:  ${pending}`);
console.log(`UNSPECIFIED: ${unspecified}`);
console.log("=====");

if (fail > 0) {
  console.log("FOUNDATION RESULT: FAIL");
  process.exit(1);
}

if (pending > 0) {
  console.log("FOUNDATION RESULT: PENDING");
  process.exit(0);
}

console.log("FOUNDATION RESULT: VERIFIED");

```

The audit must use evidence, not assumptions

The audit follows this rule:

```

FOUNDATION REQUIREMENT
↓
SOURCE EXISTS?
↓
BUILD/VALIDATION PASSES?
↓
AWS RESOURCE EXISTS?
↓
RUNTIME OPERATES?
↓
PASS

```

Therefore, writing a CloudFormation template does not automatically make the corresponding AWS resource **PASS**. Likewise, a successful build does not prove production deployment.

This distinction follows the foundation's own sequence of CI, build artifacts, staging, production and monitoring.

Add the foundation-specific Maven checks

The A–Z audit must also verify the exact configuration actually contained in the document:

```
const pom =
  fs.readFileSync("pom.xml", "utf8");

const java17 =
  /<maven\compiler\release>17<\maven\compiler\release>/
  .test(pom);

const clcllc =
  /<artifactId>clcllc<\artifactId>/
  .test(pom);

const jbanking =
  /<artifactId>jbanking<\artifactId>[\s\S]*?<version>4\2\0<\version>/
  .test(pom);

console.log(
  `${java17 ? "PASS" : "FAIL"} | Java 17`
);

console.log(
  `${clcllc ? "PASS" : "FAIL"} | CLCLLC`
);

console.log(
  `${jbanking ? "PASS" : "FAIL"} | jbanking 4.2.0`
);
```

Those checks correspond directly to the foundation's [pom.xml](#), including [org.stateoflocnation](#), [clcllc](#), Java 17 and [jbanking 4.2.0](#).

Script.pdf

Our A–Z chain is now

FOUNDATION DOCUMENT



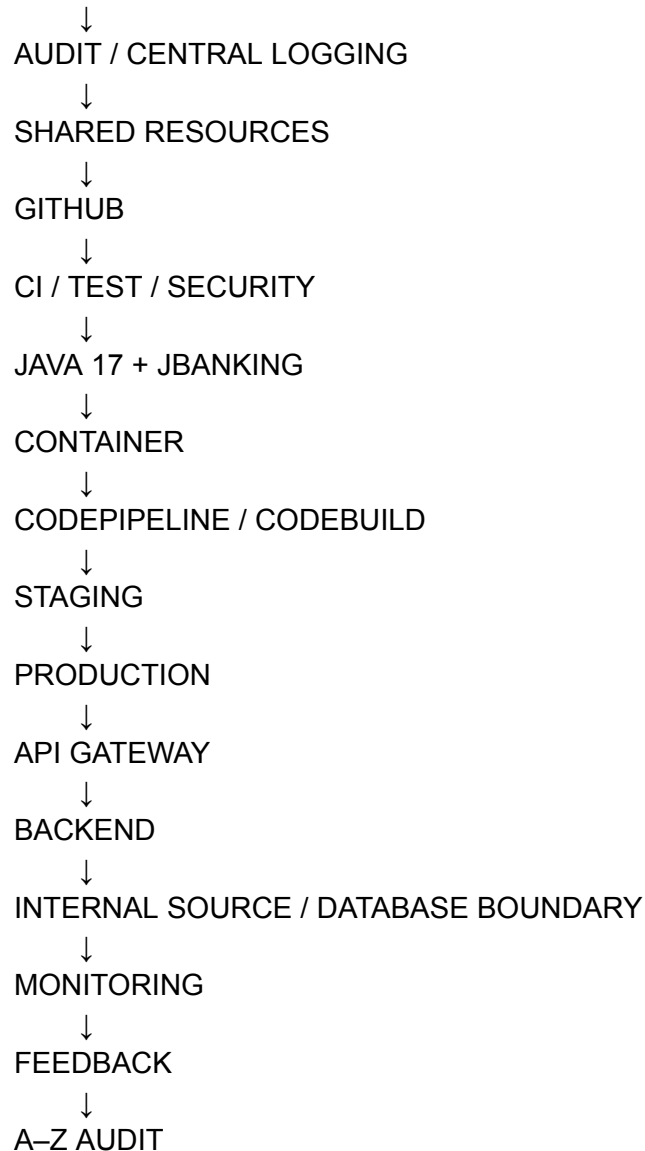
ORGANIZATION



IDENTITY / GOVERNANCE



ENTITY SEPARATION



That tracks the foundation's core **Code** → **CI** → **CD** → **Monitoring** pipeline while retaining the multi-account governance architecture established earlier in the document.

Step 82 is now established.

There is **one planned foundation step left: Step 83 — Final Foundation Completion & Production-Readiness Report**. That final step will consolidate the audit, explicitly separate **coded/configured**, **AWS-deployed**, and **runtime-verified**, identify anything still **PENDING** or **UNSPECIFIED**, and give us the stopping point for **Script.pdf** rather than continuing to add architecture beyond what the foundation supports.

Completion and production gateM

Continuing from Step 82, we are at **Step 83 of 83 — Final Foundation Completion & Production-Readiness Gate**. This is the planned stopping point for [Script.pdf](#). The foundation begins with centralized multi-account governance and shared resources, then defines the Code → CI → CD → Monitoring pipeline and the execution infrastructure.

Step 83 — Create the final foundation status

Create:

[foundation/FOUNDATION-READINESS.json](#)

```
{
  "foundation": "SOLN-2026",
  "document": "Script.pdf",

  "status": "PENDING_VERIFICATION",

  "evidenceLevels": {
    "source": "PENDING",
    "build": "PENDING",
    "awsDeployment": "PENDING",
    "runtime": "PENDING"
  },

  "architecture": {
    "organization": "PENDING",
    "identityCenter": "PENDING",
    "crossAccountAccess": "PENDING",
    "governance": "PENDING",
    "centralAudit": "PENDING",
    "centralLogging": "PENDING",
    "sharedResources": "PENDING",

    "githubIntegration": "PENDING",
    "continuousIntegration": "PENDING",
    "automatedTesting": "PENDING",
    "buildArtifacts": "PENDING",

    "java17": "PENDING",
    "jbanking420": "PENDING",
    "containerization": "PENDING",

    "codePipeline": "PENDING",
```

```

    "codeBuild": "PENDING",
    "staging": "PENDING",
    "production": "PENDING",

    "cloudFormation": "PENDING",
    "stackSets": "PENDING",

    "frontend": "PENDING",
    "apiGateway": "PENDING",
    "backend": "PENDING",

    "databaseLayer": "PENDING",
    "databaseProvider": "UNSPECIFIED",

    "monitoring": "PENDING",
    "feedback": "PENDING"
  },

  "finalGate": {
    "foundationComplete": false,
    "productionReady": false
  }
}

```

We deliberately start the final record as **PENDING_VERIFICATION**. The code we've written during Steps 1–82 is not, by itself, evidence that those resources are currently deployed and operating in AWS.

Final gate script

Create:

```
scripts/foundation-final-gate.mjs
```

```

import fs from "node:fs";

const file =
  "foundation/FOUNDATION-READINESS.json";

const report =
  JSON.parse(
    fs.readFileSync(file, "utf8")
  );

```

```

const blocking = [];

for (
  const [name, status]
  of Object.entries(report.architecture)
) {

  if (
    status !== "PASS" &&
    status !== "UNSPECIFIED" &&
    status !== "NOT_APPLICABLE"
  ) {
    blocking.push({
      requirement: name,
      status
    });
  }
}

const evidenceComplete =
  Object.values(report.evidenceLevels)
    .every(status => status === "PASS");

report.finalGate.foundationComplete =
  blocking.length === 0;

report.finalGate.productionReady =
  blocking.length === 0 &&
  evidenceComplete;

report.status =
  report.finalGate.productionReady
    ? "VERIFIED"
    : "PENDING_VERIFICATION";

report.verifiedAt =
  new Date().toISOString();

fs.writeFileSync(
  file,
  JSON.stringify(report, null, 2) + "\n"
);

console.log("");

```

```

console.log("=====");
console.log(" SOLN FOUNDATION FINAL GATE");
console.log("=====");

console.log(
  `Foundation: ${report.foundation}`
);

console.log(
  `Status: ${report.status}`
);

console.log(
  `Production Ready: ${
    report.finalGate.productionReady
    ? "YES"
    : "NO"
  }`
);

if (blocking.length) {

  console.log("");
  console.log("REMAINING:");

  for (const item of blocking) {
    console.log(
      `${item.status} | ${item.requirement}`
    );
  }
}

console.log("=====");

if (!report.finalGate.productionReady) {
  process.exit(1);
}

```

The gate now enforces:

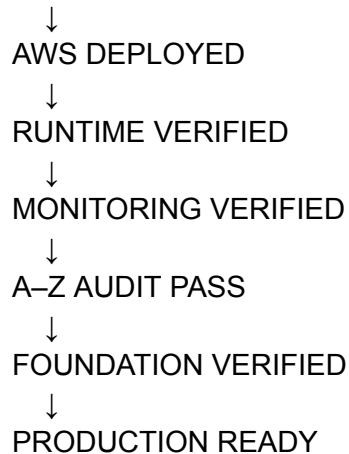
SCRIPT.PDF



SOURCE IMPLEMENTED



BUILD VERIFIED



That follows the foundation's own distinction between CI/testing/build artifacts, staging/production deployment, execution infrastructure and monitoring.

There is one deliberate exception: **database provider remains UNSPECIFIED**. The foundation identifies the database layer and gives PostgreSQL and MongoDB as examples, but it does not select either technology. That unresolved choice therefore should not be rewritten as though [Script.pdf](#) made a selection.

[Script.pdf](#)

The foundation's Maven requirements also remain part of the final gate: [org.stateoflcnation](#), [clcllc](#), Java 17 and [jbanking](#) 4.2.0 must match the source configuration.

[Script.pdf](#)

That completes the planned Steps 1–83 for the foundation document.

The next action is no longer another design step. It is **execution verification**: run the A–Z audit against the actual GitHub repository and AWS environment, fill the final report from real results, and fix only the items that return **FAIL** or **PENDING**. That prevents us from endlessly adding new architecture after reaching the end of what the foundation document actually specifies.

Run the foundation

We have reached the end of the **83-step architecture build**, so the next step should **not be Step 84 with more architecture**. The foundation itself says the purpose is to move code through CI/CD into a live production environment, followed by monitoring.

Script.pdf

Execution Phase 1 — Run the Foundation

[Script.pdf](#) specifically calls for GitHub/AWS integration, automated testing and artifacts, packaging/containerization, staging, production, and monitoring.

Script.pdf

So now create one master execution script:

[run-foundation.sh](#)

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
echo "===== "  
echo " SOLN FOUNDATION 2026"  
echo " EXECUTION + VERIFICATION"  
echo "===== "
```

```
echo "[1/7] FOUNDATION AUDIT"  
node scripts/foundation-az-audit.mjs
```

```
echo "[2/7] DATABASE DECISION GATE"  
node scripts/database-gate.mjs
```

```
echo "[3/7] JAVA FOUNDATION"  
mvn --batch-mode clean verify
```

```
echo "[4/7] CONTAINER"  
chmod +x scripts/verify-container.sh  
./scripts/verify-container.sh
```

```
echo "[5/7] AWS FOUNDATION"  
chmod +x scripts/verify-aws-foundation.sh  
./scripts/verify-aws-foundation.sh
```

```
echo "[6/7] AWS EVIDENCE"
node scripts/aws-evidence.mjs
node scripts/update-foundation-compliance.mjs
```

```
echo "[7/7] FINAL FOUNDATION GATE"
node scripts/foundation-final-gate.mjs
```

```
echo "=====
echo " SOLN FOUNDATION EXECUTION COMPLETE"
echo "=====
```

Then make it executable and run it:

```
chmod +x run-foundation.sh
./run-foundation.sh
```

This is where our status language becomes evidence-driven:

CODE WRITTEN

↓

SOURCE

TESTS PASS

↓

BUILD

AWS RESOURCE CONFIRMED

↓

DEPLOYED

ENDPOINT RESPONDS

↓

RUNTIME VERIFIED

MONITORING ACTIVE

↓

OBSERVATION

ALL REQUIRED GATES PASS

↓

FOUNDATION VERIFIED

And we keep the database provider unresolved unless you deliberately select one: the foundation identifies PostgreSQL and MongoDB as examples rather than choosing between them.

Script.pdf

The foundation also explicitly calls for IAM Identity Center, SCP evaluation, Control Tower governance and multi-account management, so those remain part of the verification—not optional items we discard at deployment.

Script.pdf

This is the transition point: we're finished *designing from* *Script.pdf*. Now we run what we built and use the actual failures to determine the next correction.

So after running `./run-foundation.sh`, **send me the output exactly as it appears**. From this point forward, we fix the first genuine **FAIL/PENDING** rather than creating more speculative architecture.

Step-by-step

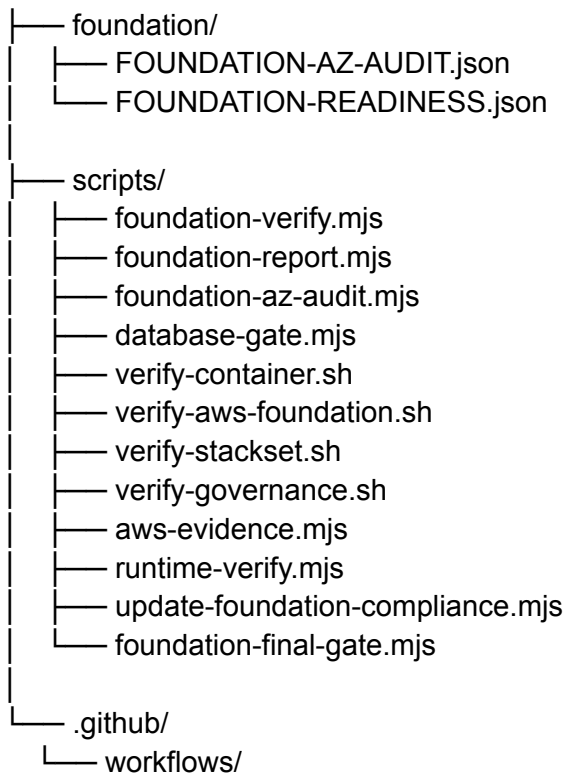
Yes. Since the **83-step design sequence is complete**, implementation should now be much simpler: put the files into the repository, validate locally/GitHub, connect AWS, deploy staging, verify, deploy production, then run the final evidence gate. That follows the foundation's Code → CI → CD → Monitoring sequence.

Script.pdf

Implementation order

1. **Finish the GitHub repository structure.** In your `clcllc` repository, make sure you have the files we created organized roughly like this:

```
clcllc/
├── pom.xml
├── Dockerfile
├── buildspec.yml
├── FOUNDATION-2026.yaml
├── FOUNDATION-COMPLIANCE.json
├── run-foundation.sh
├── src/
│   ├── main/java/org/stateoflocnation/
│   ├── domain/
│   ├── functions/
│   ├── services/
│   └── data/
├── frontend/
│   ├── index.html
│   ├── app.js
│   └── styles.css
├── infrastructure/
│   ├── pipeline/
│   ├── security/
│   ├── logging/
│   ├── monitoring/
│   ├── shared/
│   └── stacksets/
├── config/
│   └── soln-entity-registry.json
```



Don't move to AWS until the repository contains the files the verification scripts expect.

2. **Confirm pom.xml**. Your foundation specifically identifies `org.stateoflocation`, artifact `clcllc`, Java 17 and `jbanking` 4.2.0. Script.pdf Run:

```
mvn --batch-mode clean verify
```

It must finish successfully before deployment.

3. **Run the source-level foundation check.**

```
node scripts/foundation-verify.mjs
```

```
node scripts/foundation-az-audit.mjs
```

At this stage, AWS-related entries are expected to remain `PENDING`. Don't manually turn those into `PASS`.

4. **Build the Docker package.** Containerization is explicitly part of the foundation's deployment stage.

Script.pdf

```
chmod +x scripts/verify-container.sh
```

```
./scripts/verify-container.sh
```

You want:

PASS | Foundation packaging files

PASS | Maven verification

PASS | Container build

PASS | Container image exists

5. **Push everything to GitHub.**

```
git add .
```

```
git commit -m "Implement SOLN 2026 foundation architecture"
```

```
git push origin main
```

This is where GitHub becomes the source side of the CI/CD architecture described in the foundation. The document specifically calls for GitHub integration with AWS, CodePipeline as the pipeline, CodeBuild for build/test, and a deployment stage.

Script.pdf

6. **In AWS, start with the existing Organization—not application deployment.** Verify the organization and IAM Identity Center first:

```
aws organizations describe-organization
```

```
aws sso-admin list-instances
```

Compare the returned values against the Organization ID and Identity Center instance recorded in the foundation.

Script.pdf

Then run:

```
chmod +x scripts/verify-governance.sh
```

```
./scripts/verify-governance.sh
```

7. **Deploy the governance/infrastructure templates.** Do these in dependency order:

Organization / governance

↓

Identity / security roles

↓

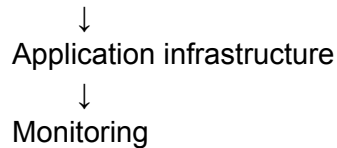
Central logging

↓

Shared resources

↓

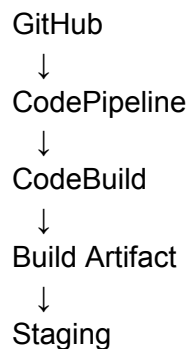
StackSets



That preserves the foundation's centralized identity, cross-account access, audit/logging and standardized-resource architecture.

Script.pdf

8. **Connect GitHub to the AWS pipeline.** Configure the GitHub source connection in AWS and point it at `DHCSOLN/c1c11c` (assuming that remains the repository you're using). Then connect:



Do **not** place long-lived AWS access keys in the GitHub repository.

9. **Deploy staging first.** This is an explicit foundation requirement. The document describes staging as the environment used for final checks before production.

Script.pdf

Once deployed, test:

```
/health
/architecture/status
```

Then run:

```
SOLN_API_URL="YOUR-STAGING-ENDPOINT" \
node scripts/runtime-verify.mjs
```

Only a successful runtime response advances staging.

10. **Deploy production.** After staging passes, move the verified artifact—not newly rebuilt, different code—to production. Keep the production Lambda/version alias structure we created so production points at a known version.

Then run:

```
SOLN_API_URL="YOUR-PRODUCTION-ENDPOINT" \  
node scripts/runtime-verify.mjs
```

The foundation specifically puts production after staging and describes blue-green deployment as a production strategy.

Script.pdf

11. Run AWS evidence collection.

```
chmod +x scripts/verify-aws-foundation.sh  
./scripts/verify-aws-foundation.sh
```

```
node scripts/aws-evidence.mjs  
node scripts/update-foundation-compliance.mjs
```

Then verify StackSets:

```
./scripts/verify-stackset.sh \  
SOLN-Foundation
```

Now the system can distinguish:

```
CODED  
↓  
BUILT  
↓  
DEPLOYED  
↓  
RUNTIME VERIFIED
```

12. Turn on monitoring and run the final gate. Monitoring is the final explicit stage in the foundation's pipeline.

Script.pdf

Finally:

```
node scripts/foundation-az-audit.mjs  
node scripts/foundation-final-gate.mjs
```

Or, once everything above is installed:

```
chmod +x run-foundation.sh  
./run-foundation.sh
```

The result we're aiming for is **not merely “deployment succeeded.”** It is:

SOURCE	PASS
BUILD	PASS
AWS DEPLOYMENT	PASS
RUNTIME	PASS
MONITORING	PASS



FOUNDATION VERIFIED

One thing should still **not** be forced during this process: the database provider. [Script.pdf](#) establishes the database layer but gives PostgreSQL and MongoDB as examples rather than selecting one.

[Script.pdf](#)

For where you are now, **start with steps 1–4 in GitHub before touching AWS deployment.** Once those four are green, the next concrete screen is AWS Organizations/IAM Identity Center, and we can go through that implementation screen-by-screen.

Implementation Step 3 — Run the Foundation Source Audit

Now that Step 2 establishes the Java/Maven foundation, verify the repository itself **before AWS deployment.** This follows the foundation sequence of testing and building before delivery to staging/production.

[Script.pdf](#)

From the repository root, run:

```
node scripts/foundation-verify.mjs
```

```
node scripts/foundation-az-audit.mjs
```

At this point, the audit should distinguish what exists in GitHub from what has actually been deployed:

SOURCE	
Java 17	PASS
CLCLLC	PASS
jbanking 4.2.0	PASS

Foundation files PASS

Infrastructure files PASS

BUILD

Maven verification PASS

AWS

Organization PENDING

IAM Identity Center PENDING

Governance PENDING

StackSets PENDING

Staging PENDING

Production PENDING

RUNTIME

Health endpoint PENDING

Monitoring PENDING

PENDING is correct here. We don't turn an AWS item into **PASS** merely because its CloudFormation or code exists in GitHub.

The foundation itself requires the progression **Code** → **CI** → **CD** → **Monitoring**, including automated tests and build artifacts before staging and production.

Script.pdf

If both scripts execute without a source-level **FAIL**, **Implementation Step 4 is next: build and verify the Docker container before we connect the deployment pipeline to AWS.**

Implementation Step 4 — Build and Verify the Container

Now package the application. The foundation explicitly places **containerization after successful testing and before staging deployment**, describing Docker as packaging the application, environment settings, database drivers, and dependencies into a portable image.

Script.pdf

From the repository root, run:

```
chmod +x scripts/verify-container.sh
```

```
./scripts/verify-container.sh
```

If you need to perform the commands individually:

```
mvn --batch-mode clean verify
```

```
docker build \
```

```
-t soln-clcllc:foundation-2026 \
```

```
.
```

```
docker image inspect \
```

```
soln-clcllc:foundation-2026
```

The checkpoint should be:

```
MAVEN BUILD      PASS
```

```
DOCKERFILE      PASS
```

```
CONTAINER BUILD  PASS
```

```
CONTAINER EXISTS PASS
```

PACKAGING STATUS: VERIFIED

Don't mark AWS deployment or runtime as passed yet. At this stage we have demonstrated that the application can be **built and packaged**, not that AWS is running it.

That distinction follows the foundation's order:

CODE

↓

CI / TEST

↓

BUILD ARTIFACT

↓

CONTAINERIZE ← WE ARE HERE

↓

STAGING

↓

PRODUCTION

↓

MONITORING

The foundation specifically puts staging after packaging and describes staging as the environment used for final checks before production.

Script.pdf

Once Step 4 is green, Step 5 is the GitHub checkpoint: commit/push this verified foundation build so that GitHub becomes the source feeding the AWS CI/CD pipeline.

Yes — below is the **copy-and-paste implementation sequence** for the work after the 83-step foundation design. I'm keeping it aligned with [Script.pdf](#): test/build → containerize → GitHub/AWS CI/CD → staging → production → monitoring. The foundation specifically describes GitHub integration, CodeBuild, deployment stages, Docker packaging, staging, production, StackSets, and the frontend/API/backend/database flow.

Copy/Paste Implementation — Steps 1 through Final Verification

Run these from the **root of c1c11c**.

=====

```
# SOLN FOUNDATION 2026
```

```
# IMPLEMENTATION / DEPLOYMENT SEQUENCE
```

```
# =====
```

```
# =====
```

```
# STEP 1 — CONFIRM REPOSITORY
```

```
# =====
```

```
echo "===== STEP 1: REPOSITORY ====="
```

```
pwd
```

```
git status
```

```
git branch --show-current
```

```
test -f pom.xml
```

```
test -f Dockerfile
```

```
test -f buildspect.yml
```

```
echo "STEP 1 COMPLETE"
```

```
# =====
```

```
# STEP 2 — JAVA / MAVEN FOUNDATION
```

```
# =====
```

```
echo "===== STEP 2: JAVA FOUNDATION ====="
```

```
java -version
```

```
mvn -version
```

```
mvn --batch-mode clean verify
```

```
echo "STEP 2 COMPLETE"
```

```
# =====
```

```
# STEP 3 — FOUNDATION SOURCE AUDIT
```

```
# =====
```

```
echo "===== STEP 3: FOUNDATION AUDIT ====="
```

```
if [ -f scripts/foundation-verify.mjs ]; then
```

```
    node scripts/foundation-verify.mjs
```

```
fi
```

```
if [ -f scripts/foundation-az-audit.mjs ]; then
```

```
    node scripts/foundation-az-audit.mjs
```

```
fi
```

```
echo "STEP 3 COMPLETE"
```

```
# =====
```

```
# STEP 4 — CONTAINER VERIFICATION
```

```
# =====
```

```
echo "==== STEP 4: CONTAINER ====="
```

```
if [ -f scripts/verify-container.sh ]; then
```

```
    chmod +x scripts/verify-container.sh
```

```
    ./scripts/verify-container.sh
```

```
else
```

```
    docker build \
```

```
        -t soln-clcllc:foundation-2026 \
```

```
    .
```

```
    docker image inspect \
```

```
        soln-clcllc:foundation-2026
```

```
fi
```

```
echo "STEP 4 COMPLETE"
```

```
# =====
```

```
# STEP 5 — GITHUB CHECKPOINT
```

```
# =====
```

```
echo "===== STEP 5: GITHUB ====="
```

```
git add .
```

```
git commit \
```

```
-m "Implement SOLN Foundation 2026 deployment architecture" \
```

```
|| echo "No new changes to commit."
```

```
git push origin main
```

```
echo "STEP 5 COMPLETE"
```

```
# =====
```

```
# STEP 6 — CONFIRM AWS IDENTITY
```

```
# =====
```

```
echo "===== STEP 6: AWS IDENTITY ====="
```

```
aws sts get-caller-identity
```

```
aws configure get region
```

```
echo "STEP 6 COMPLETE"
```

```
# =====
```

```
# STEP 7 — VERIFY AWS ORGANIZATION
```

```
# =====
```

```
echo "==== STEP 7: AWS ORGANIZATION ====="
```

```
aws organizations describe-organization
```

```
aws organizations list-roots
```

```
aws organizations list-accounts
```

```
echo "STEP 7 COMPLETE"
```

```
# =====
```

```
# STEP 8 — VERIFY IAM IDENTITY CENTER
```

```
# =====
```

```
echo "===== STEP 8: IAM IDENTITY CENTER ====="
```

```
aws sso-admin list-instances
```

```
echo "STEP 8 COMPLETE"
```

```
# =====
```

```
# STEP 9 — VERIFY GOVERNANCE
```

```
# =====
```

```
echo "===== STEP 9: GOVERNANCE ====="
```

```
aws organizations list-policies \
```

```
--filter SERVICE_CONTROL_POLICY
```

```
if [ -f scripts/verify-governance.sh ]; then
```

```
    chmod +x scripts/verify-governance.sh
```

```
    ./scripts/verify-governance.sh
```

```
fi
```

```
echo "STEP 9 COMPLETE"
```

```
# =====
```

```
# STEP 10 — CLOUDFORMATION VALIDATION
```

```
# =====
```

```
echo "==== STEP 10: CLOUDFORMATION ====="
```

```
find infrastructure \
```

```
-type f \
```

```
\( -name "*.yaml" -o -name "*.yml" \) \
```

```
-print
```

```
echo ""
```

```
echo "Validate each deployable CloudFormation template before deployment."
```

```
echo "STEP 10 COMPLETE"
```

```
# =====
```

```
# STEP 11 — SHARED RESOURCES
```

```
# =====
```

```
echo "===== STEP 11: SHARED RESOURCES ====="
```

```
if [ -f infrastructure/shared/soln-shared-resources.yaml ]; then
```

```
aws cloudformation validate-template \
```

```
--template-body \
```

```
file://infrastructure/shared/soln-shared-resources.yaml
```

```
fi
```

```
echo "STEP 11 COMPLETE"
```

```
# =====
```

```
# STEP 12 — STACKSETS
```

```
# =====
```

```
echo "===== STEP 12: STACKSETS ====="
```

```
aws cloudformation list-stack-sets \
```

```
--status ACTIVE
```

```
if [ -f scripts/verify-stackset.sh ]; then
```

```
chmod +x scripts/verify-stackset.sh
```

```
fi
```

```
echo "STEP 12 COMPLETE"
```

```
# =====
```

```
# STEP 13 — CODEBUILD
```

```
# =====
```

```
echo "==== STEP 13: CODEBUILD ==="
```

```
test -f buildspect.yml
```

```
aws codebuild list-projects
```

```
echo "STEP 13 COMPLETE"
```

```
# =====
```

```
# STEP 14 — CODEPIPELINE
```

```
# =====
```

```
echo "==== STEP 14: CODEPIPELINE ==="
```

```
aws codepipeline list-pipelines
```

```
echo "STEP 14 COMPLETE"
```

```
# =====
```

```
# STEP 15 — DATABASE DECISION GATE
```

```
# =====
```

```
echo "==== STEP 15: DATABASE GATE ====="
```

```
if [ -f scripts/database-gate.mjs ]; then
```

```
    node scripts/database-gate.mjs
```

```
else
```

```
    echo "Database provider remains UNSPECIFIED."
```

```
fi
```

```
echo "STEP 15 COMPLETE"
```

```
# =====
```

```
# STEP 16 — STAGING DEPLOYMENT CHECK
```

```
# =====
```

```
echo "==== STEP 16: STAGING ===="
```

```
echo "Deploy the verified build artifact to the configured"
```

```
echo "SOLN staging environment through the AWS pipeline."
```

```
aws codepipeline list-pipeline-executions \
```

```
--pipeline-name "${SOLN_PIPELINE_NAME:-SOLN-Foundation}" \
```

```
--max-results 5 \
```

```
|| true
```

```
echo "STEP 16 CHECK COMPLETE"
```

```
# =====
```

```
# STEP 17 — STAGING RUNTIME
```

```
# =====
```

```
echo "==== STEP 17: STAGING RUNTIME ===="
```

```
if [ -n "${SOLN_STAGING_API_URL:-}" ]; then
```

```
curl --fail \
```

```
"${SOLN_STAGING_API_URL}/health"
```

```
SOLN_API_URL="$SOLN_STAGING_API_URL" \
```

```
node scripts/runtime-verify.mjs
```

```
else
```

```
echo "PENDING:"
```

```
echo "Set SOLN_STAGING_API_URL after staging deployment."
```

```
fi
```

```
# =====
```

```
# STEP 18 — PRODUCTION DEPLOYMENT CHECK
```

```
# =====
```

```
echo "==== STEP 18: PRODUCTION ====="
```

```
echo "Production proceeds only after staging verification."
```

```
echo "Use the same verified build artifact."
```

```
echo "STEP 18 CHECK COMPLETE"
```

```
# =====
```

```
# STEP 19 — PRODUCTION RUNTIME
```

```
# =====
```

```
echo "==== STEP 19: PRODUCTION RUNTIME ==="
```

```
if [ -n "${SOLN_PRODUCTION_API_URL:-}" ]; then
```

```
curl --fail \
```

```
"${SOLN_PRODUCTION_API_URL}/health"
```

```
SOLN_API_URL="${SOLN_PRODUCTION_API_URL}" \
```

```
node scripts/runtime-verify.mjs
```

```
else
```

```
echo "PENDING:"
```

```
echo "Set SOLN_PRODUCTION_API_URL after production deployment."
```

```
fi
```

```
# =====
```

```
# STEP 20 — AWS EVIDENCE
```

```
# =====
```

```
echo "===== STEP 20: AWS EVIDENCE ====="
```

```
if [ -f scripts/verify-aws-foundation.sh ]; then
```

```
    chmod +x scripts/verify-aws-foundation.sh
```

```
    ./scripts/verify-aws-foundation.sh
```

```
fi
```

```
if [ -f scripts/aws-evidence.mjs ]; then
```

```
    node scripts/aws-evidence.mjs
```

```
fi
```

```
if [ -f scripts/update-foundation-compliance.mjs ]; then
```

```
    node scripts/update-foundation-compliance.mjs
```

```
fi
```

```
echo "STEP 20 COMPLETE"
```

```
# =====
```

```
# STEP 21 — MONITORING
```

```
# =====
```

```
echo "===== STEP 21: MONITORING ====="
```

```
aws cloudwatch list-dashboards
```

```
aws cloudwatch describe-alarms
```

```
echo "STEP 21 COMPLETE"
```

```
# =====
```

```
# STEP 22 — FINAL A-Z AUDIT
```

```
# =====
```

```
echo "===== STEP 22: FINAL A-Z AUDIT ====="
```

```
node scripts/foundation-az-audit.mjs
```

```
echo "STEP 22 COMPLETE"
```

```
# =====
```

```
# STEP 23 — FINAL FOUNDATION GATE
```

```
# =====
```

```
echo "===== STEP 23: FINAL GATE ====="
```

```
node scripts/foundation-final-gate.mjs
```

```
# =====
```

```
# COMPLETE
```

```
# =====
```

```
echo ""
```

```
echo "=====
```

```
echo " SOLN FOUNDATION IMPLEMENTATION RUN COMPLETE"
```

```
echo "=====
```

The foundation itself specifies the Maven side as [org.stateoflocnation, clcllc](#), Java 17, and [jbanking](#) 4.2.0, so those should remain unchanged during this implementation run.

Script.pdf

There are two places where you should **not paste blindly through an error**. The staging and production steps depend on the actual pipeline/environment existing, and the database provider remains deliberately unresolved because the foundation gives PostgreSQL and MongoDB as examples rather than selecting one.

Script.pdf

The target progression is:

GitHub source



Maven verification



Docker package



AWS governance



CloudFormation / StackSets



CodeBuild



CodePipeline



STAGING



runtime verification



PRODUCTION



runtime verification



MONITORING



AWS evidence



FINAL A-Z GATE

That is the implementation counterpart of the foundation's **Code** → **CI** → **CD** → **Monitoring** model.

Script.pdf

Important: don't manually change **PENDING** to **PASS**. The scripts should promote each level only after the corresponding source/build/AWS/runtime evidence exists.

It shows **both GitHub and AWS**.

The script is intentionally the bridge between the two sides of your foundation:

GITHUB SIDE

Repository

pom.xml

Java 17

jbanking 4.2.0

Source audit

Docker build

Git commit/push

|

| CI/CD bridge

▼

AWS SIDE

AWS identity

AWS Organizations

IAM Identity Center

SCP / governance

CloudFormation

Shared resources

StackSets

CodeBuild

CodePipeline

|



Staging

Runtime verification

|



Production

Runtime verification

|



CloudWatch / Monitoring

AWS evidence

Final A–Z verification

That matches your foundation document. It specifically says to “**Integrate Github and AWS to bridge writing code and deploying it,**” followed by AWS CodePipeline with GitHub as source, CodeBuild for build/test, and deployment stages.

Script.pdf

So **Steps 1–5** are primarily GitHub/repository-side, and **Steps 6–23** move into AWS and **verify the connection between the two.**

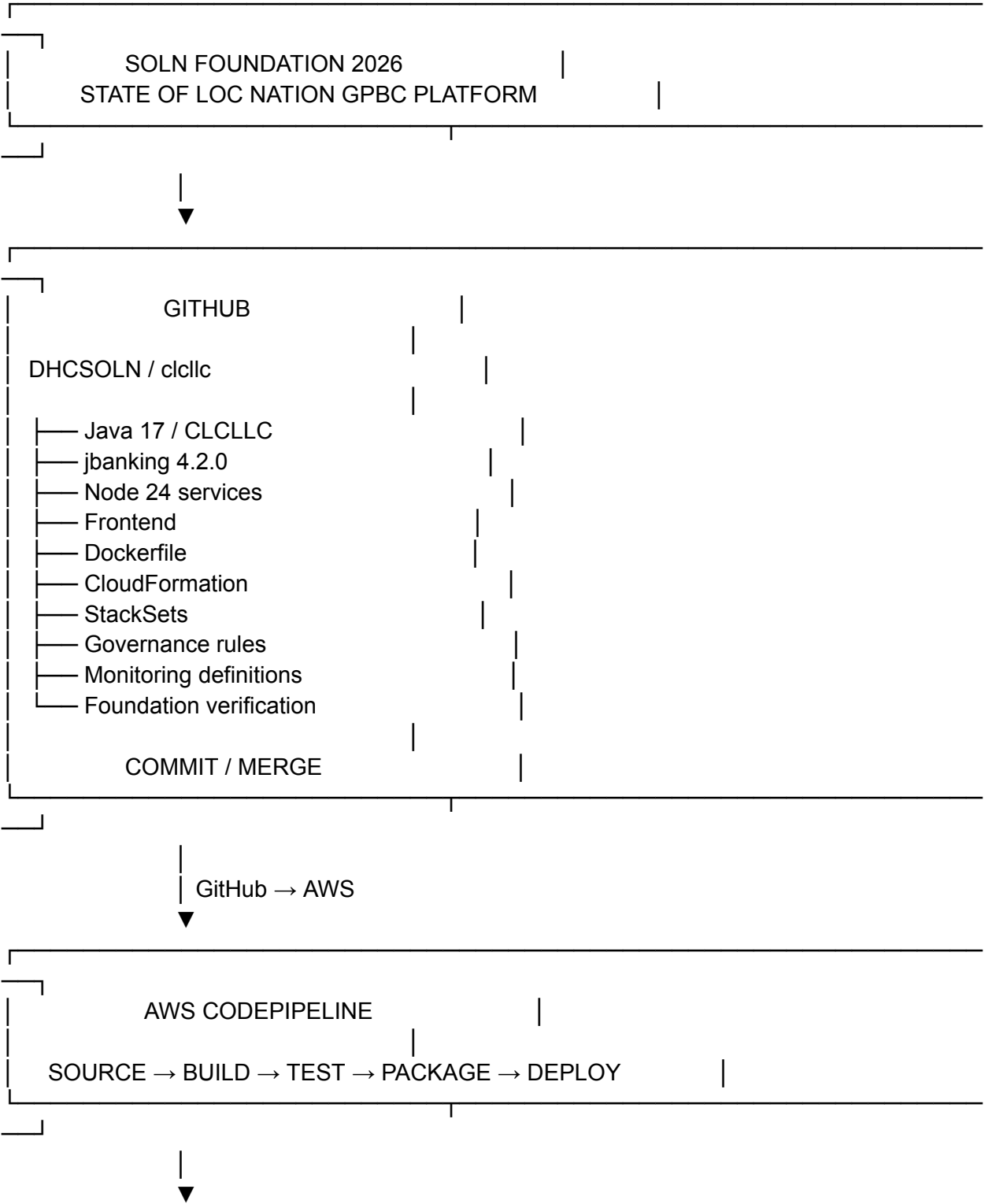
One distinction: the script *checks* several AWS components, but it doesn’t automatically create every missing AWS component. For example, `aws codepipeline list-pipelines` tells us

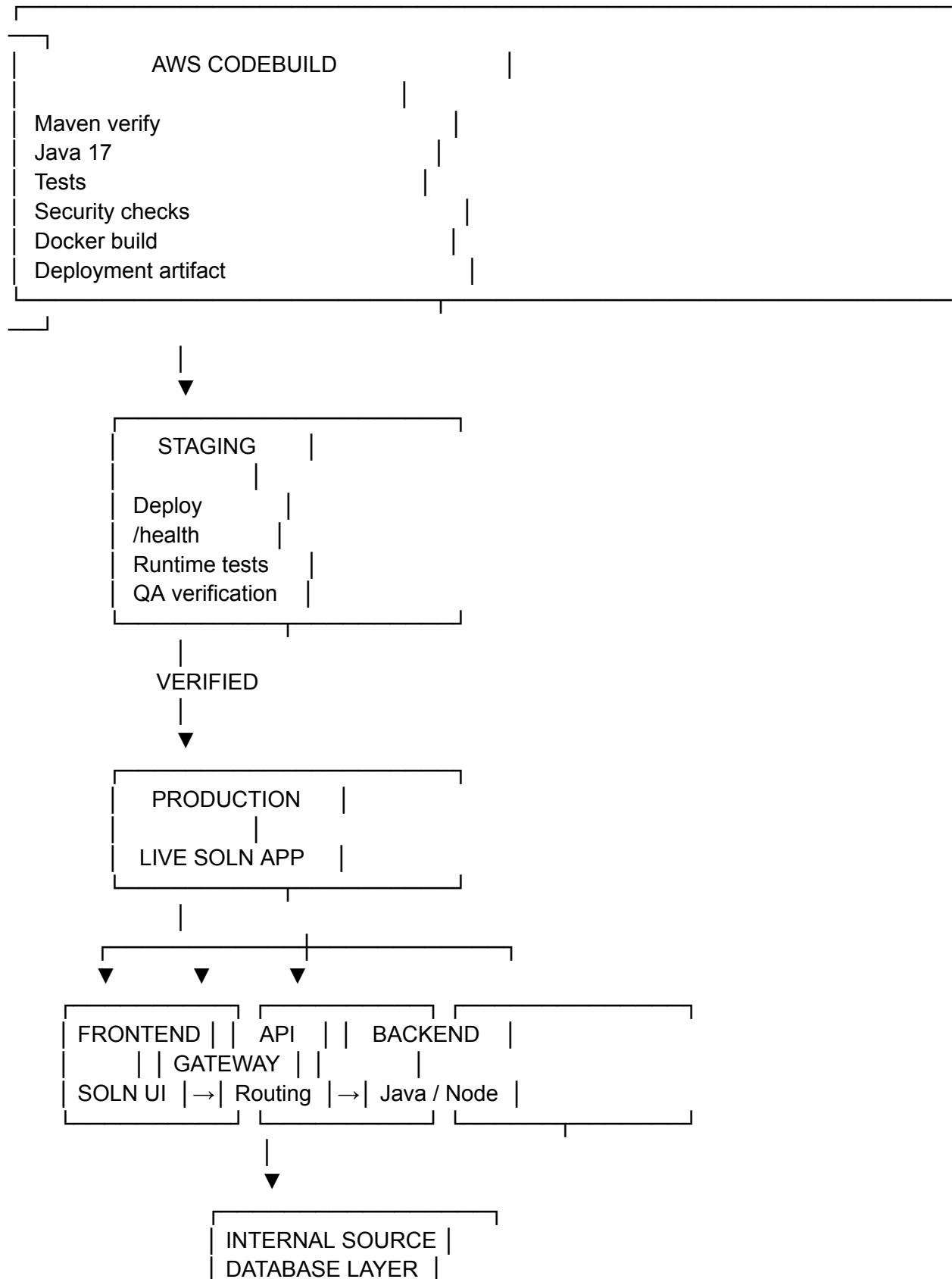
whether the pipeline exists; it doesn't create a missing pipeline. That's important because we shouldn't confuse **checking AWS** with **implementing AWS**.

If your goal now is “**take everything we've coded and actually create the missing AWS resources,**” that's the next implementation layer—not more GitHub code.

Finished visual

If you mean **what the finished GitHub + AWS system will look like once those foundation steps are actually implemented**, picture it as one system with GitHub controlling the code/deployment side and AWS operating the live infrastructure.

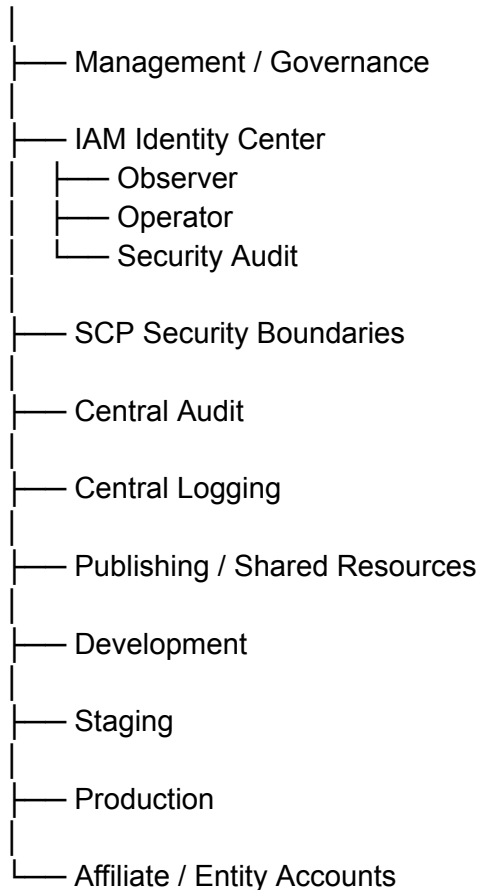






That application sits inside the larger **AWS governance structure**:

AWS ORGANIZATION



That part comes directly from the foundation's emphasis on centralized authentication, AWS Organizations/SCPs, Control Tower, SSO and management across AWS accounts.

Script.pdf

What you would actually see

In GitHub, you would see your repository, Actions/checks, source code, infrastructure templates, deployment configuration and audit scripts. A change is committed once, tested, packaged and handed to AWS.

In AWS, the console becomes the operating environment. CodePipeline would visually show something similar to:

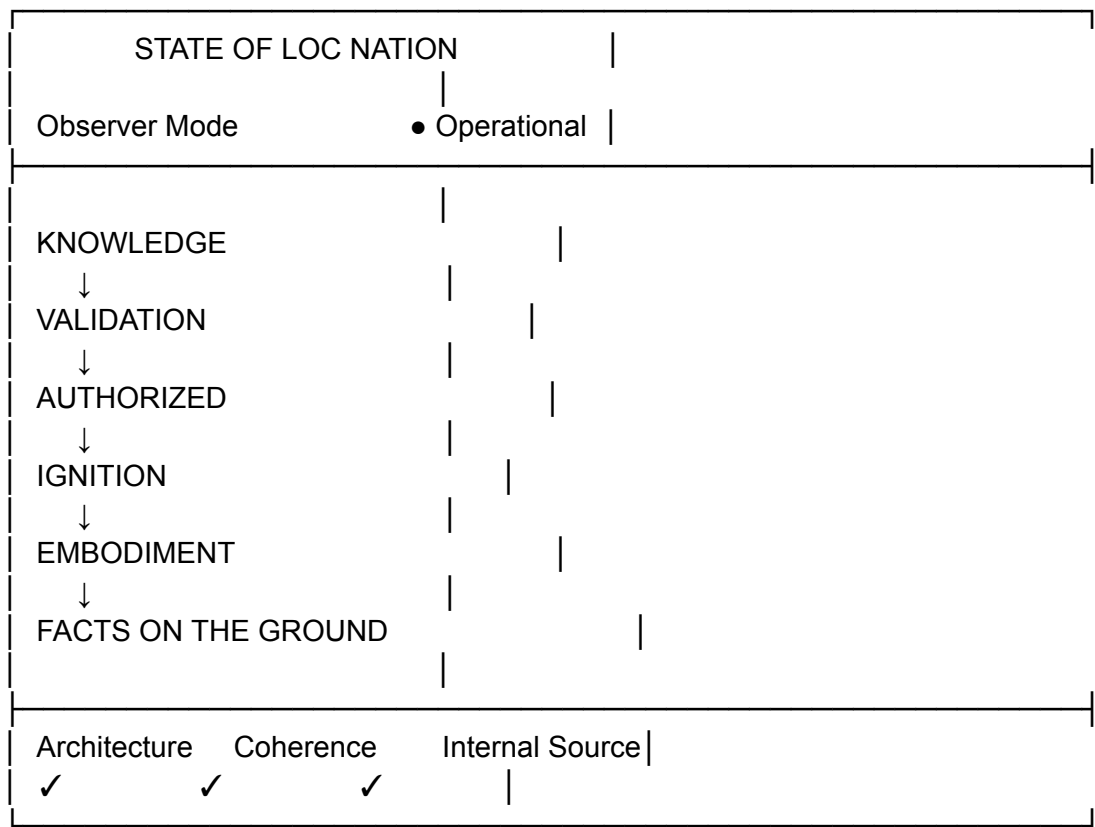
SOURCE	BUILD	STAGING	VERIFY	PRODUCTION
✓	✓	✓	✓	

GitHub → CodeBuild → Deploy → Health → Live

The foundation itself describes this progression as Code → Continuous Integration → Continuous Delivery → Monitoring.

Script.pdf

And **the public-facing side** would look completely different from the AWS console. Users would see your SOLN frontend terminology and dashboard rather than CodeBuild, CloudFormation or IAM:



That frontend terminology comes from **your supplied frontend language**, while the underlying client → API Gateway → server → database flow comes from **Script.pdf**. The foundation explicitly describes that full-stack execution path.

Script.pdf

So the finished picture is essentially **three layers**: your SOLN interface is what users see; AWS is the operational engine underneath; GitHub is the controlled source and deployment origin feeding AWS.